Bounded-Stack DAGs Reduce to Constant Depth

Hans Brende Claude Opus 5.5 (Anthropic)

November 2026

Abstract

A DAG with a spine on $[0, N)$ has the edges $t \to t-1$ and further arcs $u \to p$ with $u \geq p+2$; it is an s-stack DAG if its arcs split into s families, each of pairwise non-crossing arcs with distinct left ends and distinct right ends. We prove that bounded-stack DAGs reduce to constant depth: for all s and $\varepsilon > 0$ there is a D such that every s-stack DAG with a spine has a set of at most εN edges whose deletion leaves no path with more than D arcs. Equivalently, the trajectory cut problem TCP(k) of this series holds for every number k of walks: the walk DAG of any k walks of length T, with or without pauses, has a cut of at most εT edges and depth at most $D(k, \varepsilon)$, independent of T, where $\log \log D(k, \varepsilon) = O(k^2 \log(k/\varepsilon))$ for $\varepsilon \leq \frac{1}{2}$. This settles SR(3), the first open case left by the eighth paper of the series. It also shows that the deep-nesting pairs, which the series regarded as the sharpest candidates for depth-robust walk DAGs, are cut to a depth independent of their nesting depth, so Conjectures A′ and A″ of the fourth and eighth papers are false. For oblivious machines, depth reduction now holds unconditionally, and the series' reduction of a lower bound for linear-time matrix transposition rests on the coding side alone; the same genies prove the eleventh paper's conjecture on code-level depth reduction and, given a native coding bound, exclude every linear-time transposer whose trace carries less than half of the input's information.

The proof builds a random label from three ingredients: a decomposition of each walk into ladders of nested hops, which makes the hops of each walk cheap on average over a random grid (Theorem A); a display of suspended frames, which controls the interaction between walks (view invariance); and random break bits, which keep the display bounded where exact displays are unbounded (Lemma D). The main theorem, every numbered result of its proof and all its corollaries are formalized in Lean 4 with Mathlib, with no axioms beyond `propext`, `Classical.choice` and `Quot.sound`: the corollaries for machines in the frozen machine model of the series, and the refutation of the conjectures with the conjectures stated from the definitions of the fourth and eighth papers. Not formalized are the remarks, which explain why simpler constructions fail or discuss the consequences.

Contributions. Hans Brende chose the problem, fixed the verification standard, and directed the research. Claude Opus 5.5, a large language model developed by Anthropic, developed the mathematics, wrote the Lean formalization, and wrote this paper under Hans Brende's direction.

Contents

1 Introduction **4**
- 1.1 The problem . . . 4
- 1.2 The expected answer, and the actual one . . . 4
- 1.3 Results . . . 4
- 1.4 How the proof works . . . 5
- 1.5 Formal status and how to read the paper . . . 6

1.6 Related work . . . 6
1.7 Organization . . . 7

2 Preliminaries **7**
2.1 Walks, hops and walk DAGs . . . 7
2.2 Cuts, labels, TCP and SR . . . 8
2.3 Owners and prominence . . . 10
2.4 Bands and the random grid . . . 11

3 Ladders: the structure of one walk **12**
3.1 The decomposition by tops . . . 12
3.2 Excursions and structures . . . 14
3.3 Frame systems . . . 15
3.4 The frames of class U . . . 16
3.5 Frame weight . . . 18
3.6 Kept and lost hops . . . 19
3.7 The per-excursion bound . . . 20
3.8 Theorem A . . . 21

4 The label **22**
4.1 Shown sets and digits . . . 22
4.2 The label of k walks . . . 25

5 Displays with break bits **26**
5.1 Why break bits . . . 26
5.2 Displays at one time . . . 27
5.3 Entries and the composed display . . . 28
5.4 Lemma D: the display is small . . . 29
5.5 The display above a group . . . 31
5.6 View invariance . . . 32
5.7 Exposure . . . 34

6 Accounting **34**
6.1 Events and standard events . . . 35
6.2 The counting facts . . . 36
6.3 Truncation . . . 37
6.4 Wraps . . . 37
6.5 The cut state edges . . . 38
6.6 The violated hops . . . 39
6.7 Totals . . . 40

7 Proof of the main theorem **41**
7.1 The parameters . . . 41
7.2 The cut, summed over the random choices . . . 42
7.3 The main estimate . . . 43
7.4 Theorem T . . . 43

8 Why each ingredient is needed **44**

9 Consequences for the series and for tape machines **45**
9.1 The deep-nesting pairs 45
9.2 Oblivious machines: the crux alone 47
9.3 What a linear-time transposer would need 52

10 Open problems **52**

A Index of Lean declarations **53**

B Building and verifying the formalization **58**

1 Introduction

1.1 The problem

Can a deterministic multitape Turing machine transpose a $d \times d$ matrix with $b(d) = \max(1, \lceil \log_2 d \rceil)$-bit entries, written row by row, in time linear in the input length $L = d^2 b(d)$? This is the statement (T) of the first paper of this series [6]. By a reduction of Harvey and van der Hoeven [17, Cor. 6.9], a linear-time multiplication machine would give a linear-time transposer, so a proof of ¬(T) would give the first superlinear lower bound for off-line integer multiplication on multitape Turing machines. The question is open, and this paper does not decide it.

For oblivious machines the fourth paper of the series [5] reduced ¬(T) to two statements of different kinds (its Theorem D). The first is *depth reduction*: the time–space dependency DAG of every oblivious linear-time run can be cut to bounded depth by revealing εL bits. At the level of head walks this is the *trajectory cut problem*. A *walk* is a sequence $x(0), x(1), \ldots$ of integers with $|x(t+1) - x(t)| \le 1$, the positions of one head; a *long hop* is a pair $p < u$ of consecutive visits to one cell with $u \ge p + 2$. The *walk DAG* of k walks of length T has the vertices $0, \ldots, T-1$, the state edges $t \to t-1$, and an edge $u \to p$ for every long hop of every walk. TCP(k) says: for every $\varepsilon > 0$ there is a D such that, for every T and every k walks of length T, deleting some εT edges leaves no path with more than D hop edges. The second statement, the *crux*, is about codes: every index code for the side information of the remaining bounded-depth cones needs more than εL bits.

Depth reduction has been settled step by step. The fourth paper proved TCP for home-based work heads, whose legs all begin or end near fixed homes, for every coupling of the heads (its Theorem B). The eighth paper [4] proved it for walks of bounded nesting load (its Theorem F), and for one arbitrary walk together with walks of bounded load at boundedly many scales (its Theorem M). It also showed that TCP is, up to a factor 2 in k, depth reduction for *stack DAGs* ([4, Proposition 4.5]; Proposition 2.4 here). An s-stack DAG on $[0, N)$ has the spine edges $t \to t-1$ and arcs $u \to p$ with $u \ge p + 2$, partitioned into s families in which no two arcs cross and each vertex is the left end of at most one arc and the right end of at most one. SR(s) says that every family of s-stack DAGs is depth-reducible in the above sense. Two stacks are planar, so SR(2) holds; SR(3) was the first open case [4, Problem 10.2].

1.2 The expected answer, and the actual one

The series expected TCP(2) to fail. Its sharpest candidate for a depth-robust family of walks was the *deep-nesting pair* of the fourth paper: a walk whose strokes are nested at $m+1$ levels, together with a copy shifted in time. Conjecture A′ of the fourth paper and its weakening, Conjecture A″ of the eighth, would make these pairs depth-robust as m grows, with depth $2^{\Omega(m)}$ at fixed cost [4, Corollary 5.26]; lower bounds for several restricted classes of labels supported this expectation [5, 4, 9, 7]. The actual answer is the opposite: TCP(k) holds for every k.

1.3 Results

- **Theorem T** (Theorem 7.5). For every $k \ge 1$ and $\varepsilon > 0$ there is a $D(k, \varepsilon)$ such that the walk DAG of any k walks of any length T, with or without pauses, has a cut of at most εT edges that leaves no path with more than $D(k, \varepsilon)$ hop edges. So TCP(k) holds for every k, and by the reduction of the eighth paper SR(s) **holds for every** s: bounded-stack DAGs with a spine reduce to constant depth. The bound is explicit, $D(k, \varepsilon) = K^P - 1$ with the parameters of Definition 7.1, and $\log\log D(k, \varepsilon) = O(k^2 \log(k/\varepsilon))$ for $\varepsilon \le \frac{1}{2}$ (Section 10).

- **Corollary N** (Corollary 9.1). The deep-nesting pairs are cut to depth $D(2, \varepsilon)$, independently of the nesting depth m and of the shift. Hence Conjectures A′ and A″ are false: on a cyclic cover, the label of Theorem T is cheap and doubly serves almost every level with a number of values independent of m. Both statements are Lean-verified.

- **Corollary O** (Corollary 9.3). Every oblivious machine with k work tapes running in time $C(L+1)$ has, for every $\varepsilon > 0$, a genie of cost at most εL whose full depth is bounded in terms of k, C, the value width and ε. So the crux alone excludes oblivious linear-time transposers, whatever their head motion, and for machines with a linear data path the linear crux alone does. These statements are Lean-verified in the frozen machine model of the first paper. In particular, the native form of Question 9.1 of *Directed Recovery Structures* [10], the half bound for all codes on the cone relations of linear-time machines, would exclude them (Remark 9.5). On the side of constructions, an oblivious linear-time transposer must carry codes of rate close to 1 on the cone relations of its own schedule at bounded depth: the coding side is the whole question (Section 9.3).

- **Corollary HV** (Corollary 9.6). The same genies give the ball systems of oblivious linear-time transposers index codes of at most εL bits at a bounded radius, and, for machines with a linear data path, minrank at most εL. So Conjecture HV* of the eleventh paper, code-level depth reduction, holds, and its Problem 9.1 has an affirmative answer [7]. This is Lean-verified as well.

- **Corollary L** (Corollary 9.7). Given the native half bound for the cone relations of frozen traces, no machine transposes in linear time with a trace of entropy at most ηL, $\eta < \frac{1}{2}$, however often it enters its cells: this is the eighth paper's Corollary F″ without its bounded-entry hypothesis. The eleventh paper's Theorem SE loses its bounded-entry hypothesis in the same way. Both are Lean-verified, with the native bound as an explicit hypothesis.

1.4 How the proof works

The proof has three layers.

Cuts are labels. A labelling $\Psi : [0, T) \to \{0, \dots, q-1\}$ defines a cut: the state edges where Ψ decreases and the hop edges $u \to p$ with $\Psi(p) \geq \Psi(u)$. Every path avoiding the cut increases Ψ at each hop edge, so its depth is less than q ([4]; Proposition 2.2 here). We build a random label with $q = K^P$ values and expected cut at most εT; some outcome gives the cut (Proposition 2.7).

Ladders, one walk at a time (Section 3). The label gives each hop a *deciding group*, a coarse scale, and pays for the hops whose two ends see different scales. To make this rare we measure scales from homes adapted to the walk. Every upper hop has a *top*, the first time inside it where the walk is highest; the hops with a common top are nested, one per cell, and form the *ladder* of the top. Their ends are the last visits to each cell before the top and the first visits after it. The ladders partition the upper hops. The outermost hop of a ladder, the *canonical excursion* of its top, starts and ends at the *base* y of the ladder; the walk between consecutive rungs forms a *hanging structure* on the climb and a *descent structure* on the descent, and the canonical excursion of every other top that it contains lies in one of these structures. Measuring the distance from the base of each excursion, suspending the structures as *frames* and restarting the descent at the levels $y + 2^e$, makes every hop cheap on average over a random grid: *Theorem A* (Theorem 3.16) bounds the expected number of hops lost per walk by 6 times the number of hops divided by the grid width h_0.

Displays with break bits, all walks together (Sections 4 to 6). Frames create an interaction between walks: while a frame runs, the segment that it suspends keeps its band frozen, and the band reappears when the segment resumes. The label therefore shows, besides the current band

group of each walk, a *display* of suspended frames, and digits that count events at each shown scale. *Lemma V* (Lemma 4.3) shows that a hop whose deciding group is shown at both ends increases the label unless the display is truncated, the shown set above the hop's group changes, or a digit wraps. The second case is the heart of the matter. *View invariance* (Proposition 5.10) shows that the part of the display above a group depends only on a *view* that changes only at standard events, which are paid for; the display is defined by random *break bits*, and the hops exposed despite view invariance are those with a broken candidate, at most a $2n/Q$ fraction (Proposition 5.11). Exact displays would be unbounded (Remark 8.3); with break bits, *Lemma D* (Lemma 5.7) bounds the size of the display independently of the depth of the stacks.

1.5 Formal status and how to read the paper

Theorem T, every numbered result of Sections 2 to 7, and Corollaries N, O, HV and L with their definitions are formalized in Lean 4 with Mathlib, in the library `BoundedStackDepth` next to this paper, which builds on the eighth paper's library and, for the corollaries, on the eleventh paper's library [7], which formalizes the deep-nesting walk and its cyclic covers, and traces, dependency DAGs and index codes in the frozen machine model of the first paper. Each result is followed by a status line naming its Lean declarations and file, and by an outline of the formal proof; the Lean statement is authoritative. The formal main theorem is `tcpAt_direct`: for k walks with pauses, cuts of at most εT edges and depth at most $K^P - 1$. Its corollaries are `tcp` and `main` (TCP(k) for every k, the statement `MainStatement` of `Statement.lean`) and `sr` (SR(s) for every s, through the eighth paper's reduction). All of them depend only on the axioms `propext`, `Classical.choice` and `Quot.sound` (Appendix B). The results of the eighth paper that the proof uses are Lean-verified there and are marked as cited.

Not formalized are the remarks of Sections 8 and 9. They explain why simpler constructions fail, or discuss the consequences; they are labelled as proof sketches, heuristics or numerical observations, and no proof uses them.

1.6 Related work

Depth-robust graphs. Erdős, Graham and Szemerédi [14] constructed sparse DAGs in which every large set of vertices leaves a long path; Valiant [23] used depth reduction as a route to lower bounds for circuits; Schnitger [22] constructed depth-robust graphs of constant degree; and depth-robust graphs are a basic tool for memory-hard functions [1]. Theorem T says that no family of bounded-stack DAGs with a spine is depth-robust, when only arcs are counted along paths.

Time and space. The computation graphs of multitape machines were studied through pebbling by Hopcroft, Paul and Valiant [18], through edge-deletion depth reduction by Paul and Reischuk [19], and through segregators, whose removal leaves every vertex with few ancestors, by Paul, Pippenger, Szemerédi and Trotter [20] and by Santhanam [21]. Theorem T is a statement of another kind about the walk DAGs of the work heads: after the removal of εT edges, every path uses a bounded number of long hops, although it may use many state edges.

Layouts. Galil, Kannan and Szemerédi [16, 15] studied separators of k-page graphs and showed that 3-page graphs can lack small separators. Some bounded-degree expanders have 3-page and 2-queue layouts [12], and monotone expanders exist [13, 3]. Expansion is not depth-robustness in our sense: the spine is free, and only arcs are counted.

Long paths. The decomposition of a tree into long paths is a standard data-structure tool, used for example for level ancestors [2], where the paths extended upwards are also called ladders. Our ladders are such a decomposition, without the extension, of the forest of nested upper hops; its use

to bound label costs appears to be new.

1.7 Organization

Section 2 fixes walks, hops, cuts, prominence and the random grid. Section 3 proves Theorem A, one walk at a time. Section 4 defines the label and proves Lemma V. Section 5 defines displays with break bits and proves Lemma D, view invariance and the exposure bound. Section 6 adds up the costs, and Section 7 chooses the parameters and proves Theorem T. Section 8 explains why each ingredient is needed, Section 9 draws the consequences for tape machines, and Section 10 lists open problems. Appendix A indexes the Lean declarations, and Appendix B explains how to build and verify them.

2 Preliminaries

This section fixes the objects of the paper and the facts about them that the proof uses: walks, hops and walk DAGs (Section 2.1); cuts, labels, TCP and SR, and the probabilistic method for label cuts (Section 2.2); owners and prominence (Section 2.3); and bands and the random grid (Section 2.4). We follow the eighth paper of the series [4] and reuse its formal definitions and results; its Lean declarations carry the namespace `BoundedNesting`. All other Lean names are relative to the namespace `BoundedStackDepth` of this paper's library.

The proof is an averaging argument. Its random choices are finitely many and equally likely, so every probability is a number of choices divided by the number of all choices, and every expectation is an average. The Lean statements are the corresponding inequalities between sums over finite sets; we state them in this form and read them as probabilities.

2.1 Walks, hops and walk DAGs

A *walk* is a map $x\colon \mathbb{N} \to \mathbb{Z}$ with $|x(t+1) - x(t)| \le 1$ for every t (`BoundedNesting.IsWalk`); a step with $x(t+1) = x(t)$ is a *pause*. A *horizon* $T \in \mathbb{N}$ fixes the times $[0, T) = \{0, \dots, T-1\}$. The objects defined below depend only on the values of the walks at times below T, so a walk of length T in the sense of the eighth paper [4, Section 2.2] is a walk extended by $x(t) = x(T-1)$ for $t \ge T$. A walk is *pause-free* before T if $x(t+1) \ne x(t)$ whenever $t+1 < T$. A tuple of k walks $x_1, \dots, x_k$ is, in Lean, a map $\mathrm{Fin}\, k \to (\mathbb{N} \to \mathbb{Z})$; the walks may be coupled arbitrarily.

A *long hop*, or simply a *hop*, of a walk x before T is a pair (p, u) of times with $p + 2 \le u < T$, $x(u) = x(p)$ and $x(t) \ne x(p)$ for $p < t < u$ (`BoundedNesting.IsHop`, `BoundedNesting.hops`): two consecutive visits to the *cell* $z = x(p)$ that are not consecutive times. The hop *spans* a time c if $p < c \le u$. Two hops at one cell that span a common time coincide, and two hops with the same start coincide (`BoundedNesting.hop_unique`, `BoundedNesting.hop_start_unique`); in particular a walk has at most T hops before T. A hop is *upper* if $x(t) > z$ for all $t \in (p, u)$ and *lower* if $x(t) < z$ for all of them (`BoundedNesting.IsUpperHop`, `BoundedNesting.IsLowerHop`). Its *owner* is the first time $o \in (p, u)$ at which the distance $|x(o) - z|$ is largest (`BoundedNesting.hopOwner`), and its *extent* is this largest distance, $\mathrm{ext}(p, u) = \max_{p<t<u} |x(t) - z|$ (`extent`).

Lemma 2.1 (Sides and owners of hops)**.** Lean: `IsHop.upper_or_lower`, `isOwner_hopOwner_of_upper` [`Walks.lean`]

Let x be a walk.

(a) Every hop of x is upper or lower.

(b) The owner of an upper hop (p, u) is the first time in (p, u) at which x is largest. (This part holds for every $x\colon \mathbb{N} \to \mathbb{Z}$.)

Outline of the formal proof. (a) The first step of a hop leaves its cell, so $x(p+1) > z$ or $x(p+1) < z$; a hop whose first step goes up lies strictly above its cell, since the walk would otherwise return to z inside the hop (the eighth paper's `BoundedNesting.above_of_step`), and the case of a first step down is the same for $-x$. (b) Inside an upper hop $|x(t) - z| = x(t) - z$, and the characterization of `BoundedNesting.hopOwner` by `BoundedNesting.hopOwner_spec` gives the owner predicate `BoundedNesting.IsOwner`. □

The lower hops of x are the upper hops of $-x$ (`BoundedNesting.IsLowerHop.neg`), and the owner of a lower hop is the first time inside it at which x is smallest. The proof treats lower hops in this way, as upper hops of the reflected walk. Upper hops do not cross: if $p < p' < u < u'$, then $x(p') > x(p)$ because p' lies inside (p, u), while u lies inside (p', u') and $x(u) = x(p) < x(p')$ (`BoundedNesting.upper_no_cross`).

The *walk DAG* of walks $x_1, \dots, x_k$ before T has the vertex set $[0, T)$, a *state edge* $t \to t-1$ for $1 \le t < T$, and a *hop edge* $u \to p$ for every pair (p, u) that is a hop of at least one of the walks (`BoundedNesting.allHops`, `BoundedNesting.hopPairs`); an edge is determined by its endpoints. A *cut* is a set of state edges, given by their upper ends $t \in [1, T)$, together with a set of hop edges. A path $t_0 \to t_1 \to \cdots \to t_n$ *avoids* the cut if none of its edges belongs to it (`BoundedNesting.AvoidPath`), and the *depth* of the cut is the largest number of hop edges on a path that avoids it (`BoundedNesting.arcCount` counts them). Paths run backwards in time.

2.2 Cuts, labels, TCP and SR

A *label* with q values is a map $\Psi\colon \mathbb{N} \to \{0, \dots, q-1\}$; only its values on $[0, T)$ matter. Its *descents* are the times $t \in [1, T)$ with $\Psi(t) < \Psi(t-1)$ (`BoundedNesting.descentsOf`), and a hop edge $u \to p$ is *violated* if $\Psi(u) \le \Psi(p)$ (`BoundedNesting.violatedOf`). The cut F_Ψ consists of the state edges at the descents and of the violated hop edges, and its size is the *cost* of Ψ (`labelCutSize`).

Proposition 2.2 (Cuts are labels)**.** Cited: the eighth paper [4], Lemma 2.3; Lean-verified there (`BoundedNesting.labelCut_depth_walkDag`, from `BoundedNesting.labelCut_depth` and `BoundedNesting.depth_le_of_label`).

Let Ψ be a label with $q \ge 1$ values. For all walks and every horizon, the cut F_Ψ of the walk DAG has depth at most $q - 1$.

Along a path that avoids F_Ψ, the label does not increase along a state edge and decreases strictly along a hop edge, and it takes q values. Every cut in this paper is of this form, so the task is to find labels with few values and small cost.

Definition 2.3 (TCP and SR)**.** Cited: the eighth paper [4], Definition 2.4 and Section 4.3, where these are Lean definitions (`BoundedNesting.TCPat`, `BoundedNesting.TCP`, `BoundedNesting.TCPnpAt`, `BoundedNesting.TCPnp`, `BoundedNesting.StackDag`, `BoundedNesting.SRat`, `BoundedNesting.SR`).

Let $k, s \in \mathbb{N}$, $\varepsilon \in \mathbb{R}$ and $D \in \mathbb{N}$.

(a) $\mathrm{TCP}(k)$ holds at (ε, D) if for every horizon T and all walks $x_1, \dots, x_k$ the walk DAG before T has a cut of at most εT edges and depth at most D. $\mathrm{TCP}_{\mathrm{np}}(k)$ holds at (ε, D) if the same is true for all walks that are pause-free before T.

(b) $\mathrm{TCP}(k)$ is the statement that for every $\varepsilon > 0$ there is a D such that $\mathrm{TCP}(k)$ holds at (ε, D), and likewise for $\mathrm{TCP}_{\mathrm{np}}(k)$.

(c) An *s-stack DAG* on $[0, N)$ has the *spine edges* $t \to t-1$ and *arcs* $u \to p$, given as pairs (p, u) with $p + 2 \le u < N$ and partitioned into s families; within a family no two arcs cross, and every vertex is the left endpoint of at most one arc of the family and the right endpoint of at most one. Cuts, paths and depth are as for walk DAGs, with arcs in place of hop edges.

(d) $\mathrm{SR}(s)$ is the statement that for every $\varepsilon > 0$ there is a D such that every s-stack DAG on $[0, N)$, for every N, has a cut of at most εN edges and depth at most D.

Proposition 2.4 (TCP is depth reduction for stack DAGs)**.** Cited: the eighth paper [4], Proposition 4.5; Lean-verified there (`BoundedNesting.walkDag_isStack`, `BoundedNesting.srAt_of_tcpnpAt`, `BoundedNesting.tcpAt_of_srAt`, `BoundedNesting.sr_of_tcpnp`, `BoundedNesting.tcp_of_sr`, `BoundedNesting.tcp_of_tcpnp_two_mul`).

(a) The walk DAG of k walks, pauses allowed, is a $2k$-stack DAG; its families are indexed by a walk and a side.

(b) Every s-stack DAG on $[0, N)$ is realized by s walks of length $2N$ without pauses, the arc (p, u) of family i being the upper hop $(2p, 2u)$ of walk i, and a cut of their walk DAG pulls back to a cut of the stack DAG of no larger size and depth.

Consequently, if $\mathrm{TCP}_{\mathrm{np}}(s)$ *holds at* $(\varepsilon/2, D)$*, then every s-stack DAG on $[0, N)$ has a cut of at most εN edges and depth at most D; if every $2k$-stack DAG on $[0, T)$ has a cut of at most εT edges and depth at most D, then* $\mathrm{TCP}(k)$ *holds at* (ε, D)*; and* $\mathrm{TCP}_{\mathrm{np}}(2k)$ *implies* $\mathrm{TCP}(k)$*.*

We prove TCP for walks with pauses directly; the construction never uses pause-freeness. By Proposition 2.4, $\mathrm{SR}(s)$ then follows from the pause-free form for s walks, a special case.

Definition 2.5 (The main statement)**.** Lean: `MainStatement` [`Statement.lean`]

The *main statement* is $\mathrm{TCP}(k)$ for every $k \in \mathbb{N}$.

Corollary 2.6 (Stack DAGs)**.** Lean: `sr_of_main` [`Statement.lean`]

If the main statement holds, then $\mathrm{SR}(s)$ *holds for every s.*

Outline of the formal proof. Pause-free walks are walks, so $\mathrm{TCP}(s)$ gives $\mathrm{TCP}_{\mathrm{np}}(s)$, and Proposition 2.4 turns it into $\mathrm{SR}(s)$ (`BoundedNesting.tcpnp_of_tcp`, `BoundedNesting.sr_of_tcpnp`). □

The main statement is proved by the probabilistic method. For k walks we build a finite family of labels, indexed by the random choices, whose values lie below a bound q that depends only on k and ε and whose average cost is at most εT. Some member of the family is then a cut of the required kind. The following proposition is this argument; the rest of the paper constructs the families and bounds their average cost.

Proposition 2.7 (The probabilistic method for label cuts)**.** Lean: `labelCutSize`, `exists_cut_of_avg` [`Assembly.lean`]

Let $x_1, \dots, x_k$ be walks, T a horizon, $\varepsilon \in \mathbb{R}$, Ω a nonempty finite set, and $(\Psi_\omega)_{\omega \in \Omega}$ labels with values below q. If $\sum_{\omega \in \Omega} |F_{\Psi_\omega}| \le |\Omega| \cdot \varepsilon T$, then the walk DAG before T has a cut of at most εT edges and depth at most $q - 1$.

Outline of the formal proof. Some $\omega \in \Omega$ has $|F_{\Psi_\omega}| \le \varepsilon T$, since the sum is at most $|\Omega|$ times this bound (Mathlib's `Finset.exists_le_of_sum_le`). The cut F_{Ψ_ω} consists of the descents and the violated hop pairs of Ψ_ω (`labelCutSize`), and Proposition 2.2 bounds its depth by $q - 1$ (`exists_cut_of_avg`). □

2.3 Owners and prominence

By Lemma 2.1(b), the owner of an upper hop is the first time inside it at which the walk is highest. We call the owners of the upper hops of x its *tops* (`tops`), and the number of upper hops owned by a top τ its *height* H_τ (`height`); Section 3 is about the hops of one top. The proof measures the scale of a top by how far the walk must descend from it before it reaches a higher point. Ties are broken in time: an earlier point of equal height counts as higher, a later point only if it is strictly higher.

Definition 2.8 (Prominence)**.** Lean: `colL`, `colR`, `peak`, `valley`, `prom`, `extent` [`Prominence.lean`]

Let $x\colon \mathbb{N} \to \mathbb{Z}$, T a horizon, $t < T$ and $v = x(t)$. Let $\min x$ denote the minimum of x over $[0, T)$.

(a) The *left col* $L(t)$ is the minimum of x over $[s_L, t]$, where s_L is the last time $s < t$ with $x(s) \geq v$; if there is no such time, $L(t) = \min x$.

(b) The *right col* $R(t)$ is the minimum of x over $[t, s_R]$, where s_R is the first time s with $t < s < T$ and $x(s) > v$; if there is no such time, $R(t) = \min x$.

(c) The *peak prominence* is $\max(0,\, v - \max(L(t), R(t)))$, the *valley prominence* is the peak prominence of $-x$ at t, and the *prominence* $\mathrm{prom}(t)$ is the larger of the two.

A missing side contributes the global minimum, which is at most the other col, so it is ignored; if both sides are missing, the peak prominence is $v - \min x$. For example, on the walk $0, 1, 2, 1, 2, 3, 2, 1, 0$ ($T = 9$) the peak prominences are $0, 0, 1, 0, 0, 3, 0, 0, 0$ and the valley prominences $3, 0, 0, 1, 0, 0, 0, 0, 3$; on $0, 1, 1, 0$ the peak prominences are $0, 1, 0, 0$, so of two equal neighbouring maxima only the first is prominent. These values are checked by the kernel in `Prominence.lean`. By definition, the prominence does not change when x is replaced by $-x$.

Proposition 2.9 (Prominence)**.** Lean: `sub_le_prom`, `sum_prom_le`, `sum_prom_le_two_mul`, `card_spanning_hops_lt` [`Prominence.lean`]

Lean: `height_le_prom`, `prom_le_of_left`, `prom_le_of_right` [`TopProminence.lean`]

Let $x\colon \mathbb{N} \to \mathbb{Z}$ and let T be a horizon.

(P1) (Owners.) *If (p, u) is an upper hop with $u < T$ and owner o, then $\mathrm{prom}(o) \geq x(o) - x(p)$, the extent of the hop.*

(P2) (The prominence sum.) *If x is a walk, then $\sum_{t<T} \mathrm{prom}(t)$ is at most the number of non-pause steps among the first $T-1$ steps plus $\max x - \min x$ (over $[0, T)$), and hence at most $2(T-1)$.*

(P3) (Spanning hops.) *For every time c and every $R \geq 1$, fewer than $2R$ hops (p, u) of x before T with $p < c \leq u$ have extent less than R.*

(P4) (Tops.) *If x is a walk, every top τ has $\mathrm{prom}(\tau) \geq H_\tau$.*

(P5) (Caps.) *Let $1 \leq t < T$ with $x(t-1) < x(t)$, and let $y \in \mathbb{Z}$. If some $s < t$ has $x(s) \geq x(t)$ and $x \geq y$ on $[s, t]$, or some s with $t < s < T$ has $x(s) > x(t)$ and $x \geq y$ on $[t, s]$, then $\mathrm{prom}(t) \leq x(t) - y$.*

Outline of the formal proof. (P1) The extent is attained at the owner (`extent_le_owner`). The walk is below $x(o)$ on $[p, o)$, since o is the first maximum, and at most $x(o)$ on $(o, u]$. So the last earlier time at least as high as o lies before p, the first later time strictly higher lies after u, and both cols are at most $x(p) = x(u)$ (`colL_le_of`, `colR_le_of`, `sub_le_peak`). (P2) A double count over levels (`sum_peak_le`, `sum_valley_le`). Fix a level y and call a time τ *active* if $x(\tau) > y$ and

both of its cols are at most y, that is, if y lies in $[x(\tau) - \mathrm{peak}(\tau), x(\tau))$. Two active times never lie in one maximal run of times at which the walk is above y: of the two, the lower one, or the later one at equal height, has a point at least as high on the other one's side, which lifts its col above y (`active_unique`). Each such run starts at time 0 or at an up-step through y (`card_active_le`), so summing over y bounds the peak prominences by the number of up-steps plus $x(0) - \min x$, and the valley prominences by the number of down-steps plus $\max x - x(0)$ (`sum_prom_le`). Both terms of the sum are at most $T - 1$ for a walk. (P3) The cell of such a hop is within $R - 1$ of $x(c)$: either $c = u$ and $x(c)$ is the cell, or c lies inside the hop. These are $2R - 1$ cells, and at each of them at most one hop spans c (`BoundedNesting.hop_unique`). (P4) The hop of τ at the lowest cell has extent H_τ (Lemma 3.2), so (P1) applies to it. (P5) Since the walk steps up into t, the valley prominence of t vanishes (`valley_eq_zero_of_step_up`); the higher point s, reached without going below y, lifts the corresponding col above $y - 1$ (`lt_colL`, `lt_colR`). □

The owner bound (P1) makes every owner a turning point of positive prominence, which is how the label sees a hop (Lemma 3.13); (P2) and (P3) bound the weight of turning points and the number of short hops at a time in the accounting of Section 6; (P4) and (P5) are the two estimates for tops that Section 3 needs.

2.4 Bands and the random grid

Distances enter the label only through their binary orders of magnitude, grouped in a random grid of width h_0.

Definition 2.10 (Bands, groups and floors)**.** Lean: `band`, `grp`, `flr` [`Basic.lean`]

The *band* of $M \in \mathbb{N}$ is $\mathrm{band}(M) = \lfloor \log_2 M \rfloor$ for $M \geq 1$, and $\mathrm{band}(0) = 0$. For a *width* $h_0 \geq 1$ and an *offset* $r \in \{0, \ldots, h_0 - 1\}$, the *group* of a band b is $G(b) = \lfloor (b + r)/h_0 \rfloor$, and the *floor* of a group g is $\theta(g) = \max(h_0 g - r, 0)$. The *grid floors* of the offset r are the floors $\theta(g)$ with $g \geq 1$.

The group map and the floors depend on h_0 and r, which will be clear from the context. The groups are the blocks $[\theta(g), \theta(g+1))$ of h_0 consecutive bands (the block of group 0 is shorter when $r > 0$), and the offset shifts the blocks. The offset will be uniform on $\{0, \ldots, h_0 - 1\}$. Every probability bound that the paper needs about the grid is then an upper bound on a number of offsets, and all of them come from one fact: an integer $\theta \geq 1$ is a grid floor for at most one offset. Indeed, $\theta = h_0 g - r$ with $g \geq 1$ means $h_0 \mid \theta + r$, and two offsets below h_0 that are congruent modulo h_0 are equal.

Lemma 2.11 (The random grid)**.** Lean: `flr_grp_le`, `lt_flr_grp_succ`, `grp_mono`, `card_dvd_add_le_one`, `exists_floor_of_grp_lt`, `card_grp_ne_le`, `card_one_le_grp_le`, `card_flr_grp_succ_le`, `card_lt_flr_grp_le`, `sum_two_pow_flr_grp_le` [`Grid.lean`]

Let $h_0 \geq 1$, let the offset r range over $\{0, \ldots, h_0 - 1\}$, and let $a, b, c, s, \beta \in \mathbb{N}$. Subtraction of natural numbers is truncated at 0.

(a) $\theta(G(b)) \leq b < \theta(G(b) + 1)$, and G is nondecreasing.

(b) For every $\theta \in \mathbb{N}$, at most one offset $r < h_0$ has $h_0 \mid \theta + r$.

(c) If $G(a) < G(b)$, then $h_0 \mid \theta + r$ for some θ with $a < \theta \leq b$.

(d) If $a \leq b$, then $G(a) \neq G(b)$ for at most $b - a$ offsets. Moreover $G(s) \geq 1$ for at most s offsets; $\theta(G(b) + 1) \leq c$ for at most $c - b$ offsets; and $\theta(G(\beta)) > s$ for at most $\beta - s$ offsets.

(e) The sum of $2^{\theta(G(s))}$ over the offsets with $\theta(G(s)) \geq 1$ is at most 2^{s+1}.

Outline of the formal proof. (a) Division with remainder of $b + r$ by h_0. (b) Two offsets $r, r' < h_0$ with $\theta + r \equiv \theta + r' \pmod{h_0}$ are equal (`card_dvd_add_le_one`). (c) The floor $h_0 G(b) - r$ lies in $(a, b]$. (d) Each count is bounded by the number of integers in an interval that are grid floors, each for at most one offset by (b). If $G(a) \neq G(b)$, then $G(a) < G(b)$ by (a), and a grid floor lies in $(a, b]$ by (c), an interval of $b - a$ integers (`card_grp_ne_le`). The other three reduce to this one: $G(s) \geq 1$ means $G(0) \neq G(s)$, since $G(0) = 0$; $\theta(G(b) + 1) \leq c$ forces $G(b) < G(c)$; and $\theta(G(\beta)) > s$ forces $G(s) < G(\beta)$. (e) Distinct offsets give distinct positive floors $\theta(G(s)) \leq s$ (`flr_grp_injOn`), so the sum is at most $\sum_{\theta \leq s} 2^\theta < 2^{s+1}$. □

With r uniform, part (d) says that the groups of two bands $a \leq b$ differ with probability at most $(b - a)/h_0$: this is how band differences become costs in Lemma 3.12. Part (e) prices the events of the accounting: for an event of scale s, the largest grid floor $\theta \leq s$, weighted by 2^θ, averages at most $2^{s+1}/h_0$ over the offsets. The costs of one walk are added up with the following exact sum, Lemma E, which bounds the climb side of every ladder in Section 3.

Lemma 2.12 (Lemma E: band sums)**.** Lean: `sum_band_sub_eq` [`BandSums.lean`]

Let $N \in \mathbb{N}$ and $B = \mathrm{band}(N)$. Then $\sum_{x=1}^{N} (B - \mathrm{band}(x)) = 2^{B+1} - B - 2$.

Outline of the formal proof. From N to $N + 1$ the band either stays or rises by one, and it rises only when $N + 1$ is a power of two (`band_succ`); induction on N gives $\sum_{x=1}^{N} \mathrm{band}(x) + 2^{B+1} = NB + B + 2$ (`sum_band_add_pow`), which is the identity (`sum_band_sub_eq`). □

Since $2^B \leq N$ for $N \geq 1$, the sum is at most $2N$: most integers up to N have a band close to $\mathrm{band}(N)$.

3 Ladders: the structure of one walk

This section is about a single walk x, pauses allowed, and a horizon T. It equips x with two frame systems, one for its upper hops (*class U*) and one for its lower hops (*class L*, the same construction for $-x$), and proves Theorem A (Theorem 3.16): for a uniformly random offset of the grid of width h_0, the expected number of lost hops of each class is at most $6/h_0$ times the number of hops of the class. The other walks and the label enter only in Sections 4 to 6.

A frame system gives the walk a *band* $\beta(t)$ at every time: the band of the running maximum of its distance from a reference cell, its *home*. A hop is cheap if the bands at its ends lie in the same group of the grid and two side conditions hold; bands that differ by Δ lie in different groups for at most Δ of the h_0 offsets (Lemma 2.11). So the homes must make the band differences, summed over all hops, linear in the number of hops. We treat upper hops; lower hops are the upper hops of $-x$ (Lemma 3.2(e)).

3.1 The decomposition by tops

Definition 3.1 (Tops, ladders and excursions)**.** Lean: `upperHops`, `lowerHops`, `tops`, `ladder`, `height` [`Decomposition.lean`]

Lean: `base`, `ladderHop`, `ladL`, `ladD`, `exc`, `levels` [`FramesU.lean`]

Let x be a walk and T a horizon.

(a) The *upper hops* of x are its hops (p, u) with $x(p + 1) > x(p)$, and its *lower hops* those with $x(p + 1) < x(p)$. An upper hop stays above its cell and a lower hop below it (Lemma 2.1).

(b) The *top* of an upper hop is its owner. The *ladder* of a top τ is the set of upper hops with top τ, its *height* H_τ is the number of these hops, its *base* is $y_\tau = x(\tau) - H_\tau$, and its *levels* are the integers w with $y_\tau < w < x(\tau)$.

(c) For $y_\tau \le z < x(\tau)$, $(L_\tau(z), D_\tau(z))$ denotes the hop of the ladder of τ at the cell z; by Lemma 3.2(a) there is exactly one.

(d) The *canonical excursion* of τ is $E_\tau = (L_\tau(y_\tau), D_\tau(y_\tau))$, the hop of its ladder at the base.

Lemma 3.2 (Ladders)**.** Lean: `image_cell_ladder`, `ladder_cell_iff`, `ladder_eq_of_cell`, `mem_ladder_hopOwner`, `eq_of_mem_ladder`, `ladder_lt`, `ladder_above`, `ladder_last_visit`, `ladder_first_visit`, `ladder_nested`, `exists_ladder_outer`, `top_spec`, `top_ne_start`, `top_ne_end`, `card_upperHops_add_card_tops_le`, `sum_height_eq`, `upperHops_neg`, `upperHops_union_lowerHops` [`Decomposition.lean`]

Let x be a walk, T a horizon, and τ a top of height H and base y.

(a) (One hop per cell.) *The cells of the hops of the ladder of τ are exactly $y, y+1, \dots, x(\tau)-1$, and the ladder has exactly one hop at each of them. Every upper hop lies in exactly one ladder, that of its top.*

(b) (Rungs.) *Let (p, u) be the hop of the ladder at the cell z. Then $p < \tau < u$, the walk is above z on (p, u), p is the last visit to z at or before τ, and u is the first visit to z at or after τ. A higher cell gives a later start and an earlier end, so E_τ contains every hop of the ladder.*

(c) (Tops.) *$1 \le \tau$, $\tau + 1 < T$, $x(\tau-1) < x(\tau)$ and $x(\tau+1) \le x(\tau)$. A top is neither the start nor the end of an upper hop.*

(d) (Counting.) *The number of upper hops plus the number of tops is at most T, and $\sum_\tau H_\tau$, over all tops, is the number of upper hops.*

(e) (Reflection.) *The upper hops of $-x$ are the lower hops of x, and every hop of x is upper or lower.*

Outline of the formal proof. (a) Every hop of the ladder contains τ (`ladder_lt`), so two of them at one cell coincide by `BoundedNesting.hop_unique` (`ladder_eq_of_cell`). If the ladder has a hop at the cell z_0 and $z_0 \le z < x(\tau)$, the last visit to z before τ and the first visit after it (`exists_last_visit`, `exists_first_visit`) form an upper hop on which τ is again the first maximum (`exists_ladder_cell`); so the cells form an interval ending at $x(\tau) - 1$. Owners are unique (`mem_ladder_hopOwner`, `eq_of_mem_ladder`). (b) A visit to z in $(p, \tau]$ or $[\tau, u)$ would lie inside the hop. For $z < z'$ the walk passes z' between $L_\tau(z)$ and τ, so $L_\tau(z) < L_\tau(z')$; symmetrically $D_\tau(z') < D_\tau(z)$. (c) τ is the first maximum of a hop (p, u), so $x(\tau-1) < x(\tau)$ and $x(\tau+1) \le x(\tau)$, with $\tau+1 \le u < T$; an upper hop starting at τ would need $x(\tau+1) > x(\tau)$, and one ending at τ would need $x(\tau-1) > x(\tau)$. (d) Hops with the same start coincide (`BoundedNesting.hop_start_unique`), so the starts of the upper hops are distinct, and by (c) they are not tops. The ladders are the fibres of the owner map. (e) Reflection preserves hops and reverses their first step, which leaves the cell. □

Example 3.3. On the walk $0, 1, 2, 3, 2, 3, 2, 1, 0$ with $T = 9$, the upper hops are $(0, 8)$, $(1, 7)$, $(2, 4)$ and $(4, 6)$, the only lower hop is $(3, 5)$, and the tops are 3 and 5. The ladder of 3 is $\{(0, 8), (1, 7), (2, 4)\}$, of height 3 and base 0; the ladder of 5 is $\{(4, 6)\}$, of height 1 and base 2. The hop $(2, 4)$ of the top 3 ends where the excursion $E_5 = (4, 6)$ starts: an excursion may start at a ladder time of another top. On $0, 1, 1, 0$ the only top is 1, and the walk pauses there. These values are checked by the kernel in `Decomposition.lean`.

Why ladders make every hop cheap. The ladders partition the upper hops, and a top of height H owns H of them, one per cell between its base y and its peak (Lemma 3.2); Theorem A charges every hop to its ladder. The rung at the cell $y + d$ starts on the climb, after the walk has risen by d from the base, and ends on the descent, after the walk has been at the top. Measured from the base, the band at its start is about $\mathrm{band}(d)$, and the band at its end, a running maximum, is about $B = \mathrm{band}(H)$. The rung costs at most the sum of the deficits of the two bands below B (Lemma 3.12). The deficit $B - \mathrm{band}(d)$ exceeds j only for $d < 2^{B-j} \le H/2^j$, so the deficits along the climb sum to $O(H)$ (Lemma E, Lemma 2.12); restarting the descent at geometrically spaced passages adds $O(H)$ more (Lemma 3.14). Between the rungs the walk is arbitrary and may contain tall excursions; the frames below suspend them, so that they do not disturb the bands on the ladder.

3.2 Excursions and structures

Let τ be a top with base y, and w a level of τ. Between the rungs $L_\tau(w-1)$ and $L_\tau(w)$ of the climb the walk stays at or above w, and it is at w at both ends of the *hanging structure* $[L_\tau(w-1)+1, L_\tau(w)]$. Symmetrically, $[D_\tau(w), D_\tau(w-1)-1]$ is the *descent structure* at level w. A structure is a sequence of upper excursions above its level, possibly with pauses at the level. The *hanging frame* and the *descent frame* of τ at level w are

$$[L_\tau(w-1)+1,\ L_\tau(w)) \qquad \text{and} \qquad [D_\tau(w)+1,\ D_\tau(w-1))$$

(`hangFrame`, `descFrame`), possibly empty. The descent frame starts one step after its structure, so that the ladder time $D_\tau(w)$ stays a time of the frame of the ladder. The frames of τ are its hanging and descent frames at all its levels (`framesOf`), and the *suspend frames* of x are the frames of all its tops (`suspendFrames`).

Lemma 3.4 (Excursions and structures)**.** Lean: `exc_range`, `upper_laminar`, `exc_laminar`, `exc_structure`, `framesOf_bounds`, `exists_framesOf_nested`, `frames_laminar`, `ladder_time_not_mem_framesOf` [`Excursions.lean`]

Let x be a walk and T a horizon; let τ and τ' be tops, with $E_\tau = (a, b)$ and $E_{\tau'} = (a', b')$. Say that $E_{\tau'}$ lies in *E_τ if $a \le a'$ and $b' \le b$.*

(a) On $[a, b]$ the walk stays in $[y_\tau, x(\tau)]$. Two upper hops are nested or meet in at most one endpoint; in particular, so are E_τ and $E_{\tau'}$.

(b) (Structures.) *If $\tau' \ne \tau$ and $E_{\tau'}$ lies in E_τ, then some level w of τ with $w \le y_{\tau'}$ satisfies either $\tau' < \tau$ and $[a', b'] \subseteq [L_\tau(w-1)+1, L_\tau(w)]$, or $\tau < \tau'$ and $[a', b'] \subseteq [D_\tau(w), D_\tau(w-1)-1]$.*

(c) Every frame $[f_1, f_2)$ of τ has $a < f_1$ and $f_2 \le b$. In the situation of (b), $[a'+1, b')$ lies in a frame of τ. The root $[0, T)$ and the suspend frames form a laminar family: any two of them are nested or disjoint.

(d) The ladder times of τ, that is, τ and the times $L_\tau(z)$ and $D_\tau(z)$ for $y_\tau \le z < x(\tau)$, lie in no frame of τ and in no frame of a top whose canonical excursion lies in E_τ.

Outline of the formal proof. (a) The walk is above y_τ inside the outermost ladder hop and at most its maximum $x(\tau)$ (`exc_range`). Upper hops do not cross (`BoundedNesting.upper_no_cross`), hence are nested or meet in at most an endpoint (`upper_laminar`). (b) $\tau \notin [a', b']$, since there it would be the first maximum of $E_{\tau'}$ (`top_eq_of_mem`). Say $b' < \tau$. The time a' is a visit to $y_{\tau'}$ before τ, so $a' \le L_\tau(y_{\tau'})$, while $L_\tau(y_\tau) = a < a'$ because ladder hops with a common start have a common top (`top_eq_of_start`). Take the level w with $L_\tau(w-1) < a' \le L_\tau(w)$ (`exists_switch`);

an upper hop that ends by τ cannot contain the last visit $L_\tau(w)$ strictly inside (`not_lt_ladL_lt`), so $b' \le L_\tau(w)$. The case $\tau < a'$ is symmetric (`not_lt_ladD_lt`, `top_eq_of_end`). (c) The frames of τ lie between consecutive rungs inside (a, b) and are pairwise disjoint (`framesOf_disjoint`); by (b) the interior of a nested excursion lies in one of them, so frames of nested excursions are nested or disjoint (`framesOf_laminar_of_nested`), and frames of disjoint excursions are disjoint. (d) A frame of τ lies strictly between consecutive rungs. A frame of another top whose excursion lies in E_τ lies strictly inside that excursion, which does not contain τ by (b), nor any $L_\tau(z)$ or $D_\tau(z)$ strictly inside (`not_lt_ladL_lt`, `not_lt_ladD_lt`). □

The excursions thus form a forest under inclusion, and a nested excursion lies in a single structure of its parent. In Example 3.3, $E_5 = (4, 6)$ starts at $D_3(2) = 4$, a ladder time of its parent, and lies in the descent structure $[4, 6]$ of the top 3 at level 2, whose descent frame is $[5, 7)$: this is why the structures are closed intervals.

3.3 Frame systems

Definition 3.5 (Frame systems)**.** Lean: `FrameSystem`, `FrameSystem.Own`, `FrameSystem.inner`, `FrameSystem.segStart`, `FrameSystem.runMax`, `FrameSystem.home`, `FrameSystem.beta` [`FrameSystem.lean`]

A *frame system* $\mathcal{S}$ consists of a horizon T, a map $x\colon \mathbb{N} \to \mathbb{Z}$, a finite set of *frames*, pairs $F = (f_1, f_2)$ with $f_2 \le T$ read as intervals $[f_1, f_2)$, possibly empty, and a finite set R of *restart points*, such that the root $[0, T)$ is a frame and any two frames are nested or disjoint.

(a) A time t is an *own time* of a frame F if F contains t and lies inside every frame that contains t. For $t < T$, $\mathrm{inner}(t)$ is the frame of which t is an own time (unique by Lemma 3.6).

(b) The *segment start* $s_F(t)$ is the largest element of the set consisting of f_1 and of the restart points $s \in R$ with $f_1 < s \le t$ that are own times of F. The pair $(F, s_F(t))$ is the *segment* of F at t.

(c) The *running maximum* $M_F(t)$ is the largest $|x(t') - x(s_F(t))|$ over the own times t' of F with $s_F(t) \le t' \le t$, and 0 if there is none.

(d) The *home* of t is $\mathrm{home}(t) = x(s_{\mathrm{inner}(t)}(t))$, and its *band* is $\beta(t) = \mathrm{band}(M_{\mathrm{inner}(t)}(t))$.

Lemma 3.6 (Dynamics of a frame system)**.** Lean: `FrameSystem.Restart`, `FrameSystem.segStart_of_restart`, `FrameSystem.segStart_start`, `FrameSystem.segStart_succ`, `FrameSystem.runMax_of_restart`, `FrameSystem.runMax_start`, `FrameSystem.runMax_succ_of_not_own`, `FrameSystem.runMax_succ_of_own`, `FrameSystem.runMax_frozen`, `FrameSystem.runMax_le_succ` [`FrameDynamics.lean`]

Lean: `FrameSystem.exists_own`, `FrameSystem.Own.unique`, `FrameSystem.band_dist_home_le_beta` [`FrameSystem.lean`]

Let $\mathcal{S}$ be a frame system and $F = (f_1, f_2)$ a pair of times. Call t a restart *of F if $t \in R$, $f_1 < t$, and t is an own time of F.*

(a) Every time $t < T$ is an own time of exactly one frame.

(b) $s_F(f_1) = f_1$ and $M_F(f_1) = 0$. If t is a restart of F, then $s_F(t) = t$ and $M_F(t) = 0$.

(c) Let $f_1 < t$, and let t not be a restart of F. Then $s_F(t) = s_F(t-1)$; moreover $M_F(t) = M_F(t-1)$ if t is not an own time of F, and $M_F(t) = \max(M_F(t-1), |x(t) - x(s_F(t))|)$ if it is.

(d) *If $f_1 \le t_0 \le t$ and F has no own time in $(t_0, t]$, then $M_F(t) = M_F(t_0)$. If $f_1 < t$ and t is not a restart of F, then $M_F(t-1) \le M_F(t)$.*

(e) *For $t < T$, $\beta(t) \ge \mathrm{band}|x(t) - \mathrm{home}(t)|$.*

Outline of the formal proof. (a) A shortest frame containing t lies inside every frame containing t, since such frames are nested (`FrameSystem.nested_of_contains`). (b), (c) The candidate segment starts up to t are those up to $t-1$, and t if it is a restart (`FrameSystem.segStart_eq`); the own times of the current segment up to t are those up to $t-1$, and t if it is an own time. (d) Induction with (c). (e) t is an own time of $\mathrm{inner}(t)$ in its current segment. □

The hybrid convention. Only own times enter a running maximum: while a nested frame is innermost, its parent's running maximum is frozen, and when the nested frame ends, the parent resumes it. Frames are nested, their maxima are not; and a restart point starts a new segment only in the frame of which it is an own time. The proof uses three consequences. The bands at the ends of every rung are bounded in terms of the ladder alone, however tall the suspended structures between the rungs are (Lemma 3.8). Consecutive own times of a frame of class U are at most one level apart, so the maximum of a segment is less than its number of own times, which bounds the weight of the segments as events of the label (Lemma 3.10). And a suspended frame's band is a fixed value that the label can display while the frame is suspended (Section 5); since every suspend frame ends at band 0 (Lemma 3.9), the end of a frame only returns the walk from band 0 to its parent's frozen band, up to the update of the parent's running maximum when it resumes.

3.4 The frames of class U

Definition 3.7 (The frames of class U)**.** Lean: `hangFrame`, `descFrame`, `suspendFrames`, `passages`, `innerDescents`, `restartPts` [`FramesU.lean`]
Lean: `framesOf`, `upperFrames` [`Excursions.lean`]

Let x be a walk and T a horizon.

(a) The *suspend frames* are the hanging frames $[L_\tau(w-1)+1, L_\tau(w))$ and the descent frames $[D_\tau(w)+1, D_\tau(w-1))$ of all tops τ at all their levels w.

(b) The *passages* of a top τ are the times $D_\tau(y_\tau + 2^e)$ for the integers $e \ge 0$ with $2^e < H_\tau$.

(c) The *restart points* are the passages of all tops, and the starts $L_\tau(y_\tau)$ of the canonical excursions of all tops except those that equal a descent-ladder time $D_\sigma(w)$ of a top σ at a level w of σ.

(d) The frame system of *class U* of x has the horizon T, the walk x, the frames $[0, T)$ and the suspend frames, and these restart points; it is a frame system by Lemma 3.4(c). The frame system of *class L* of x is that of class U of $-x$. The upper hops of x belong to class U, its lower hops to class L.

The starts excluded in (c) are those of the first excursions of descent structures that begin with their structure, at a ladder time of the parent; their segments start one step later, at the start of the descent frame. Inside E_τ, the own times of the frame $\mathrm{inner}(\tau)$ are the times in no frame of τ or of a descendant, among them the ladder times, and its restarts are the passages (see the outline of Lemma 3.8): the climb, the top and the descent down to the highest passage form one segment, with home y_τ or $y_\tau + 1$.

Why passages at powers of two. Every suspend frame must end at band 0 (Lemma 3.9 below; the display needs this), and its last time typically lies at the end of the descent of the frame's last excursion. Without a restart on the descent, the running maximum reached at the top would stay until the excursion ends. The lowest passage, $D_\tau(y_\tau + 1)$, restarts the segment one level above the base; it is the passage that the proof of Lemma 3.9 uses. Every restart lowers the bands at the ends of the rungs below it, and each lowered band costs its deficit below $B = \mathrm{band}(H)$. Restarting at the levels $y_\tau + 2^e$ lets the band on the descent come down with the remaining height, as the band on the climb goes up with the height reached, and it keeps the cost linear: the rung at $y_\tau + d$ with $2^{e-1} < d \le 2^e < H_\tau$ ends at a band of at least $\mathrm{band}(2^e - d)$, and the deficits $B - \mathrm{band}(2^e - d)$ over this dyadic block add up to $2^{e-1}(B - e + 3) - 2$ (Lemma 3.14(a)), a series that is geometric in $B - e$. Restarts at every level of the descent would instead cost about B per rung, about $H \log_2 H$ on a single excursion.

To state the bands at the ladder times, let $\lceil \log_2 d \rceil$ be the least e with $d \le 2^e$, and for $1 \le d < H$ let

$$\mathrm{ub}(H, d) = \begin{cases} \mathrm{band}(2^{\lceil \log_2 d \rceil} - d) & \text{if } 2^{\lceil \log_2 d \rceil} \le H - 1, \\ \mathrm{band}(H - 1) & \text{otherwise} \end{cases}$$

(`uband`). The first case concerns the rungs at and below the *top passage*, the highest passage of the excursion.

Lemma 3.8 (Bands at ladder times)**.** Lean: `home_mem_exc`, `beta_le_band_height`, `beta_ladL_bounds`, `beta_ladD_bounds`, `ladL_lt_segStart_ladD` [`LadderBands.lean`]

Let x be a walk, take the frame system of class U of x, and let τ be a top with $E_\tau = (a, b)$, base y, height H and $B = \mathrm{band}(H)$.

(a) (Homes.) *For $a < t < b$, $y \le \mathrm{home}(t) \le x(\tau)$ and $\beta(t) \le B$.*

(b) (The climb.) *For $1 \le d < H$, $\mathrm{band}(d - 1) \le \beta(L_\tau(y + d)) \le B$.*

(c) (The descent.) *For $1 \le d < H$, $\mathrm{ub}(H, d) \le \beta(D_\tau(y + d)) \le B$.*

(d) (Restarted rungs.) *If $1 \le d < H$ and $2^{\lceil \log_2 d \rceil} < H$, then the segment of the innermost frame at $D_\tau(y + d)$ starts after $L_\tau(y + d)$.*

Outline of the formal proof. Let $F = \mathrm{inner}(\tau)$. Call a time *free* if no frame of τ or of a top whose excursion lies in E_τ contains it (`Free`, `frame_cases`); ladder times are free (Lemma 3.4(d)). For $a < t < b$, t is an own time of F if and only if it is free (`own_of_free`, `free_of_own`), and the free restart points are the passages (`restart_inside`, `passage_spec`). The excursion's segment starts at $\max(f_1, a)$, at the cell y or $y + 1$: F starts at or before a, or it is a descent frame starting at $a + 1$ (`inner_fst_cases`, `excStart_spec`, `x_excStart`). It is the segment of F at the climb ladder times, at τ, and at the descent ladder times above the top passage (`segStart_eq_excStart`, `segStart_ladD_of_not`); below it, the segment at $D_\tau(y + d)$ starts at the passage $D_\tau(y + 2^{\lceil \log_2 d \rceil})$ (`segStart_ladD`). (a) Strictly inside the excursion every segment starts in $[a, t]$ (`segStart_inner_mem`), so homes and own times lie in $[y, x(\tau)]$ (`runMax_le_height`). (b) At $L_\tau(y + d)$ the walk is at distance at least $d - 1$ from the home (Lemma 3.6(e)). (c) Below the top passage the home is $y + 2^{\lceil \log_2 d \rceil}$; above it the segment contains the own time τ, at distance at least $H - 1$ from the home. (d) The passage follows τ. □

Lemma 3.9 (Lemma W2: suspend frames end at band 0)**.** Lean: `beta_last_eq_zero`, `runMax_last_le_one`, `struct_exc`, `own_of_le_level` [`LadderBands.lean`]

Let x be a walk, and take the frame system of class U of x. For every nonempty suspend frame $[f_1, f_2)$, $\beta(f_2 - 1) = 0$.

Outline of the formal proof. It suffices that the running maximum at $\ell = f_2 - 1$ is at most 1. Let w be the level of $J = [f_1, f_2)$; on J the walk is at least w, and $x(f_2) \le w$ (`frameAt_walk`), so $x(\ell) \in \{w, w+1\}$. Times of J at the level w are own times of J (`own_of_le_level`), and the upper hops at the cell w in the structure are canonical excursions with frame J (`struct_exc`). If $x(\ell) = w$ (`runMax_last_le_one`), either the walk stays at w on J, or the segment at ℓ started at the lowest passage $D_E(w+1)$ of the last excursion E of the structure, or, if $H_E = 1$, at the start of E or one step later; as the descent structures of E are suspended, the own times of the segment are at the levels w and $w+1$. If $x(\ell) = w+1$, the last excursion E ends at f_2, and ℓ is a time of E of height 1, the restart $D_E(w+1)$, or the last time of the descent frame of E at level $w+1$, to which the first case applies. □

Lemma W2 is the hypothesis `FrameSystem.PopsAtZero` of view invariance (Proposition 5.10), for the frame systems of all heads (`popsAtZero_headFrames`).

3.5 Frame weight

The label counts segment starts as events, weighted by 2^{scale}, where the scale of a segment is the band of its maximum. Lemma W1 bounds the total weight of each class of each walk by T.

Lemma 3.10 (Lemma W1: frame weight)**.** Lean: `sup_add_one_le_card`, `FrameSystem.segOf`, `FrameSystem.segments`, `FrameSystem.segTimes`, `FrameSystem.segMax`, `FrameSystem.sum_segWeight_le` [`FrameWeight.lean`]

Lean: `own_step`, `exists_outer_subframe`, `own_frame_end`, `segMax_add_one_le`, `sum_segWeight_upperFrames_le`, `card_levels`, `card_framesOf_le`, `card_suspendFrames_le` [`FrameWeightU.lean`]

Let $\mathcal{S}$ be a frame system. The segment *of a time $t < T$ is $(\mathrm{inner}(t), s_{\mathrm{inner}(t)}(t))$, and the segments of $\mathcal{S}$ are the segments of these times. The* own times *of a segment (F, s) are the times $t < T$ whose segment it is, and its* maximum *$m(F, s)$ is the largest $|x(t) - x(s)|$ over its own times (0 if there is none).*

(a) If every segment (F, s) has $m(F, s)$ smaller than its number of own times, then the weights $2^{\mathrm{band}(m(F,s))}$ of all segments add up to at most T.

(b) The frame system of class U of a walk satisfies the hypothesis of (a), so its segment weights add up to at most T.

(c) A walk has at most $2T$ suspend frames.

Outline of the formal proof. (a) The segments partition $[0, T)$, and $2^{\mathrm{band}(m)} \le m + 1$. (b) The *staircase* (`sup_add_one_le_card`): a function that vanishes at the least element of a finite set A, and at every other element is at most one above its value at some earlier element, has maximum at most $|A| - 1$. Consecutive own times of a frame are at most one level apart (`own_step`): they are consecutive, or the times between them form a frame of a top whose excursion lies directly in the frame (`exists_outer_subframe`), and they are the rungs $L(w-1), L(w)$ or $D(w), D(w-1)$ around it (`x_frame_ends`, `own_frame_end`). The first own time of a segment is at its home: it is s, or, if s is the start of F but not an own time, the end of the outermost hanging frame that starts there, where the walk is again at its level (`x_hangFrame_fst`). (c) A top of height H has $H - 1$ levels (`card_levels`), hence at most $2(H-1)$ frames (`card_framesOf_le`), and the heights add up to at most T (Lemma 3.2(d)). □

3.6 Kept and lost hops

The following definitions apply to every frame system $\mathcal{S}$ with walk x and horizon T, and every hop $e = (p, u)$ of x, with owner τ and cell $z = x(p)$. In Theorem A the frame system is that of the hop's class.

Definition 3.11 (Kept and lost hops)**.** Lean: `bandP`, `bandU`, `ownerScale`, `homeBand`, `segScale`, `pushMax`, `FrameMicro`, `Lost`, `cstar` [`HopData.lean`]
Lean: `Aligned`, `Ground`, `hopCost` [`CostModel.lean`]

The *data* of the hop are the bands $\beta_p = \beta(p)$ and $\beta_u = \beta(u)$; the *owner scale* $\sigma_e = \mathrm{band}(\mathrm{prom}(\tau))$; the *home band* $\lambda_e = \mathrm{band}(S_e)$, where $S_e = \max_{p \le c \le u} |z - \mathrm{home}(c)|$; the largest scale π_e of a segment that starts in $(p, u]$, that is, the maximum of $\mathrm{band}(m(F, s))$ over the frames F and the times $s \in (p, u]$ that are the start or a restart of F (0 if there are none); and whether the hop is *frame-micro*, that is, whether the segment of u started at or before p: $s_{\mathrm{inner}(u)}(u) \le p$. Let $h_0 \ge 1$ and $r < h_0$, with G and θ as in Definition 2.10. The hop is

- *aligned* at r if $G(\beta_p) = G(\beta_u) =: d$, $\lambda_e < \theta(d+1) + 2$ (it is *home-near*: $S_e < 2^{\theta(d+1)+2}$), and it is not frame-micro or $\theta(d) \le \sigma_e$ or $\theta(d) \le \pi_e$ (it is *eventful*);
- *ground* at r if $G(\sigma_e) = 0$;
- *kept* at r if it is aligned or ground, and *lost* at r otherwise.

Its *cost* is

$$c_e = \big(\max(\beta_p, \beta_u) - \min(\beta_p, \beta_u)\big) + \big(\lambda_e - 2 - \max(\beta_p, \beta_u)\big)^+ + [\text{frame-micro}] \cdot \big(\beta_u - \max(\sigma_e, \pi_e)\big)^+,$$

where $a^+ = \max(a, 0)$ and $[\cdot] \in \{0, 1\}$, and its *capped cost* is $c_e^* = \min(c_e, \sigma_e)$.

By Lemma V (Lemma 4.3), a kept hop is cut only if the display is truncated, changes above the hop's deciding group d, or a digit wraps. In one group, both ends read the same digit; home-nearness places the cell within $2^{\theta(d+1)+2}$ of every home active during the hop, which bounds the number of kept hops that span a time (Section 6); eventfulness provides an event of scale at least $\theta(d)$ inside the hop (Lemma 3.13). A ground hop has extent less than $2^{\theta(1)}$, by Proposition 2.9 (`sub_le_prom`). Lost hops are cut, and Theorem A bounds their number.

Lemma 3.12 (The cost model)**.** Lean: `card_lost_le`, `hopCost_le_of_bands` [`CostModel.lean`]
Lean: `sum_card_lost_le_sum_cstar`, `sum_cstar_le_of_ladders` [`HopData.lean`]

Let $h_0 \in \mathbb{N}$.

(a) For all values of the data $(\beta_p, \beta_u, \sigma, \lambda, \pi)$ and of frame-micro, the hop is lost for at most $\min(c, \sigma)$ of the offsets $r < h_0$, where c is its cost.

(b) If $\beta_p, \beta_u, \lambda \le B$, and $\beta_u \le \max(\sigma, \pi)$ in case the hop is frame-micro, then $c \le B - \min(\beta_p, \beta_u)$.

(c) For every frame system and every finite set $\mathcal{H}$ of hops, $\sum_{r<h_0} \#\{e \in \mathcal{H} : e \text{ lost at } r\} \le \sum_{e \in \mathcal{H}} c_e^*$.

(d) Let f be a function on hops and $C \in \mathbb{N}$. If $\sum_{e \in \mathrm{ladder}(\tau)} f(e) \le C\, H_\tau$ for every top τ of a map $x \colon \mathbb{N} \to \mathbb{Z}$, then $\sum_e f(e) \le C \cdot \#\{\text{upper hops of } x\}$, the sum running over the upper hops of x.

Outline of the formal proof. (a) A union bound with Lemma 2.11. A hop that is neither aligned nor ground has differing groups at its ends, for at most $\max - \min$ offsets; or equal groups and $\theta(G(\max(\beta_p, \beta_u)) + 1) \le \lambda - 2$, for at most $(\lambda - 2 - \max)^+$ offsets; or it is frame-micro with $\max(\sigma, \pi) < \theta(G(\beta_u))$, for at most $(\beta_u - \max(\sigma, \pi))^+$ offsets. And a hop that is not ground has $G(\sigma) \ge 1$, for at most σ offsets. (b) The last term vanishes, and $(\lambda - 2 - \max)^+ \le B - \max$; so $c \le B - \min$. (c) Exchange the sums and apply (a). (d) Group the upper hops by their tops and use $\sum_\tau H_\tau = \#\{\text{upper hops}\}$ (`sum_height_eq`). □

So, for a uniform offset, a hop is lost with probability at most c_e^*/h_0. The cap σ_e matters for hops whose owner is not prominent: their ends may have unrelated bands, but they can be lost only if a grid floor lies at or below the owner's scale.

Events. The *events* of a frame system are the *turning points*, the times $s < T$ with $\mathrm{prom}(s) \ge 1$, of scale $\mathrm{band}(\mathrm{prom}(s))$, and the *segment starts*, the pairs (F, s) of a frame F and a time $s < T$ that is the start or a restart of F, of scale $\mathrm{band}(m(F, s))$. $N_\theta(t)$ is the number of events at times at most t of scale at least θ (`eventCount`); it is nondecreasing in t (`eventCount_mono`). The *deciding group* of a hop at the offset r is $G(\beta_u)$ if the hop is aligned at r, and 0 otherwise (`decGroup`).

Lemma 3.13 (Validity)**.** Lean: `turnCount`, `segStarts`, `segEventCount`, `eventCount`, `eventCount_mono`, `decGroup`, `owner_turn`, `eventCount_lt_decGroup` [`Events.lean`]

Let $\mathcal{S}$ be a frame system with walk x and horizon T, let (p, u) be a hop of x that is an upper hop, and let $h_0, r \in \mathbb{N}$. If d is the deciding group of the hop at r, then $N_{\theta(d)}(p) < N_{\theta(d)}(u)$.

Outline of the formal proof. The owner $\tau \in (p, u)$ has $\mathrm{prom}(\tau) \ge x(\tau) - x(p) \ge 1$ by Proposition 2.9 (`owner_turn`), so it is a turning point of scale σ_e. If the hop is not aligned, $d = 0$ and $\theta(0) = 0 \le \sigma_e$. If it is aligned, eventfulness provides the event: if it is not frame-micro, the segment of u starts in $(p, u]$, a segment start of scale at least $\beta(u) \ge \theta(G(\beta_u))$ (`beta_le_segScale`, Lemma 2.11); if $\theta(d) \le \sigma_e$, the owner serves; if $\theta(d) \le \pi_e$, the owner serves when $\theta(d) = 0$, and otherwise a segment start in $(p, u]$ of scale $\pi_e \ge \theta(d)$. □

Lemma V needs this only for kept hops; the formal statement holds for every upper hop, since a hop that is not aligned has deciding group 0. For class L it applies to the frame system of $-x$.

3.7 The per-excursion bound

We add up the capped costs along one ladder. The arithmetic is separate from the walk: the rung at the level $d \ge 1$ above the base costs at most the deficits $B - \mathrm{band}(d - 1)$ and $B - \mathrm{ub}(H, d)$ of its ends, and the hop at the base at most B.

Lemma 3.14 (The ladder sum)**.** Lean: `uband`, `sum_range_two_pow_sub_band`, `sum_sub_band_two_pow_clog`, `ladder_sum_le_six`, `sum_range_le_six_of_ladder` [`PerExcursion.lean`]

Subtraction of natural numbers is truncated at 0.

(a) (A dyadic block.) *For $C, e \in \mathbb{N}$ with $e \le C + 1$, $\sum_{w<2^e}(C - \mathrm{band}(w)) + 2 = 2^e(C + 2 - e)$.*

(b) (The descent up to a passage.) *For $C, e \in \mathbb{N}$ with $e \le C + 1$, $\sum_{d=1}^{2^e}(C - \mathrm{band}(2^{\lceil \log_2 d \rceil} - d)) + 2e + 4 = 2^e(C + 4 - e)$.*

(c) (The ladder sum.) *For every $H \in \mathbb{N}$, with $B = \mathrm{band}(H)$,*

$$B + \sum_{d=1}^{H-1} \Big((B - \mathrm{band}(d-1)) + (B - \mathrm{ub}(H,d)) \Big) \le 6H.$$

(d) Consequently, if $c\colon \mathbb{N} \to \mathbb{N}$ satisfies $c(0) \le B$ and $c(d) \le (B - \mathrm{band}(d-1)) + (B - \mathrm{ub}(H,d))$ for $1 \le d < H$, then $\sum_{d<H} c(d) \le 6H$.

Outline of the formal proof. (a) Induction on e: the levels $w \in [2^e, 2^{e+1})$ have band e (`band_eq_of_mem_Ico`). (b) Induction on e: the levels $d \in (2^e, 2^{e+1}]$ have $\lceil \log_2 d \rceil = e+1$ (`clog_eq_of_lt_of_le`), and $w = 2^{e+1} - d$ maps them onto the block of (a). (c) For $H \le 1$ by inspection. Otherwise let $e = \mathrm{band}(H-1)$, so that $2^e \le H - 1 < 2^{e+1}$, $B \in \{e, e+1\}$, and the top passage is at the level 2^e. The climb side is at most $\sum_{j=0}^{H}(B - \mathrm{band}(j)) = 2^{B+1} - 2$, by Lemma E (`sum_band_sub_eq`, Lemma 2.12). The descent side contributes $2^e(B + 4 - e) - 2e - 4$ for $d \le 2^e$, by (b) with $C = B$, and at most 1 for each $d \in (2^e, H)$, where $\mathrm{ub}(H,d) = \mathrm{band}(H-1) \ge B - 1$. If $B = e$, the total is at most $5 \cdot 2^e + H - e - 7 \le 6H$; if $B = e+1$, then $H = 2^{e+1}$ and the total is at most $5H - e - 6$. (d) Apply (c). □

Lemma 3.15 (The per-excursion bound)**.** Lean: `ownerScale_ladderHop`, `cstar_exc_le`, `cstar_ladder_le`, `sum_ladder_cstar_le` [`TheoremA.lean`]

Let x be a walk, take the frame system of class U of x, and let τ be a top of height H and base y, with $B = \mathrm{band}(H)$. Every hop of the ladder of τ has owner scale $\mathrm{band}(\mathrm{prom}(\tau))$, and:

(a) (The base hop.) *$c^*_{E_\tau} \le B$.*

(b) (The rungs.) *For $1 \le d < H$, the hop of the ladder at the cell $y + d$ has $c^* \le (B - \mathrm{band}(d-1)) + (B - \mathrm{ub}(H,d))$.*

(c) (The ladder.) *$\sum_{e \in ladder(\tau)} c^*_e \le 6H$.*

Outline of the formal proof. (b) The owner is τ, so $\sigma \ge B$ by Proposition 2.9 (`height_le_prom`). By Lemma 3.8, the bands at the ends lie between $\mathrm{band}(d-1)$, respectively $\mathrm{ub}(H,d)$, and B, and the home band is at most B. The hop is eventful: below the top passage it is not frame-micro (Lemma 3.8(d)), and above it $\beta_u \le B \le \sigma$. So $c \le B - \min(\beta_p, \beta_u)$ by Lemma 3.12(b). (a) If E_τ is the first excursion of a descent structure of a top P, then P is earlier and at least as high as τ; if it is the last excursion of a hanging structure of P, then P is later and strictly higher (`inner_fst_cases`, `inner_snd_cases`). In both cases the walk stays at or above y in between, so $\mathrm{prom}(\tau) \le H$ by Proposition 2.9 (`prom_le_of_left`, `prom_le_of_right`) and $c^* \le \sigma \le B$: this is where the cap is needed, as one end of the hop lies in the parent's frame. Otherwise the segments at the times of $[a,b]$ start in the excursion (`segStart_inner_mem_Icc`), the bands and the home band are at most B (`runMax_le_height`), and the hop is frame-micro only if $H = 1$, when $\beta_u = 0$: for $H \ge 2$ the passage $D_\tau(y+1)$ restarts the segment of b after a. (c) The hops at the cells $y + d$, $d < H$, are the ladder (Lemma 3.2(a)); apply Lemma 3.14(d). □

3.8 Theorem A

Theorem 3.16 (Theorem A: lost hops of one walk)**.** Lean: `thmA_upper`, `thmA_lower` [`TheoremA.lean`]

Let x be a walk, T a horizon and $h_0 \in \mathbb{N}$. Then

$$\sum_{r=0}^{h_0-1} \#\{\textit{upper hops of } x \textit{ lost at } r \textit{ in class } U\} \le 6 \cdot \#\{\textit{upper hops of } x\},$$
$$\sum_{r=0}^{h_0-1} \#\{\textit{lower hops of } x \textit{ lost at } r \textit{ in class } L\} \le 6 \cdot \#\{\textit{lower hops of } x\}.$$

That is, for $h_0 \ge 1$ and a uniformly random offset $r \in \{0, \dots, h_0 - 1\}$, the expected number of lost hops of each class is at most $6/h_0$ times the number of hops of that class.

Outline of the formal proof. For class U, Lemma 3.12(c) bounds the left side by $\sum_e c_e^*$ over the upper hops, and Lemma 3.12(d) with Lemma 3.15(c) bounds this by 6 times their number (`thmA_upper`). For class L, apply this to the walk $-x$, whose upper hops are the lower hops of x (Lemma 3.2(e), `thmA_lower`). □

Since a walk has at most T hops (Section 2.1), the expected number of lost hops of its two classes together is at most $6T/h_0$; the proof of the main theorem chooses h_0 so large that this is a small fraction of T (Section 7). Lemmas W1 and W2 are the other two inputs from this section; the kept hops are the subject of the following sections.

Remark 3.17 (No pause-freeness is needed)**.** Every result of this section assumes only that x is a walk; pauses are allowed. A walk may pause at its top (Example 3.3), and then the own times of an excursion's frame are not all ladder times; the proofs use only own times, ladder times and passages. Since the composition of the following sections does not use pause-freeness either, the main theorem is proved for walks with pauses directly, at the depth of k walks (`tcpAt_direct`), with no detour through pause-free walks.

4 The label

The label combines the walks. At every time t it shows a finite set of band groups, and each of the P highest shown groups g contributes one digit: the number of events of scale at least the floor $\theta(g)$ up to time t, shifted by a random offset and reduced modulo a radix K. The digits are ordered by their groups, the highest group first, and labels are compared as digit vectors in the lexicographic order. A hop whose deciding group is shown at both of its ends then increases the label unless its group is not among the P highest shown groups, the shown groups above it change, or a digit wraps: this is Lemma V (Lemma 4.3). Section 4.1 defines the label for arbitrary shown sets and counters. Section 4.2 specializes it to k walks, with the shown sets of the displays of Section 5, and reduces the cut of the walk DAG to the hops of the individual heads.

4.1 Shown sets and digits

Throughout this subsection, O maps every time $t \in \mathbb{N}$ to a finite set $O(t) \subseteq \mathbb{N}$ of *shown groups*, $\mathrm{cnt}(g,t) \in \mathbb{N}$ is a *counter* for every group g and time t, $K \ge 1$ is the *radix*, $P \in \mathbb{N}$ is the number of *digits*, and $\omega = (\omega_0, \dots, \omega_{P-1}) \in \mathbb{N}^P$ are the *digit offsets*. Vectors in $\mathbb{N}^P$ are compared in the lexicographic order, position 0 being the most significant.

Definition 4.1 (The label of a family of shown sets)**.** `Lean`: `topRank`, `digits`, `lexRank`, `label` [`Label.lean`]

The *rank* of $g \in \mathbb{N}$ in a finite set $S \subseteq \mathbb{N}$, counted from the top, is $\mathrm{rk}_S(g) = |\{g' \in S : g' > g\}|$; the largest element of S has rank 0. The *digits* at time t are, for $i < P$,

$$\delta_i(t) = \begin{cases} (\mathrm{cnt}(g,t) + \omega_i) \bmod K & \text{if } g \in O(t) \text{ and } \mathrm{rk}_{O(t)}(g) = i, \\ 0 & \text{if no element of } O(t) \text{ has rank } i. \end{cases}$$

The *lexicographic rank* $\mathrm{lex}(a)$ of a vector $a \in \mathbb{N}^P$ is the number of vectors $y \in \{0, \dots, K-1\}^P$ that are lexicographically smaller than a. The *label* at t is $\Psi(t) = \mathrm{lex}(\delta_0(t), \dots, \delta_{P-1}(t))$.

By Lemma 4.2(a) below, at most one element of $O(t)$ has rank i; Lean writes $\delta_i(t)$ as the sum of $(\mathrm{cnt}(g,t) + \omega_i) \bmod K$ over the elements g of rank i. We say that the digit at position i *wraps* for the group g between the times p and u if

$$\lfloor (\mathrm{cnt}(g,p) + \omega_i)/K \rfloor \neq \lfloor (\mathrm{cnt}(g,u) + \omega_i)/K \rfloor.$$

If $\mathrm{cnt}(g,p) \le \mathrm{cnt}(g,u)$ and the digit does not wrap, then $(\mathrm{cnt}(g,\cdot) + \omega_i) \bmod K$ does not decrease from p to u, and it increases strictly if the counter does (`BoundedNesting.digit_le_of_no_wrap`, `BoundedNesting.digit_lt_of_no_wrap`).

Lemma 4.2 (Ranks and lexicographic ranks)**.** Lean: `topRank_lt_of_lt`, `lt_of_topRank_lt`, `topRank_injOn`, `topRank_lt_card`, `topRank_eq_of_above`, `toLex_lt_of_prefix`, `lexRank_le_of_le`, `lexRank_lt_of_lt`, `lexRank_lt_pow`, `digits_lt`, `label_lt` [`Label.lean`]

Let $S, S' \subseteq \mathbb{N}$ be finite.

(a) If $g < g'$ and $g' \in S$, then $\mathrm{rk}_S(g') < \mathrm{rk}_S(g)$; conversely, if $g \in S$ and $\mathrm{rk}_S(g') < \mathrm{rk}_S(g)$, then $g < g'$. Hence rk_S is injective on S, and $\mathrm{rk}_S(g) < |S|$ for every $g \in S$.

(b) If $d \le g$ and $S \cap (d, \infty) = S' \cap (d, \infty)$, then $\mathrm{rk}_S(g) = \mathrm{rk}_{S'}(g)$: the rank of a group depends only on the elements above it.

(c) If $a \le b$ lexicographically, then $\mathrm{lex}(a) \le \mathrm{lex}(b)$; if $a < b$ lexicographically and all entries of a are below K, then $\mathrm{lex}(a) < \mathrm{lex}(b)$. If $a_{i'} \le b_{i'}$ for all $i' < i$ and $a_i < b_i$, then $a < b$ lexicographically.

(d) Every digit is below K, and the label takes fewer than K^P values.

Outline of the formal proof. (a) Every element of S above g' is above g, and g' itself is above g but not above g' (`topRank_lt_of_lt`); the converse follows, since $g' = g$ gives equal ranks and $g' < g$ the opposite inequality (`lt_of_topRank_lt`). So two elements of S with the same rank are equal (`topRank_injOn`), and the elements above $g \in S$ form a subset of S that misses g (`topRank_lt_card`). (b) The elements of S above $g \ge d$ are the elements of $S \cap (d, \infty)$ above g (`topRank_eq_of_above`). (c) Every vector below a is below b; in the strict case a itself is counted for b and not for a (`lexRank_le_of_le`, `lexRank_lt_of_lt`). For the last statement, a and b first differ at a position $i' \le i$, and there $a_{i'} < b_{i'}$ (`toLex_lt_of_prefix`). (d) A digit is a residue modulo K or 0 (`digits_lt`), and the vectors below the digit vector form a subset of $\{0, \dots, K-1\}^P$ that misses the digit vector itself (`lexRank_lt_pow`, `label_lt`). □

Lemma 4.3 (Lemma V)**.** Lean: `label_lt_label`, `trunc_or_above_or_wrap` [`Label.lean`]

Let $p, u, d \in \mathbb{N}$ with $d \in O(p) \cap O(u)$. Suppose that $\mathrm{cnt}(g,p) \le \mathrm{cnt}(g,u)$ for every $g \in O(p)$ with $g \ge d$, and that $\mathrm{cnt}(d,p) < \mathrm{cnt}(d,u)$. If

(i) $\mathrm{rk}_{O(p)}(d) < P$,

(ii) $O(p) \cap (d, \infty) = O(u) \cap (d, \infty)$, *and*

(iii) *for every* $g \in O(p)$ *with* $g \geq d$ *and* $i = \mathrm{rk}_{O(p)}(g) < P$, *the digit at position* i *does not wrap for* g *between* p *and* u,

then $\Psi(p) < \Psi(u)$. *Consequently, if* $\Psi(u) \leq \Psi(p)$, *then one of the following holds:*

(TRUNC) $\mathrm{rk}_{O(p)}(d) \geq P$;

(ABOVE) $O(p) \cap (d, \infty) \neq O(u) \cap (d, \infty)$;

(WRAP) *for some* $g \in O(p)$ *with* $g \geq d$, *the digit at position* $i = \mathrm{rk}_{O(p)}(g) < P$ *wraps for* g *between* p *and* u.

Outline of the formal proof. Let $i_0 = \mathrm{rk}_{O(p)}(d) < P$. By Lemma 4.2(c) it suffices that $\delta_i(p) \leq \delta_i(u)$ for $i < i_0$ and $\delta_{i_0}(p) < \delta_{i_0}(u)$ (`toLex_lt_of_prefix`, `lexRank_lt_of_lt`). Let $i < i_0$. If no element of $O(p)$ has rank i, then $\delta_i(p) = 0$. Otherwise the element g of rank i lies above d by Lemma 4.2(a), so $g \in O(u)$ by (ii), and g has rank i in $O(u)$ as well by Lemma 4.2(b); its counter does not decrease and its digit does not wrap, so $\delta_i(p) \leq \delta_i(u)$ (`BoundedNesting.digit_le_of_no_wrap`). At position i_0, the group d has rank i_0 at both ends by Lemma 4.2(b) with $g = d$; its counter increases strictly and its digit does not wrap, so $\delta_{i_0}(p) < \delta_{i_0}(u)$ (`BoundedNesting.digit_lt_of_no_wrap`, `label_lt_label`). The second statement is the contrapositive (`trunc_or_above_or_wrap`). □

Lemma V is where the order of the digits matters. The groups above d occupy the leading positions, so nothing that happens below d affects the comparison, provided the shown groups above d stay the same and their digits do not wrap. In Section 4.2, (p, u) is an upper hop of a head, d is its deciding group, and the strict increase of the counter of d is validity (Lemma 3.13). The three exceptions are bounded separately: (TRUNC) by the size of the display (Lemma D, Lemma 5.7), (ABOVE) by view invariance and exposure (Propositions 5.10 and 5.11), and (WRAP) by the randomness of the digit offsets (Lemma 4.5).

Lemma 4.4 (State edges)**.** Lean: `label_le_label_of_eq`, `shown_ne_or_wrap` [`Label.lean`]
Let $p, u \in \mathbb{N}$ *with* $\mathrm{cnt}(g, p) \leq \mathrm{cnt}(g, u)$ *for every* $g \in O(p)$. *If* $O(p) = O(u)$, *and for no* $g \in O(p)$ *the digit at position* $\mathrm{rk}_{O(p)}(g) < P$ *wraps for* g *between* p *and* u, *then* $\Psi(p) \leq \Psi(u)$. *Consequently, if* $\Psi(u) < \Psi(p)$, *then* $O(p) \neq O(u)$, *or the digit at position* $\mathrm{rk}_{O(p)}(g) < P$ *wraps for some* $g \in O(p)$ *between* p *and* u.

Outline of the formal proof. Since $O(p) = O(u)$, each position holds at both ends the digit of the same group, and this digit does not decrease (`BoundedNesting.digit_le_of_no_wrap`), or it holds 0 at p. So the digit vector does not decrease in any coordinate, hence not lexicographically, and Lemma 4.2(c) applies (`label_le_label_of_eq`). The second statement is the contrapositive (`shown_ne_or_wrap`). □

With $p = t - 1$ and $u = t$, the lemma says that a *descent* at t, a cut state edge $t \to t - 1$, requires a change of the shown set at t or a wrap at t (Lemma 6.5).

Lemma 4.5 (Wraps over the digit offsets)**.** Lean: `card_wrap_offsets_le` [`Label.lean`]
Let $i < P$ *and* $n_1 \leq n_2$. *Of the* K^P *vectors* $\omega \in \{0, \ldots, K-1\}^P$, *at most* $(n_2 - n_1)K^{P-1}$ *satisfy* $\lfloor (n_1 + \omega_i)/K \rfloor \neq \lfloor (n_2 + \omega_i)/K \rfloor$.

Outline of the formal proof. The condition depends only on ω_i, and the vectors whose ith coordinate lies in a set of m values number mK^{P-1} (`BoundedNesting.Fam.card_piFinset_coord`). At most $n_2 - n_1$ values $c < K$ have $\lfloor (n_1 + c)/K \rfloor \neq \lfloor (n_2 + c)/K \rfloor$ (`BoundedNesting.Fam.card_wrap_le`): if $n_2 - n_1 < K$, the residue of $n_1 + c$ modulo K is then one of the $n_2 - n_1$ largest residues, and it determines c. Both counts are lemmas of the eighth paper's library. □

So, for uniform digit offsets, the digit of a group whose counter rises by Δ wraps with probability at most Δ/K; this is how the radix K pays for the wraps (Lemma 6.4).

4.2 The label of k walks

Fix $k \in \mathbb{N}$, walks $x_1, \dots, x_k \colon \mathbb{N} \to \mathbb{Z}$ with $|x_i(t+1) - x_i(t)| \leq 1$ for all t (pauses are allowed), a horizon T, and a grid (h_0, r) with groups G and floors θ (Definition 2.10); put $n = 2k$. Recall the *events* of a frame system $\mathcal{S}$ with walk x and horizon T (Lemma 3.13): its *turning points*, the times $s < T$ with $\mathrm{prom}(s) \geq 1$, of *scale* $\mathrm{band}(\mathrm{prom}(s))$; and its *segment starts*, the pairs (F, s) of a frame F and a time $s < T$ that is the start of F or a restart of F, of scale $\mathrm{band}(m(F, s))$, where $m(F, s)$ is the largest distance $|x(t) - x(s)|$ over the own times t of the segment (F, s) (`segStarts`, `segScale`). The counter $N_\theta(t)$ is the number of events at times at most t of scale at least θ (`eventCount`); it does not decrease in t (`eventCount_mono`). The *deciding group* $d_r(e)$ of an upper hop $e = (p, u)$ of x is $G(\beta(u))$ if e is aligned at offset r (Definition 3.11), and 0 otherwise (`decGroup`). By validity (Lemma 3.13), $N_{\theta(d_r(e))}(p) < N_{\theta(d_r(e))}(u)$ for every upper hop e, kept or not.

Definition 4.6 (The label of k walks)**.** Lean: `headOf`, `headWalk`, `headFrames`, `globalCount`, `shown`, `labelOf` [`Global.lean`]

(a) *Heads.* The $n = 2k$ *heads* are the pairs (i, j) of a walk $i < k$ and a *class* $j \in \{0, 1\}$, indexed by $\{0, \dots, n-1\}$ through Mathlib's bijection `finProdFinEquiv` (`headOf`). The head $(i, 0)$, of class U, sees the walk x_i, and the head $(i, 1)$, of class L, sees $-x_i$ (`headWalk`, through the eighth paper's `BoundedNesting.sw`). Write $x^{(v)}$ for the walk seen by head v. Head v carries the frame system $\mathcal{S}_v$ of class U of $x^{(v)}$ on $[0, T)$ (Definition 3.7, `headFrames`), so class L uses the frames of $-x_i$; β_v is the band of $\mathcal{S}_v$, and $g_v(t) = G(\beta_v(t))$ is the *band group* of head v.

(b) *Counters.* With $N_\theta^{(v)}(t)$ the counter of $\mathcal{S}_v$, the *global counter* $\bar{N}_\theta(t) = \sum_v N_\theta^{(v)}(t)$ counts the events of all heads up to t of scale at least θ; the turning points of x_i and of $-x_i$ are counted separately.

(c) *Shown sets.* For bits ξ, one value in $\{0, \dots, Q-1\}$ for every frame of every head, $\mathcal{O}_{r,\xi}(t)$ is the shown set at t of the composed display of the heads (Definition 5.3). It contains 0 and the band groups $g_v(t)$ of all heads.

(d) *The label.* For a radix $K \geq 1$, a number of digits P and digit offsets $\omega \in \mathbb{N}^P$, $\Psi_{r,\omega,\xi}$ is the label of Definition 4.1 for the shown sets $\mathcal{O}_{r,\xi}$ and the counters $\mathrm{cnt}(g, t) = \bar{N}_{\theta(g)}(t)$: the digit of a shown group g counts the events of all heads whose scale reaches the floor of g.

The heads, frames, events and counters depend only on the walks. The shown sets depend on the offset r, through the band groups and the tops of entries, and on the bits; the label depends in addition on the digit offsets. So the label of a time t is determined by the walks, r, ω and ξ, and Section 6 averages over these three random choices.

Lemma 4.7 (The label of k walks)**.** Lean: `labelOf_lt`, `hops_subset_heads`, `card_violated_le_heads`, `decGroup_mem_shown`, `trunc_or_above_or_wrap_head` [`Global.lean`]

Let $K \geq 1$.

(a) *$\Psi_{r,\omega,\xi}$ takes fewer than K^P values.*

(b) *Every long hop of x_i is an upper hop of the head $(i,0)$ or of the head $(i,1)$.*

(c) *For every map $\Psi\colon \mathbb{N} \to \mathbb{N}$, the number of violated hop edges of the walk DAG of $x_1, \dots, x_k$ before T is at most the sum over the heads v of the number of upper hops (p,u) of $x^{(v)}$ with $\Psi(u) \le \Psi(p)$.*

(d) *Let $e = (p,u)$ be an upper hop of head v, and let $d = d_r(e)$ be its deciding group in $\mathcal{S}_v$. Then d is shown at p and at u, and if $\Psi_{r,\omega,\xi}(u) \le \Psi_{r,\omega,\xi}(p)$, then (TRUNC), (ABOVE) or (WRAP) of Lemma 4.3 holds for $O = \mathcal{O}_{r,\xi}$ and the counters $\mathrm{cnt}(g,t) = \bar{N}_{\theta(g)}(t)$.*

Outline of the formal proof. (a) is Lemma 4.2(d) (`labelOf_lt`). (b) A long hop is an upper or a lower hop (`upperHops_union_lowerHops`), and the lower hops of x_i are the upper hops of $-x_i$ (`upperHops_neg`, `headWalk_up`, `headWalk_down`). (c) A violated hop edge $u \to p$ comes from a long hop (p,u) of some x_i; a pair that is a long hop of several walks is a single edge (`BoundedNesting.hopPairs`). By (b) it is an upper hop of a head of x_i with $\Psi(u) \le \Psi(p)$, and a union bound over the heads gives the claim. (d) If e is aligned at offset r, then $d = G(\beta_v(u)) = G(\beta_v(p))$ is the band group of v at both ends, a seed of the composed display (`headGroup_mem_shown`); otherwise $d = 0$, which is shown at all times (`zero_mem_shown`, `decGroup_mem_shown`). The global counters do not decrease (`globalCount_mono`), and $\bar{N}_{\theta(d)}(p) < \bar{N}_{\theta(d)}(u)$, because the counter $N^{(v)}_{\theta(d)}$ of $\mathcal{S}_v$ alone increases strictly by validity (Lemma 3.13) and is a summand of the global counter (`globalCount_lt_of_head`). So Lemma 4.3 applies (`trunc_or_above_or_wrap_head`). □

Every violated upper hop of a head is therefore lost (Definition 3.11) or, if it is not lost, satisfies (TRUNC), (ABOVE) or (WRAP). Theorem A (Theorem 3.16) pays for the lost hops. Section 5 constructs the shown sets and provides the tools for (TRUNC) and (ABOVE), and Section 6 adds up all terms.

5 Displays with break bits

This section constructs the shown sets of the label: at every time, the band groups of all heads together with a *display* of the frozen groups of suspended frames. Section 5.1 explains informally why the display is random. Section 5.2 defines displays at one time, Section 5.3 the entries of a frame system and the composed display of all heads, and Section 5.4 proves that the display is small (Lemma D). The last three subsections control the case (ABOVE) of Lemma V: the display above a group is a function of a *view* (Section 5.5), the view changes only at standard events (Proposition W, Section 5.6), and a hop without a standard event above its group is exposed only through a broken bit (Section 5.7).

5.1 Why break bits

This subsection is an informal explanation; nothing below depends on it.

By Lemma V, an upper hop (p,u) of a head with deciding group d is violated only through truncation, a wrap, or the case (ABOVE): the shown groups above d differ at p and at u. The shown sets must contain the band groups of all heads, since the deciding group of an aligned hop is the band group of its head at both ends. Band groups alone are not enough. When a head enters a frame (a *push*), the band of the suspended segment freezes, and when the frame ends (a *pop*) the segment resumes at its frozen band. If only the current band groups were shown, a pop at a high

band would add a group to the shown set, and hops of other heads spanning the pop, with lower deciding groups, would fall into (ABOVE). The display therefore also shows frozen groups, the *tops* of the *entries* of Definition 5.2.

The question is which tops to show. To keep the shown groups above d independent of what happens below d, an entry should be displayed as soon as its *window*, the range of groups between the current activity inside it and its top, contains a shown value. With this rule the display above d is a function of the configuration above d (Lemma 5.8), and that part of the configuration changes only at standard events, which are paid for (Proposition 5.10). But the rule cascades: an entry displayed because its window contains a displayed top makes the next entry whose window contains its own top displayed, and so on up the stacks of suspended frames. When the stacks are deep, so is the cascade, and no bounded number of digits can carry the display.

Random *break bits* cut the cascade. Every entry carries a bit that is 0, making the entry *broken*, with probability $1/Q$, independently of the other bits, and the cascade passes only through unbroken entries. From a value y it moves to the tops of the *candidates* at y, at most one per head, and it stops as soon as all the candidates it meets are broken; each step reveals at most n new bits, so long cascades are unlikely (Lemma D, Lemma 5.7). The display above d still depends only on the configuration above d, except through the candidates at d: when these are unbroken and d is shown, they are displayed exactly as under the cascading rule. So a hop can be exposed only through one of at most $2n$ candidates, n at each end, being broken, which has probability at most $2n/Q$ (Proposition 5.11).

5.2 Displays at one time

A *configuration* consists of a finite set I of *entries*, each with a *head* $h(e) \in \{0, \dots, n-1\}$, a *bottom* $m(e) \in \mathbb{N}$ and a *top* $c(e) \in \mathbb{N}$; a set $U \subseteq I$ of *unbroken* entries; and a finite set $\Sigma \subseteq \mathbb{N}$ of *seeds*. The *window* of e is the set of groups $[m(e), c(e)]$, which is empty if $m(e) > c(e)$. An entry e is a *candidate* at a value y if $m(e) \le y < c(e)$.

Definition 5.1 (Displays)**.** `Lean`: `Disp`, `OneCandidate`, `dispVals`, `shownSet` [`Display.lean`]
The *display* $\mathcal{D}$ of a configuration is the least set of entries with the following three properties.

(Z) If $m(e) = 0$, then $e \in \mathcal{D}$.

(S) If $e \in U$, $z \in \Sigma$ and $m(e) \le z \le c(e)$, then $e \in \mathcal{D}$.

(C) If $e \in U$, $e' \in \mathcal{D}$ and $m(e) \le c(e') \le c(e)$, then $e \in \mathcal{D}$.

The *displayed values* are the seeds and the tops of the displayed entries (`dispVals`), and the *shown set* is $\mathcal{O} = \{0\} \cup \Sigma \cup \{c(e) : e \in \mathcal{D}\}$ (`shownSet`). The configuration has the *candidate property* if for every head v and every value y at most one entry e with $h(e) = v$ is a candidate at y (`OneCandidate`).

Rule (Z) displays the *fresh* entries, whose window starts at 0, whatever their bits; rules (S) and (C) are the cascade, through unbroken entries only. In Lean, $\mathcal{D}$ is an inductive predicate with one constructor for each rule. An entry with an empty window is never displayed. The only structural fact about stacks that Lemma D uses is a consequence of the candidate property: every value has at most n candidates (`cands`, `card_cands_le`).

5.3 Entries and the composed display

Let $\mathcal{S}$ be a frame system on $[0,T)$ with walk x (Definition 3.5), and fix a grid (h_0, r). A frame $F = (f_1, f_2)$ is the interval $[f_1, f_2)$, and $F' \subseteq F$ denotes inclusion of intervals. We write $\mathrm{inner}(t)$ for the innermost frame of t, and $s_F(t)$ and $M_F(t)$ for the segment start and the running maximum of a frame F at t (`FrameSystem.segStart`, `FrameSystem.runMax`); the band is $\beta(t) = \mathrm{band}(M_{\mathrm{inner}(t)}(t))$. The *segment of F current at t* is the segment of F that starts at $s_F(t)$.

Definition 5.2 (Entries, tops and bottoms)**.** Lean: `FrameSystem.parent`, `FrameSystem.IsEntry`, `FrameSystem.headGroup`, `FrameSystem.entryTop`, `FrameSystem.innerEntries`, `FrameSystem.entryBot` [`Entries.lean`]

(a) The *parent* $\pi(F)$ of a frame F is a shortest frame strictly containing F, and the root if there is none. For a nonempty frame other than the root, $\pi(F)$ lies inside every frame strictly containing F (`FrameSystem.parent_within`).

(b) A frame F is an *entry* if it is nonempty, it is not the root, and its *parent segment*, the segment of $\pi(F)$ current at f_1, is innermost again after F ends: some time $t' \geq f_2$ is an own time of $\pi(F)$ with $s_{\pi(F)}(t') = s_{\pi(F)}(f_1)$. The entries containing t form the *stack* at t.

(c) The *band group* at t is $g(t) = G(\beta(t))$.

(d) The *top* of a frame F is $c_F = G\big(\mathrm{band}(M_{\pi(F)}(f_1))\big)$, the group of the band at which the parent segment is suspended.

(e) The *bottom* of F at t is

$$m_F(t) = \max\big(\{g(t)\} \cup \{c_{F'} : F' \text{ an entry with } F' \subseteq F,\ F' \neq F,\ t \in F'\}\big),$$

and $[m_F(t), c_F]$ is the *window* of F at t.

An entry is a *suspend child*: a frame after which its parent segment resumes. For a nonempty frame other than the root that is not an entry, the parent segment is never innermost again from the start of the frame on. While an entry is open, the running maximum of its parent does not change, because the times of the entry are not own times of the parent (Lemma 3.6); so its top is the group of the frozen band of the parent segment, the band group with which that segment resumes, unless its running maximum rises at that time. The bottom starts at the band group of the head and rises with the tops of the entries stacked inside; the window becomes empty when the activity inside F rises above c_F. Lean chooses $\pi(F)$ among the frames of minimal length; only nonempty frames are entries, and their parents are determined.

Definition 5.3 (The composed display)**.** Lean: `EntryIdx`, `dispBot`, `dispTop`, `seeds` [`Composition.lean`]

Let $\mathcal{S}_0, \ldots, \mathcal{S}_{n-1}$ be frame systems on $[0,T)$, one for each head, with a common grid (h_0, r), and write g_v, $c_F^{(v)}$ and $m_F^{(v)}$ for the band groups, tops and bottoms of $\mathcal{S}_v$. The *composed display* at time t is the configuration whose entries are the pairs (v, F) of a head v and a frame F of $\mathcal{S}_v$ (`EntryIdx`); ι denotes their number. The entry (v, F) has head v and top $c_F^{(v)}$; its bottom is $m_F^{(v)}(t)$ if F is an entry of $\mathcal{S}_v$ containing t, and $c_F^{(v)} + 1$ otherwise. The seeds are the band groups $g_v(t)$ of all heads. Bits ξ, one value $\xi_{(v,F)} \in \{0, \ldots, Q-1\}$ for every entry, decide the unbroken entries: those with $\xi_{(v,F)} \neq 0$.

The bits do not depend on t. A pair (v, F) whose frame is not on the stack of v at t has an empty window, so it is neither a candidate nor displayed at t; the composed display lives on the stacks of the heads.

Lemma 5.4 (One candidate per head)**.** Lean: `FrameSystem.entryTop_le_entryBot`, `FrameSystem.entry_candidate_unique` [`Entries.lean`]
Lean: `onStack_of_cand`, `card_seeds_le`, `oneCandidate_disp` [`Composition.lean`]

(a) If $F' \subseteq F$ are distinct entries of $\mathcal{S}$ containing t, then $c_{F'} \le m_F(t)$.

(b) If F and F' are entries of $\mathcal{S}$ containing t with $m_F(t) \le y < c_F$ and $m_{F'}(t) \le y < c_{F'}$, then $F = F'$.

(c) At every time, the composed display has the candidate property and at most n seeds.

Outline of the formal proof. (a) holds by the definition of the bottom as a maximum (`FrameSystem.entryTop_le_entryBot`). (b) Two frames containing t are nested, by laminarity (Definition 3.5). If $F' \subseteq F$ and $F' \neq F$, then $y < c_{F'} \le m_F(t) \le y$ by (a), a contradiction, and symmetrically (`FrameSystem.entry_candidate_unique`). (c) A candidate has a nonempty window, so its frame is an entry of its head's stack and its bottom is the true bottom (`onStack_of_cand`); apply (b) within one head (`oneCandidate_disp`). The seeds are the n band groups (`card_seeds_le`). □

Along the stack of a head, inner windows lie below outer bottoms. This is why the candidates at any value are few: at most one per head, and at most n in all.

5.4 Lemma D: the display is small

Fix a configuration with entries I in which every value has at most n candidates; this holds for every configuration with n heads and the candidate property (`card_cands_le`). Let a bit vector $\xi \in \{0, \dots, Q-1\}^I$ decide the unbroken entries, $U = \{e : \xi_e \neq 0\}$. Probability bounds are counts of bit vectors: a bound "for at most N bit vectors" means probability at most $N/Q^{|I|}$ for uniformly random bits.

The *front* from a value z is the sequence $y_0 = z$ and

$$y_{i+1} = \min\{c(e) : e \text{ an unbroken candidate at } y_i\},$$

with $y_{i+1} = y_i$ if there is no unbroken candidate at y_i (`front`, `frontStep`). The front *runs* at step i if some candidate at y_i is unbroken (`Running`); then $y_i < y_{i+1}$ (`lt_frontStep`). A front that does not run at step i stays at y_i from then on (`front_add_of_not_running`), so the steps at which it runs form an initial segment (`Running.anti`). Let Σ_0 be the set of the tops $c(e) \ge 1$ of the entries with $m(e) = 0$ (`zeroTops`); these entries are candidates at 0, so $|\Sigma_0| \le n$ (`card_zeroTops_le`). The *new candidates* of step i are the candidates at y_i that are not candidates at any $y_{i'}$ with $i' < i$ (`newCands`).

The front reads the bits adaptively: which bits it reads next depends on the bits read so far. The following lemma is the probabilistic tool for such processes.

Lemma 5.5 (Adaptive revealing)**.** Lean: `RevAgree`, `IsRevealing`, `card_le_pow_mul_card_zero`, `card_le_pow_mul_card_block`, `card_blocks_le` [`Revealing.lean`]
Let I be a finite set and $Q \ge 1$. For every $i \in \mathbb{N}$ and every bit vector $\xi \in \{0, \dots, Q-1\}^I$ let $R_i(\xi) \subseteq I$ be the set of entries revealed *in round i. Suppose that the rounds are* adaptive*, $R_i(\xi') = R_i(\xi)$*

whenever ξ' agrees with ξ on $R_{i'}(\xi)$ for all $i' < i$, and fresh, *$R_i(\xi) \cap R_{i'}(\xi) = \emptyset$ for $i' < i$. If every round reveals at most μ entries, then for all $a, \ell, j \in \mathbb{N}$ at most*

$$(1 - Q^{-\ell\mu})^j \, Q^{|I|}$$

bit vectors have the property that each of the j blocks of rounds $[a + b\ell, a + b\ell + \ell)$, $b < j$, contains a round that reveals an unbroken entry.

Outline of the formal proof. Zeroing (`card_le_pow_mul_card_zero`). Let A be a set of bit vectors determined by the rounds before i: if $\xi \in A$ and ξ' agrees with ξ on these rounds (`RevAgree`), then $\xi' \in A$. Setting the bits of round i to 0 maps A into itself and keeps the earlier rounds and round i itself, by adaptivity and freshness. The vectors with a given image differ only on round i, so there are at most Q^μ of them. Hence $|A| \le Q^\mu |\{\xi \in A : \text{round } i \text{ reveals only broken entries}\}|$, and for ℓ consecutive rounds the factor is $Q^{\ell\mu}$ (`card_le_pow_mul_card_block`). *Blocks* (`card_blocks_le`), by induction on j. Let E_j be the set of bit vectors whose first j blocks each reveal an unbroken entry; it is determined by the rounds before $a + j\ell$. By zeroing, at least a fraction $Q^{-\ell\mu}$ of E_j has block j all broken, and these vectors are not in $E_{j+1} \subseteq E_j$; so $|E_{j+1}| \le (1 - Q^{-\ell\mu})|E_j|$, and $|E_0| \le Q^{|I|}$. □

Lemma 5.6 (The front process)**.** Lean: `exists_front_eq_top`, `disp_top`, `shownSet_subset`, `card_shownSet_le`, `isRevealing_newCands`, `card_newCands_le`, `card_ucands_succ_lt`, `card_ucands_add_le`, `exists_newCands_unbroken`, `card_running_le` [`DisplayTail.lean`]

Suppose that every value has at most n candidates.

(a) Closure. *Every element of the shown set is 0 or a value y_i of the front from some $z \in \Sigma \cup \Sigma_0$. Hence, if no front from a value of $\Sigma \cup \Sigma_0$ runs at step N_0, the shown set has at most $1 + |\Sigma \cup \Sigma_0|(N_0 + 1)$ elements.*

(b) For each z, the new candidates of the steps of the front from z form an adaptive and fresh revealing scheme with at most n entries per round. A front that runs at step $a + n$ has an unbroken new candidate at one of the steps $a + 1, \ldots, a + n$.

(c) For all z and j, at most $(1 - Q^{-n^2})^j Q^{|I|}$ bit vectors make the front from z run at step jn.

Outline of the formal proof. (a) The key fact (`exists_front_eq_top`): an unbroken candidate e at y_i has its top among the later values of the front. Indeed $y_{i+1} \le c(e)$, and if $y_{i+1} < c(e)$, then e is still an unbroken candidate at y_{i+1}; induct on $c(e) - y_i$. Now induct over the rules of the display (`disp_top`). An entry displayed by (Z) has top 0 or a top in Σ_0, the value y_0 of its own front. An entry e displayed by (S) through a seed z has $c(e) = z$ or is an unbroken candidate at $z = y_0$. An entry e displayed by (C) through e', whose top is 0 or a front value y_i by induction, has $c(e) = c(e')$, or $m(e) = 0$ as under (Z), or is an unbroken candidate at y_i. The seeds are the values y_0 of their own fronts (`shownSet_subset`). A front that does not run at step N_0 is constant from then on, so it takes at most $N_0 + 1$ values (`card_shownSet_le`). (b) The first i steps of the front read only the bits of the candidates at $y_0, \ldots, y_{i-1}$ (`front_congr`), and each of these is a new candidate at some earlier step (`exists_mem_newCands`); this gives adaptivity, and freshness holds by definition (`isRevealing_newCands`). The new candidates of a step are candidates at one value, so there are at most n of them (`card_newCands_le`). If the front runs at step i and the new candidates of step $i + 1$ are all broken, then the unbroken candidates at y_{i+1} form a proper subset of those at y_i (`card_ucands_succ_lt`): an unbroken candidate e at y_{i+1} is not new, so it is a candidate at some $y_{i'}$ with $i' \le i$, and hence at y_i, because $m(e) \le y_{i'} \le y_i \le y_{i+1} < c(e)$; and the candidate whose top is y_{i+1} is no longer a candidate at y_{i+1}. A running front has at least one and

at most n unbroken candidates, so n consecutive steps whose new candidates are all broken stop it (`card_ucands_add_le`, `exists_newCands_unbroken`). (c) If the front runs at step jn, it runs at step $bn+n$ for every $b < j$ (`Running.anti`), and by (b) each block of steps $[1+bn, 1+bn+n)$ reveals an unbroken entry. Lemma 5.5 with $a = 1$ and $\ell = \mu = n$ gives the bound (`card_running_le`). □

Lemma 5.7 (Lemma D: the display is small)**.** Lean: `card_shownSet_tail` [`DisplayTail.lean`]
Suppose that every value has at most n candidates, and let $j \in \mathbb{N}$. At most $(|\Sigma|+n)(1-Q^{-n^2})^j Q^{|I|}$ bit vectors give a shown set with more than $1+(|\Sigma|+n)(jn+1)$ elements. Equivalently, for uniformly random bits,

$$\Pr\big[|\mathcal{O}| > 1 + (|\Sigma|+n)(jn+1)\big] \le (|\Sigma|+n)(1-Q^{-n^2})^j.$$

Outline of the formal proof. If the shown set is larger, then some front from a value of $\Sigma \cup \Sigma_0$ runs at step jn: otherwise Lemma 5.6(a) with $N_0 = jn$ bounds the shown set by $1 + |\Sigma \cup \Sigma_0|(jn+1)$, and $|\Sigma \cup \Sigma_0| \le |\Sigma| + n$. A union bound over the at most $|\Sigma| + n$ starting values, with Lemma 5.6(c), gives the claim. □

The bound depends neither on the number of entries nor on the depth of the stacks. The seeds are the values y_0 of their own fronts, so they need no separate term. The composed display has the candidate property and $|\Sigma| \le n$ (Lemma 5.4), so every value has at most n candidates (`card_cands_le`) and a time has more than $1 + 2n(jn+1)$ shown groups with probability at most $2n(1-Q^{-n^2})^j$ (Lemma 6.3); this is what the number of digits P of Definition 7.1 is chosen for. The rate $1-Q^{-n^2}$ per block of n steps comes from asking all of the at most n^2 new bits of a block to be broken; it is the reason why P, and with it the depth, is so large.

5.5 The display above a group

Fix a configuration and a value d. An entry e is *visible* above d if $d < m(e) \le c(e)$; a candidate at d has $m(e) \le d < c(e)$. The *view* above d consists of the set X_d of the seeds above d and the tops of the candidates at d (`aboveVals`), and the set V_d of the windows $(m(e), c(e))$ of the unbroken visible entries (`visPairs`). A value is *reachable* from X_d through V_d if it belongs to the least set that contains X_d and contains c whenever it contains a value y with $m \le y \le c$ for some $(m, c) \in V_d$ (`Reach`).

Lemma 5.8 (The display above d)**.** Lean: `aboveVals`, `visPairs`, `Reach`, `disp_of_cand`, `mem_dispVals_iff_reach`, `shown_above_eq` [`DisplayView.lean`]

(a) A candidate at d is displayed if its bottom is 0, or if it is unbroken and d is a seed.

(b) If every candidate at d is displayed, then a value $y > d$ is displayed if and only if it is reachable from X_d through V_d.

(c) Consider two configurations on the same set of entries, with the same view above d, and in each of which every candidate at d is displayed. Then their shown sets have the same elements above d.

Outline of the formal proof. (a) Rules (Z) and (S), the latter with the seed d (`disp_of_cand`). (b) (`mem_dispVals_iff_reach`) A seed above d lies in X_d. The top of a displayed entry e with $c(e) > d$ is reachable, by induction over the rules: if $m(e) \le d$, then e is a candidate and $c(e) \in X_d$; otherwise e is unbroken and visible, displayed by (S) through a seed z or by (C) through an entry e', and z, resp. $c(e')$, lies in the window of e and above d, hence is reachable, so that $c(e)$ is reachable in one more step. Conversely, by induction over reachability: the elements of X_d are seeds or tops of

candidates, and these are displayed by hypothesis; and if a displayed value lies in the window of an unbroken visible entry e, then e is displayed by (S) or (C). (c) Apply (b) to both configurations (`shown_above_eq`). □

So, once the candidates at d are displayed, the display above d is a function of the view above d. The view is local to the heads: it consists of their band groups above d, the tops of their candidates at d and their visible windows. The next subsection shows that the part of the view belonging to one head changes only at standard events of that head.

5.6 View invariance

Let $\mathcal{S}$ be a frame system on $[0, T)$ with a grid (h_0, r), and let $d \in \mathbb{N}$. Its entries, tops, bottoms and band groups are those of Definition 5.2.

Definition 5.9 (Standard events and views)**.** Lean: `FrameSystem.RiseAt`, `FrameSystem.LastStintAt`, `FrameSystem.PopsAtZero`, `FrameSystem.stackEntries`, `FrameSystem.headAbove`, `FrameSystem.headVisible` [`ViewInvariance.lean`]

Let $1 \le t < T$ and $g \ge 1$.

(a) There is a *rise* into g at t if t is an own time of $J = \mathrm{inner}(t)$, after the start of J and not a restart of J, at which the running maximum of J crosses the floor of g: $\mathrm{band}(M_J(t-1)) < \theta(g) \le \mathrm{band}(M_J(t))$.

(b) There is a *last-stint end* at t at the group g if, with $J = \mathrm{inner}(t-1)$, $\theta(g) \le \mathrm{band}(M_J(t-1))$ and the segment of J current at $t-1$ is never innermost again from t on: no own time $t' \ge t$ of J has $s_J(t') = s_J(t-1)$.

(c) Rises and last-stint ends are the *standard events*; t is the *time* and g the *group* of the event.

(d) $\mathcal{S}$ *pops at band zero* if every nonempty frame F other than the root ends at band 0: $\beta(f_2 - 1) = 0$.

(e) The *view* of $\mathcal{S}$ above d at any time $t \in \mathbb{N}$ consists of the set $\mathcal{A}_d(t)$ of the values above d among the band group $g(t)$ and the tops c_F of the entries F of the stack with $m_F(t) \le d < c_F$, the *candidates* at d (`FrameSystem.stackEntries`); and the set $\mathcal{V}_d(t)$ of the pairs $(F, m_F(t))$ for the entries F of the stack with $d < m_F(t) \le c_F$, the *visible* entries.

The frame system of class U of every walk pops at band zero (Lemma 3.9); so do the frame systems $\mathcal{S}_v$ of the heads (`popsAtZero_headFrames`).

Proposition 5.10 (Proposition W: view invariance)**.** Lean: `FrameSystem.view_step`, `FrameSystem.view_interval` [`ViewInvariance.lean`]

Suppose that $\mathcal{S}$ pops at band zero and $r < h_0$, and let $p \le u < T$. If no standard event of a group $g \ge d+1$ has its time in $(p, u]$, then the view above d is the same at p and at u: $\mathcal{A}_d(p) = \mathcal{A}_d(u)$ and $\mathcal{V}_d(p) = \mathcal{V}_d(u)$.

Outline of the formal proof. The statement for $u = p + 1$ is `FrameSystem.view_step`, and `FrameSystem.view_interval` follows by induction on u. Fix t with $1 \le t < T$ and no standard event of a group $g \ge d + 1$ at t. A frame containing $t - 1$ or t is *common* if it contains both, *popped* if it contains only $t - 1$ (it ends at t), and *pushed* if it contains only t (it starts at t); popped and pushed frames lie strictly inside every common frame (`FrameSystem.within_of_contains_not`). Let C be the innermost common frame

(`FrameSystem.InnermostCommon`, `FrameSystem.exists_innermostCommon`; the root is common), and let $c = G(\mathrm{band}(M_C(t-1)))$.

Popped and pushed entries. If some frame is popped at t, the innermost frame at $t-1$ is popped too, so $g(t-1) = 0$ because $\mathcal{S}$ pops at band zero (`FrameSystem.headGroup_pred_eq_zero`). No entry lies strictly inside a popped entry on the stack at $t-1$: it would end at t, and its parent, which lies inside the popped entry, would have no own time from t on. So popped entries have bottom 0 at $t-1$ (`FrameSystem.entryBot_popped`). If some frame is pushed at t, the innermost frame at t starts at t, so $g(t) = 0$; an entry strictly inside a pushed entry and containing t starts at t together with its parent, so its top is the group of a running maximum at the start of a frame, which is 0 (`FrameSystem.headGroup_eq_zero_of_push`, `FrameSystem.entryTop_eq_zero`). So pushed entries have bottom 0 at t (`FrameSystem.entryBot_pushed`). Hence popped and pushed entries are never visible, and they are candidates at d exactly when their tops exceed d.

Children of C. The parent of a popped entry is C: it contains $t-1$, and it contains t because it has an own time after the entry ends (`FrameSystem.parent_eq_of_popped`). The parent of a pushed frame is C or starts at t (`FrameSystem.parent_of_pushed`), and in the second case the pushed frame has top 0. A popped or pushed child of C has top c, since the running maximum of C is frozen while C is suspended (`FrameSystem.runMax_frozen`), and it is an entry if and only if the segment of C current at $t-1$ *returns*, that is, is innermost again at some time $t' \geq t$ (`FrameSystem.Returns`, `FrameSystem.popped_child`, `FrameSystem.pushed_child`).

Local values. The *local values* at $t-1$ are the band group $g(t-1)$ and the tops of the entries popped at t; the local values at t are $g(t)$ and the tops of the entries pushed at t (`FrameSystem.LocalPre`, `FrameSystem.LocalPost`). By the above, a positive value is a local value at $t-1$ if and only if it equals c and C is innermost at $t-1$ or its segment returns; and it is a local value at t if and only if either C is innermost at t and the value equals $G(\mathrm{band}(M_C(t)))$, or C is not innermost at t, its segment returns, and the value equals c (`FrameSystem.localPre_iff`, `FrameSystem.localPost_iff`). The key claim is that the local values at $t-1$ and at t agree above d (`FrameSystem.localPre_iff_localPost`). First, if C is innermost at $t-1$ and its segment does not return, then $c \leq d$: otherwise the last stint of that segment ends at t at the group $c \geq d+1$, whose floor lies within its band. If C is innermost at t and restarts there, its band group at t is 0 and its old segment does not return (`FrameSystem.not_returns_of_restart`), so neither side has a value above d. If C is innermost at t without a restart, its segment returns at t itself, and its group can only rise, from c to $c' = G(\mathrm{band}(M_C(t)))$; a rise to $c' > \max(c, d)$ would be a rise into c' at t, so c and c' agree above d. If C is not innermost at t, so that some frame is pushed, both sides are $\{c\}$ when the segment returns; otherwise only the side at $t-1$ can contain c, which is then at most d.

Common entries. The bottom of a common entry at $t-1$, resp. at t, is the largest of the tops of the common entries stacked inside it and of the local values at $t-1$, resp. at t (`FrameSystem.entryBot_pre_le_iff`, `FrameSystem.entryBot_post_le_iff`). Since the local values agree above d, $\max(d, m_F(t-1)) = \max(d, m_F(t))$ for every common entry F (`FrameSystem.max_entryBot_eq`): it is a candidate at d at both times or at neither, and if it is visible, its bottom does not change.

The view. A value above d in $\mathcal{A}_d(t-1)$ is a local value at $t-1$ (the band group, or the top of a popped entry, which is a candidate) or the top of a common candidate (`FrameSystem.above_pre_iff`), and likewise at t (`FrameSystem.above_post_iff`); so $\mathcal{A}_d(t-1) = \mathcal{A}_d(t)$. A visible entry at $t-1$ is not popped, since popped entries have bottom 0; so it is common, and visible at t with the same bottom. Conversely for t, and so $\mathcal{V}_d(t-1) = \mathcal{V}_d(t)$. □

Pushes and pops of entries cost nothing in this proof. At a push, the top of the pushed entry,

which is a candidate, equals the band group of the suspended segment that it replaces; at a pop, the top of the popped entry is replaced by the equal band group of the resumed segment, unless the running maximum rises at the resumption, which is a rise. What remains are the rises and last-stint ends above d. A segment rises into a given group at most once and ends its last stint at most once, and on average over the offset these events are rare (Lemma 6.1).

5.7 Exposure

Let $\mathcal{S}_0, \dots, \mathcal{S}_{n-1}$ be frame systems on $[0, T)$ with the composed display at offset r (Definition 5.3), let bits ξ decide the unbroken entries, and let $d \in \mathbb{N}$. Write $\mathcal{A}_d^{(v)}(t)$ and $\mathcal{V}_d^{(v)}(t)$ for the view of $\mathcal{S}_v$ above d at t, and $\mathcal{O}_\xi(t)$ for the shown set of the composed display at t.

Proposition 5.11 (Exposure)**.** Lean: `mem_aboveVals_iff`, `mem_visPairs_iff`, `views_eq`, `exposed_of_ne`, `card_exposed_le` [`Exposure.lean`]

Let $p, u \in \mathbb{N}$.

(a) The view of the composed display is assembled from the views of the heads: at every time t, X_d is the union of the sets $\mathcal{A}_d^{(v)}(t)$, and V_d consists of the windows $(m, c_F^{(v)})$ of the unbroken entries (v, F) with $(F, m) \in \mathcal{V}_d^{(v)}(t)$. Hence, if every head has the same view above d at p and at u, so does the composed display, whatever the bits.

(b) Suppose that d is a seed at p and at u, and that every head has the same view above d at p and at u. If $\mathcal{O}_\xi(p) \cap (d, \infty) \neq \mathcal{O}_\xi(u) \cap (d, \infty)$, then some candidate at d, at p or at u, is broken.

(c) Under the hypotheses of (b), at most $2nQ^{\iota-1}$ of the Q^ι bit vectors give $\mathcal{O}_\xi(p) \cap (d, \infty) \neq \mathcal{O}_\xi(u) \cap (d, \infty)$: for uniformly random bits, the probability is at most $2n/Q$.

Outline of the formal proof. (a) A candidate at d and a visible entry have nonempty windows, so their frames are entries of the stacks of their heads, and their bottoms are the bottoms of Definition 5.2 (`mem_aboveVals_iff`, `mem_visPairs_iff`); the seeds are the band groups. The bits do not depend on the time (`views_eq`). (b) (`exposed_of_ne`) If every candidate at d, at p and at u, is unbroken, then each of them is displayed, d being a seed (Lemma 5.8(a)); by (a) the two configurations have the same view above d, and Lemma 5.8(c) gives equal shown sets above d. (c) By the candidate property (Lemma 5.4) there are at most n candidates at d at each time (`cands`, `card_cands_le`). The bit vectors that break a given entry are determined by their other $\iota - 1$ coordinates, so there are $Q^{\iota-1}$ of them (`card_some_broken_le`); a union bound over the at most $2n$ candidates gives the claim (`card_exposed_le`). □

In the accounting (Lemma 6.7), $p < u$ are the ends of an upper hop of a head v, neither lost nor charged, so that no standard event above its deciding group d lies in $(p, u]$ and, by Proposition 5.10, every head has the same view above d at p and at u. If the hop is aligned, d is the band group of v at both ends, a seed, and the proposition applies. If it is not aligned, $d = 0$: every candidate at 0 has bottom 0 and is displayed by (Z), so by Lemma 5.8(c) the shown sets above 0 agree for all bits, and the hop is never exposed.

6 Accounting

This section bounds the cut of the label of Definition 4.6, summed over the random choices. Fix walks $x_1, \dots, x_k$, pauses allowed, a horizon T, $n = 2k$, the heads v with their walks $x^{(v)}$ and frame systems $\mathcal{S}_v$, and parameters $h_0, K, P, Q \in \mathbb{N}$ with $K, Q \geq 1$. The random choices are an offset

$r \in [0, h_0)$, digit offsets $\omega \in [0, K)^P$, and bits $\xi \in [0, Q)^\iota$, one for each entry of the composed display (Definition 5.3). Their set Ω has $|\Omega| = h_0 K^P Q^\iota$ elements, and an expectation over uniform choices is a sum over Ω divided by $|\Omega|$; the Lean statements are the sums. The cut F_Ψ of a label Ψ consists of its descents and its violated hop edges (`labelCutSize`), and by Lemma 4.7(c) the violated hop edges are at most the violated upper hops of the heads. We bound the descents (Lemma 6.5) and, for each head, its violated upper hops (Lemma 6.7); Lemma 6.8 collects the totals that Section 7 adds up.

6.1 Events and standard events

Two kinds of events enter the accounting. The *events* of Section 4.2, the turning points and segment starts of the heads, do not depend on the random choices. They drive the counters $N_\theta^{(v)}$ and $\bar{N}_\theta = \sum_v N_\theta^{(v)}$ (`eventCount`, `globalCount`), and for $p \le u$ the increase $N_\theta^{(v)}(u) - N_\theta^{(v)}(p)$ is the number of events of head v in $(p, u]$ of scale at least θ (`turnCount_sub`, `segEventCount_sub`). The *standard events* of Definition 5.9, rises and last-stint ends, depend on the offset r; they are what changes the views. For a head v and an offset r, let $\mathrm{Rise}_r(v)$ be the set of pairs (t, g) with $t, g < T$ such that $\mathcal{S}_v$ has a rise into g at t, and $\mathrm{Last}_r(v)$ the set of pairs (t, g) with $1 \le t < T$ and $g < T$ such that $\mathcal{S}_v$ has a last-stint end at t at the group g (`FrameSystem.rises`, `FrameSystem.lastStints`). The restriction $g < T$ loses nothing, by Lemma 6.1(e).

A standard event belongs to a segment: a rise at t to the segment of t, and a last-stint end at t to the segment of $t - 1$. Recall from Lemma 3.10 that a segment (F, s) has the maximum $m(F, s)$, the largest distance $|x(t) - x(s)|$ over its own times, and that for the frame system of class U of a walk $\sum_{(F,s)} 2^{\mathrm{band}(m(F,s))} \le T$, the sum running over the segments. A segment start (F, s) has the scale $\mathrm{band}(m(F, s))$, which is 0 if (F, s) has no own times (`FrameSystem.segScale_eq`). The *weight* of a standard event of group g is $2^{\theta(g)}$, and the weight of an event of scale s is 2^s.

Lemma 6.1 (Budgets)**.** Lean: `FrameSystem.rise_unique`, `FrameSystem.lastStint_unique`, `FrameSystem.flr_le_of_rise`, `FrameSystem.flr_le_of_lastStint`, `FrameSystem.sum_two_pow_flr_le`, `FrameSystem.sum_offsets_top_floor_le`, `FrameSystem.sum_stdWeight_le` [`Budgets.lean`]
Lean: `sum_turns_weight_le`, `card_segStarts_le`, `sum_segStarts_weight_le`, `lt_of_rise`, `lt_of_lastStint` [`EventWeight.lean`]

Let $\mathcal{S}$ be a frame system with walk x and horizon T.

(a) A segment rises into a given group at most once, and its last stint ends at most once.

(b) The floor of a standard event lies within the scale of its segment: $\theta(g) \le \mathrm{band}(m(F, s))$.

(c) For every $N \in \mathbb{N}$, the standard events with groups below N, weighted by $2^{\theta_r(g)}$ and summed over the offsets $r < h_0$, total at most $8 \sum_{(F,s)} 2^{\mathrm{band}(m(F,s))}$, the sum running over the segments of $\mathcal{S}$.

(d) If x is a walk, the turning points of x weigh $\sum_s 2^{\mathrm{band}(\mathrm{prom}(s))} \le 2(T - 1)$; $\mathcal{S}$ has at most $\#\mathit{frames} + T$ segment starts; and the segment starts weigh at most $\sum_{(F,s)} 2^{\mathrm{band}(m(F,s))}$ plus their number.

(e) If x is a walk and $r < h_0$, every standard event at a time $t < T$ has its group below T.

Outline of the formal proof. (a) Within a segment the running maximum does not decrease (`FrameSystem.runMax_mono_of_segStart_eq`). If $t < t'$ were rises into g of one segment, with running maximum M, then $\mathrm{band}(M(t' - 1)) < \theta(g) \le \mathrm{band}(M(t)) \le \mathrm{band}(M(t' - 1))$ (`FrameSystem.rise_unique`). If $t < t'$ were last-stint ends of one segment, then $t' - 1 \ge t$ would be

an own time of the same frame with the same segment start, contrary to the last-stint end at t (`FrameSystem.lastStint_unique`). (b) The floor is at most the band of the running maximum, which is at most the scale of the segment (`beta_le_segScale`, `FrameSystem.flr_le_of_rise`, `FrameSystem.flr_le_of_lastStint`). (c) By (a) and (b), at each offset the rises inject into the pairs of a segment (F, s) and a group $g \geq 1$ with $\theta(g) \leq \mathrm{band}(m(F, s))$, and so do the last-stint ends (`FrameSystem.sum_rises_le`, `FrameSystem.sum_lastStints_le`). The floors $\theta(g)$, $g \geq 1$, are distinct, so the weights of those at most s add up to at most $2 \cdot 2^{\theta(G(s))}$ if $G(s) \geq 1$, and to 0 otherwise (`FrameSystem.sum_two_pow_flr_le`). Over the offsets with $G_r(s) \geq 1$, the top floors $\theta_r(G_r(s))$ are distinct positive integers at most s (Lemma 2.11(e)), so $\sum_r 2 \cdot 2^{\theta_r(G_r(s))} \leq 4 \cdot 2^s$ (`FrameSystem.sum_offsets_top_floor_le`, `sum_two_pow_flr_grp_le`). The two kinds of standard events give $2 \cdot 4 = 8$ (`FrameSystem.sum_stdWeight_le`). (d) $2^{\mathrm{band}(m)} \leq m$ for $m \geq 1$, and the prominences add up to at most $2(T-1)$ by (P2) of Proposition 2.9 (`sum_prom_le_two_mul`). A segment start is a frame start (F, f_1) or a restart (F, s), and then $F = \mathrm{inner}(s)$; so there are at most $\#\mathrm{frames} + T$. A segment start with own times is a segment; one without has scale 0 and weight 1. (e) $g \leq \theta(g) \leq \mathrm{band}(M) \leq M \leq t$ for the running maximum M at the time of the event, since a walk moves at most one cell per step (`le_flr`, `band_le_self`, `runMax_le_self`). □

With Lemma 3.10, part (c) bounds the weighted standard events of a head by $8T$, summed over the offsets (Proposition 7.3(a)): on average over the grid, a head has $O(T/h_0)$ weighted standard events. Their weight $2^{\theta(g)}$ is exactly what an event costs in the counting facts below.

6.2 The counting facts

A standard event, or a wrapping digit, at a time c affects the hops that span c. The counting facts bound the hops that span a time and are *low*, in the sense that their deciding group lies below a given one. For a hop $e = (p, u)$ with owner τ we use the data of Definition 3.11: the owner scale $\sigma_e = \mathrm{band}(\mathrm{prom}(\tau))$ and the home band $\lambda_e = \mathrm{band}(S_e)$, where S_e is the largest distance from the cell $x(p)$ to a home active during the hop.

Lemma 6.2 (The counting facts)**.** Lean: `card_spanning_near_lt`, `card_aligned_spanning_lt`, `card_ground_spanning_lt` [`Accounting.lean`]

Lean: `card_kept_spanning_le` [`WrapGlobal.lean`]

Let $\mathcal{S}$ be a frame system with walk x and horizon T, let c be a time and r an offset; alignment, groundness, loss and deciding groups refer to $\mathcal{S}$ and r.

(a) For every $h \in \mathbb{Z}$ and $R \geq 1$, fewer than $2R$ hops (p, u) of x span c and have $|x(p) - h| < R$.

(b) For every $\theta^* \in \mathbb{N}$, fewer than 2^{θ^*+3} hops (p, u) of x span c, are aligned, and have $\theta(G(\beta(u)) + 1) \leq \theta^*$.

(c) If x is a walk, $h_0 \geq 1$ and $\theta^* \geq \theta(1)$, fewer than $2 \cdot 2^{\theta^*}$ upper hops of x span c and are ground.

(d) If x is a walk and $r < h_0$, then for every group g at most $10 \cdot 2^{\theta(g+1)}$ upper hops of x span c, are not lost, and have deciding group at most g.

Outline of the formal proof. (a) At each cell at most one hop spans c (`BoundedNesting.hop_unique`, Section 2), and the cells lie in the open interval $(h - R, h + R)$ of $2R - 1$ integers (`card_spanning_near_lt`). (b) An aligned hop with $d = G(\beta(u))$ is home-near: $\lambda_e < \theta(d + 1) + 2 \leq \theta^* + 2$. Since $c \in [p, u]$, the distance $|x(p) - \mathrm{home}(c)|$ is at most $S_e < 2^{\lambda_e+1} \leq 2^{\theta^*+2}$, and (a) applies with $h = \mathrm{home}(c)$ and $R = 2^{\theta^*+2}$ (`card_aligned_spanning_lt`). (c) A ground hop has $G(\sigma_e) = 0$, that is, $\sigma_e < \theta(1)$. Its extent is at most $x(\tau) - x(p)$ (`extent_le_owner`), which is at most

$\mathrm{prom}(\tau) < 2^{\sigma_e+1} \le 2^{\theta(1)} \le 2^{\theta^*}$ by (P1) of Proposition 2.9; by (P3) fewer than $2 \cdot 2^{\theta^*}$ hops of extent below 2^{θ^*} span c (`card_spanning_hops_lt`, `card_ground_spanning_lt`). (d) A hop that is not lost is aligned or ground. An aligned hop with deciding group $d \le g$ has $\theta(d+1) \le \theta(g+1)$ (`flr_mono`), so (b) with $\theta^* = \theta(g+1)$ counts fewer than $8 \cdot 2^{\theta(g+1)}$ of them, and (c) with the same $\theta^* \ge \theta(1)$ counts fewer than $2 \cdot 2^{\theta(g+1)}$ ground hops (`card_kept_spanning_le`). □

So a standard event of group g is charged with at most $10 \cdot 2^{\theta(g)}$ kept hops whose deciding group lies below g, ten times its weight. This is where alignment pays off: home-nearness confines the low aligned hops spanning a time to few cells, and the cap on the owner's scale confines the ground hops to short ones.

6.3 Truncation

For an offset r, bits ξ and $P \in \mathbb{N}$, a time $t < T$ is *bad* if $|\mathcal{O}_{r,\xi}(t)| > P$; let $\mathrm{Bad}_{r,\xi}$ be the set of bad times (`badTimes`).

Lemma 6.3 (Truncation)**.** Lean: `badTimes`, `card_trunc_le_bad`, `card_bad_le` [`Truncation.lean`]

(a) For every head v, the upper hops (p,u) of $x^{(v)}$ whose deciding group has rank at least P in $\mathcal{O}_{r,\xi}(p)$ are at most $|\mathrm{Bad}_{r,\xi}|$ in number.

(b) If $P \ge 1 + 2n(jn+1)$ for some $j \in \mathbb{N}$, then for every offset r and every time t, at most $2n(1-Q^{-n^2})^j Q^\iota$ of the Q^ι bit vectors make t bad.

Outline of the formal proof. (a) The deciding group d is shown at p (Lemma 4.7(d)), so its rank is below $|\mathcal{O}_{r,\xi}(p)|$ by Lemma 4.2(a), and a rank at least P makes p bad. Distinct hops of one walk have distinct starts (`BoundedNesting.hop_start_unique`), so the starts inject the truncated hops into the bad times (`card_trunc_le_bad`). (b) The composed display at t has the candidate property (Lemma 5.4), so every value has at most n candidates (`card_cands_le`), and it has at most n seeds. Lemma D (Lemma 5.7) applies, and $1 + (|\Sigma|+n)(jn+1) \le 1 + 2n(jn+1) \le P$ and $|\Sigma| + n \le 2n$ (`card_bad_le`). □

6.4 Wraps

A digit of a group g wraps between p and u only if its counter $\bar{N}_{\theta(g)}$ rises, that is, only if events of scale at least $\theta(g)$ occur in $(p,u]$; by Lemma 4.5 a rise by Δ makes the digit wrap for at most a fraction Δ/K of the digit offsets. The following lemma turns this into a bound by exchanging the sums over hops and over events.

Lemma 6.4 (Wraps)**.** Lean: `flr_le_iff`, `flr_grp_succ_le`, `sum_two_pow_flr_succ_le`, `sum_hops_events_le` [`Wrap.lean`]
Lean: `card_wrap_le_sum`, `sum_ten_two_pow_flr_le`, `sum_kept_family_le` [`WrapGlobal.lean`]

(a) (Exchange of sums.) *Let A be a finite set of events a with times $\tau(a)$ and scales $\sigma(a)$, H a finite set of hops $e = (p(e), u(e))$ with groups $d(e)$, and $B(e)$, for each $e \in H$, a finite set of groups at least $d(e)$. Let $\vartheta, \Gamma \colon \mathbb{N} \to \mathbb{N}$ satisfy $g \le \Gamma(s)$ whenever $\vartheta(g) \le s$. Then*

$$\sum_{e \in H} \sum_{g \in B(e)} \#\{a \in A : p(e) < \tau(a) \le u(e),\ \vartheta(g) \le \sigma(a)\} \le \sum_{a \in A} \sum_{g=0}^{\Gamma(\sigma(a))} \#\{e \in H : p(e) < \tau(a) \le u(e),\ d(e) \le g\}.$$

(b) (Wraps of one hop.) *Let $O, B \subseteq \mathbb{N}$ be finite and $c, c' \colon \mathbb{N} \to \mathbb{N}$ with $c(g) \le c'(g)$ for $g \in B$. At most $\sum_{g \in B} (c'(g) - c(g)) K^{P-1}$ vectors $\omega \in [0, K)^P$ have some $g \in B$ of rank $i = \mathrm{rk}_O(g) < P$ with $\lfloor (c(g) + \omega_i)/K \rfloor \neq \lfloor (c'(g) + \omega_i)/K \rfloor$.*

(c) (The kept hops of a head.) *Let $\mathcal{S}$ be a frame system whose walk x is a walk, let $r < h_0$, and let A be a finite set of events with times $\tau(a)$ and scales $\sigma(a)$. If, for every upper hop e of x, $B(e)$ is a finite set of groups at least the deciding group of e at r, then*

$$\sum_{e} \sum_{g \in B(e)} \#\{a \in A : p < \tau(a) \le u,\ \theta(g) \le \sigma(a)\} \le \sum_{a \in A} 20 \cdot 2^{h_0} \cdot 2^{\sigma(a)},$$

the first sum running over the upper hops $e = (p, u)$ of x that are not lost at r.

Outline of the formal proof. (a) Both sides count triples (e, g, a) with $p(e) < \tau(a) \le u(e)$ and $d(e) \le g$: the left side those with $g \in B(e)$ and $\vartheta(g) \le \sigma(a)$, the right side those with $g \le \Gamma(\sigma(a))$, and the first condition implies the second (`sum_hops_events_le`). (b) A union bound over $g \in B$ (`card_wrap_le_sum`): a group of rank at least P has no digit, and a group of rank $i < P$ is Lemma 4.5. (c) Apply (a) with $\vartheta = \theta_r$ and $\Gamma = G_r$, which is valid since $\theta(g) \le s$ if and only if $g \le G(s)$ (`flr_le_iff`). For each event a and each $g \le G(\sigma(a))$, the inner count is at most $10 \cdot 2^{\theta(g+1)}$ by Lemma 6.2(d) at the time $\tau(a)$. The floors $\theta(g+1)$, $g \le G(s)$, are distinct and at most $\theta(G(s)+1) \le s + h_0$ (`flr_grp_succ_le`), so $\sum_{g \le G(s)} 2^{\theta(g+1)} < 2^{s+h_0+1}$ (`sum_two_pow_flr_succ_le`), and $\sum_{g \le G(s)} 10 \cdot 2^{\theta(g+1)} \le 20 \cdot 2^{h_0} \cdot 2^s$ (`sum_ten_two_pow_flr_le`, `sum_kept_family_le`). □

The factor 2^{h_0} comes from the top group of an event, whose floor $\theta(G(s)+1)$ can exceed the event's scale s by up to h_0; the radix K of Definition 7.1 is chosen to pay for it.

6.5 The cut state edges

Let $r < h_0$. A time $t \in [1, T)$ is an *event time* if some head has a rise or a last-stint end at t of a group g with $1 \le g < T$ (`eventTimes`). For digit offsets ω and bits ξ, it is a *wrap time* if the digit of some $g \in \mathcal{O}_{r,\xi}(t-1)$, at its rank $i < P$ in $\mathcal{O}_{r,\xi}(t-1)$, wraps between $t-1$ and t for the counter $\bar{N}_{\theta(g)}$ (`wrapTimes`).

Lemma 6.5 (The cut state edges)**.** Lean: `eventTimes`, `wrapTimes`, `shown_eq_of_not_eventTime`, `card_descents_le`, `card_eventTimes_le`, `globalCount_sub_le`, `sum_card_wrapTimes_le` [`CutDescents.lean`]

Let $r < h_0$.

(a) (The shown set changes only at standard events.) *If $t \in [1, T)$ is not an event time, then $\mathcal{O}_{r,\xi}(t-1) = \mathcal{O}_{r,\xi}(t)$ for all bits ξ.*

(b) The descents of $\Psi_{r,\omega,\xi}$ are at most as many as the event times and the wrap times together.

(c) The event times number at most $\sum_v (|\mathrm{Rise}_r(v)| + |\mathrm{Last}_r(v)|)$.

(d) For all bits ξ, the wrap times, summed over the K^P digit offsets, number at most

$$PK^{P-1} \sum_v (\#\textit{turning points of } x^{(v)} + \#\textit{segment starts of } \mathcal{S}_v).$$

Outline of the formal proof. (a) This is view invariance at $d = 0$ (`shown_eq_of_not_eventTime`). The frame systems of the heads pop at band zero (Lemma 3.9, `popsAtZero_headFrames`), and every rise or last-stint end at t has its group below T (Lemma 6.1(e)); so if t is not an event time, no head has a standard event at t of a group $g \geq 1$, and Proposition 5.10 for one step gives the same view above 0 for every head at $t-1$ and t. By Proposition 5.11(a) the composed display has the same view above 0 at both times. Every candidate at 0 has bottom 0 and is displayed by rule (Z), so Lemma 5.8(c) gives the same shown values above 0, and 0 is shown at all times. (b) The counters do not decrease (`globalCount_mono`), so by Lemma 4.4 with $p = t-1$ and $u = t$ a descent at t changes the shown set, and then t is an event time by (a), or it is a wrap time (`card_descents_le`). (c) Every event time is the time of a rise or a last-stint end (`card_eventTimes_le`). (d) Fix t. At most P groups of $\mathcal{O}_{r,\xi}(t-1)$ have rank below P (Lemma 4.2(a)), and the counter of each rises at t by at most the number $E(t)$ of turning points and segment starts of all heads at t (`globalCount_sub_le`); by Lemma 6.4(b), at most $PK^{P-1}E(t)$ digit offsets make t a wrap time. Summed over t, every turning point and every segment start is counted at one time (`sum_card_fiber_le`, `sum_card_wrapTimes_le`). □

Part (a) is the reason why pushes and pops cost no state edges: the shown set changes only when a band group rises across a grid floor or a segment ends its last stint at a positive group, both standard events.

6.6 The violated hops

Fix a head v, an offset r, digit offsets ω and bits ξ, and write $\mathcal{O} = \mathcal{O}_{r,\xi}$. For an upper hop $e = (p, u)$ of $x^{(v)}$, let d be its deciding group at r in $\mathcal{S}_v$.

Definition 6.6 (The kinds of violated hops)**.** Lean: `lostSet`, `truncSet`, `Charged`, `chargedSet`, `exposedSet`, `wrapSet` [`CutHops.lean`]

The upper hop e of head v is

- *lost* if it is lost at r (Definition 3.11);
- *truncated* if $\mathrm{rk}_{\mathcal{O}(p)}(d) \geq P$;
- *charged* if some head w has a standard event $(t, g) \in \mathrm{Rise}_r(w) \cup \mathrm{Last}_r(w)$ with $p < t \leq u$ and $g \geq d + 1$;
- *exposed* if it is neither lost nor charged, and $\mathcal{O}(p) \cap (d, \infty) \neq \mathcal{O}(u) \cap (d, \infty)$;
- *wrapping* if it is not lost, and for some $g \in \mathcal{O}(p)$ with $g \geq d$ the digit at position $\mathrm{rk}_{\mathcal{O}(p)}(g) < P$ wraps for g between p and u, for the counter $\bar{N}_{\theta(g)}$.

The *lost*, *truncated*, *charged*, *exposed* and *wrapping* sets of v consist of the upper hops of v of these kinds, the charged set of those that are moreover not lost.

The lost and charged sets depend only on r, the truncated and exposed sets on r and ξ, and the wrapping set on all three choices.

Lemma 6.7 (The violated hops of a head)**.** Lean: `card_violated_le_cut`, `card_chargedSet_le`, `views_eq_of_no_event`, `card_exposed_hop_le`, `sum_card_exposedSet_le`, `card_wrap_hop_le`, `globalCount_sub_le_events`, `sum_card_wrapSet_le` [`CutHops.lean`]

Let v be a head.

(a) For every choice (r, ω, ξ), the upper hops (p, u) of v with $\Psi_{r,\omega,\xi}(u) \le \Psi_{r,\omega,\xi}(p)$ number at most the sizes of the lost, truncated, charged, exposed and wrapping sets of v together.

Let moreover $r < h_0$.

(b) The charged set of v has at most $\sum_w \sum_{(t,g)} 10 \cdot 2^{\theta(g)}$ elements, the inner sum over $\mathrm{Rise}_r(w) \cup \mathrm{Last}_r(w)$.

(c) Every upper hop of v is exposed for at most $2nQ^{\iota-1}$ of the Q^ι bit vectors. Hence the exposed sets of v, summed over the bit vectors, have at most $\#\{\text{upper hops of } x^{(v)}\} \cdot 2nQ^{\iota-1}$ elements.

(d) For all bits ξ, the wrapping sets of v, summed over the K^P digit offsets, have at most

$$K^{P-1} \sum_w \Big(\sum_s 20 \cdot 2^{h_0} \cdot 2^{\mathrm{band}(\mathrm{prom}(s))} + \sum_{(F,s)} 20 \cdot 2^{h_0} \cdot 2^{\mathrm{band}(m(F,s))} \Big)$$

elements, where s runs over the turning points of $x^{(w)}$ and (F, s) over the segment starts of $\mathcal{S}_w$.

Outline of the formal proof. (a) (`card_violated_le_cut`) A violated hop that is not lost satisfies (TRUNC), (ABOVE) or (WRAP) by Lemma 4.7(d); in the case (ABOVE) it is charged or exposed. (b) (`card_chargedSet_le`) A charged hop spans the time t of a standard event (t, g) with $g \ge 1$ and $d \le g - 1$. By Lemma 6.2(d) with the group $g - 1$ at the time t, at most $10 \cdot 2^{\theta(g)}$ upper hops of v that are not lost are charged through that event. (c) (`card_exposed_hop_le`) Let e be neither lost nor charged. Then no head has a standard event of a group at least $d + 1$ in $(p, u]$, and by Proposition 5.10, applied to every head, every head has the same view above d at p and u (`views_eq_of_no_event`; the heads pop at band zero, and groups below T suffice by Lemma 6.1(e)). If e is aligned, $d = G(\beta_v(p)) = G(\beta_v(u))$ is the band group of v at both ends, a seed at both times, and Proposition 5.11(c) bounds the bit vectors by $2nQ^{\iota-1}$. Otherwise $d = 0$; every candidate at 0 has bottom 0 and is displayed by rule (Z), so by Proposition 5.11(a) and Lemma 5.8(c) the shown sets above 0 agree for all bits, and e is never exposed. Summing over the hops is a double count (`sum_card_eq_sum_card_filter`, `sum_card_exposedSet_le`). (d) (`sum_card_wrapSet_le`) For one hop that is not lost, Lemma 6.4(b) with $B = \{g \in \mathcal{O}(p) : g \ge d\}$ bounds the digit offsets that make it wrapping by $K^{P-1} \sum_{g \in B} (\bar{N}_{\theta(g)}(u) - \bar{N}_{\theta(g)}(p))$ (`card_wrap_hop_le`), and the increase of a global counter is at most the number of turning points and segment starts of all heads in $(p, u]$ of scale at least $\theta(g)$ (`globalCount_sub_le_events`). Exchanging the sums over hops and heads w, Lemma 6.4(c) applies to the hops of v and to each of the two event families of each head w. □

Part (c) is where the break bits are used, and only for aligned hops: a hop that is not aligned has deciding group 0 and is never exposed. Parts (b) and (d) rest on the counting fact for kept hops, Lemma 6.2(d): in (b) the deciding group lies below the group of the charged event, and in (d) at or below the group whose digit wraps.

6.7 Totals

Lemma 6.8 (Totals)**.** Lean: `card_upperHops_head_le`, `sum_events_head_le`, `card_frames_head_le`, `card_events_head_le`, `weight_events_head_le`, `sum_descents_le` [`Totals.lean`]

Let v be a head.

(a) $x^{(v)}$ has at most T upper hops.

(b) $\sum_{r<h_0} (|\mathrm{Rise}_r(v)| + |\mathrm{Last}_r(v)|) \le 8T$.

(c) $\mathcal{S}_v$ has at most $2T+1$ frames.

(d) The turning points of $x^{(v)}$ and the segment starts of $\mathcal{S}_v$ number at most $4T+1$ together.

(e) They weigh at most $6T+1$: $\sum_s 2^{\mathrm{band}(\mathrm{prom}(s))} + \sum_{(F,s)} 2^{\mathrm{band}(m(F,s))} \le 6T+1$.

(f) The descents of the labels, summed over Ω, number at most

$$K^P Q^\iota \cdot n \cdot 8T + Q^\iota h_0 \cdot PK^{P-1} \cdot n(4T+1).$$

Outline of the formal proof. (a) The upper hops and the tops of a walk together number at most T (`card_upperHops_add_card_tops_le`). (b) By Lemma 6.1(c) with $N = T$ and Lemma 3.10(b), since every weight is at least 1 (`sum_events_head_le`). (c) A walk has at most $2T$ suspend frames (Lemma 3.10(c)), and there is the root (`card_frames_head_le`). (d) The turning points are times before T, and by Lemma 6.1(d) and part (c) there are at most $(2T+1)+T$ segment starts (`card_events_head_le`). (e) By Lemma 6.1(d), the turning points weigh at most $2(T-1)$, and the segment starts at most $T+(3T+1)$, by Lemma 3.10(b) and the count of part (d) (`weight_events_head_le`). (f) By Lemma 6.5(b), each choice has at most as many descents as event times and wrap times. The event times do not depend on (ω, ξ), and summed over the offsets they number at most $\sum_v 8T = 8nT$ by Lemma 6.5(c) and part (b). For each r and ξ, the wrap times, summed over ω, number at most $PK^{P-1} \cdot n(4T+1)$ by Lemma 6.5(d) and part (d) (`sum_descents_le`). □

7 Proof of the main theorem

This section chooses the parameters and adds up the bounds of Section 6. Throughout, $k \in \mathbb{N}$, $n = 2k$ is the number of heads, and $\varepsilon > 0$.

7.1 The parameters

Definition 7.1 (The parameters)**.** Lean: `parH`, `parQ`, `parJ`, `parP`, `parK` [`Main.lean`]

The *grid width* h_0, the *bit size* Q, the number of *blocks* j, the number of *digits* P and the *radix* K are

$$h_0 = \left\lceil \frac{14n + 80n^2}{\varepsilon/4} \right\rceil + 1, \qquad Q = \left\lceil \frac{2n^2}{\varepsilon/4} \right\rceil + 1, \qquad j = \left\lceil \ln(\max(1, 8n^2/\varepsilon)) \cdot Q^{n^2} \right\rceil,$$

$$P = 1 + 2n(jn+1), \qquad K = \left\lceil \frac{5Pn + 140 \cdot 2^{h_0} n^2}{\varepsilon/4} \right\rceil + 1.$$

The parameters depend only on k and ε, and they are fixed in this order: h_0 and Q from n and ε, j from Q, P from j, and K from P and h_0. By construction $h_0, Q, K \ge 1$ and $P \ge 1$. The grid width pays for the terms that decay like $1/h_0$, the bit size for exposure, the number of blocks, and with it the number of digits, for truncation, and the radix for the wraps. The cut will have depth at most $K^P - 1$.

Lemma 7.2 (The parameters suffice)**.** Lean: `parH_spec`, `parQ_spec`, `parJ_spec`, `parK_spec` [`Main.lean`]

Lean: `le_natCeil_add_one_mul`, `one_sub_pow_le_inv`, `one_sub_pow_natCeil_le_inv`, `trunc_term_le` [`Choice.lean`]

The parameters of Definition 7.1 satisfy

$$14n + 80n^2 \le h_0 \frac{\varepsilon}{4}, \qquad 2n^2 \le Q\frac{\varepsilon}{4}, \qquad 2n^2(1 - Q^{-n^2})^j \le \frac{\varepsilon}{4}, \qquad 5Pn + 140 \cdot 2^{h_0} n^2 \le K\frac{\varepsilon}{4}.$$

Outline of the formal proof. For $\delta > 0$ and every real C, $C \le (\lceil C/\delta \rceil + 1)\delta$, where $\lceil\cdot\rceil$ is the ceiling truncated at 0 (`le_natCeil_add_one_mul`). With $\delta = \varepsilon/4$ this gives the first, the second and the fourth inequality (`parH_spec`, `parQ_spec`, `parK_spec`). For the third (`parJ_spec`, `trunc_term_le`), put $x = Q^{-n^2} \in (0,1]$ and $A = \max(1, 8n^2/\varepsilon) \ge 1$, so that $j = \lceil \ln A/x \rceil$. Then $(1-x)^j \le e^{-xj} \le 1/A$ (`one_sub_pow_le_inv`, `one_sub_pow_natCeil_le_inv`), and $2n^2/A \le \varepsilon/4$: if $8n^2/\varepsilon \le 1$, then $2n^2 \le \varepsilon/4$, and otherwise $2n^2/A = \varepsilon/4$. □

7.2 The cut, summed over the random choices

As in Section 6, $\Omega = [0,h_0) \times [0,K)^P \times [0,Q)^\iota$ is the set of the random choices (r,ω,ξ), $\Psi_{r,\omega,\xi}$ is the label of Definition 4.6, and $\mathrm{Bad}_{r,\xi}$ is the set of bad times of Lemma 6.3. The following proposition adds up the bounds of Section 6 in natural numbers, for arbitrary parameters.

Proposition 7.3 (The cut in natural numbers)**.** Lean: `sum_weight_events_head_le`, `sum_violated_head_le`, `sum_labelCutSize_le`, `sum_bad_le` [`Main.lean`]

Let $h_0, P \in \mathbb{N}$ and $K, Q \ge 1$.

(a) For every head v, $\sum_{r<h_0}\sum_{(t,g)} 2^{\theta_r(g)} \le 8T$, the inner sum over $\mathrm{Rise}_r(v) \cup \mathrm{Last}_r(v)$.

(b) For every head v, the upper hops (p,u) of v with $\Psi_{r,\omega,\xi}(u) \le \Psi_{r,\omega,\xi}(p)$, summed over $(r,\omega,\xi) \in \Omega$, number at most

$$B_v = K^P Q^\iota \cdot 6T + K^P \sum_{r<h_0}\sum_{\xi} |\mathrm{Bad}_{r,\xi}| + K^P Q^\iota \cdot 80nT + K^P h_0 T \cdot 2nQ^{\iota-1} + h_0 Q^\iota K^{P-1} \cdot 20 \cdot 2^{h_0} n(6T+1).$$

(c) The cuts, summed over Ω, satisfy

$$\sum_{(r,\omega,\xi)\in\Omega} |F_{\Psi_{r,\omega,\xi}}| \le K^P Q^\iota \cdot 8nT + Q^\iota h_0 \cdot PK^{P-1} \cdot n(4T+1) + n \cdot B,$$

where B is the common value of the bounds B_v of (b).

(d) If $P \ge 1 + 2n(jn+1)$ for some $j \in \mathbb{N}$, then $\sum_{r<h_0}\sum_\xi |\mathrm{Bad}_{r,\xi}| \le h_0 T \cdot 2n(1-Q^{-n^2})^j Q^\iota$.

Outline of the formal proof. (a) Lemma 6.1(c) with $N = T$, and Lemma 3.10(b) (`sum_weight_events_head_le`). (b) (`sum_violated_head_le`) By Lemma 6.7(a), for each choice the violated upper hops of v are at most its lost, truncated, charged, exposed and wrapping hops; we sum each kind over Ω, the offsets in the sums being below h_0. *Lost:* the lost set depends only on r, and by Theorem A (Theorem 3.16) its size, summed over the offsets, is at most 6 times the number of upper hops, hence at most $6T$ (Lemma 6.8(a)); the other two coordinates contribute the factor $K^P Q^\iota$. *Truncated:* at most $|\mathrm{Bad}_{r,\xi}|$ for every ω (Lemma 6.3(a), `card_truncSet_le_bad`). *Charged:* the charged set depends only on r; by Lemma 6.7(b) and part (a), summed over the offsets, its size is at most $10\sum_w 8T = 80nT$. *Exposed:* independent of ω, and summed over ξ at most $T \cdot 2nQ^{\iota-1}$ for every r (Lemma 6.7(c), Lemma 6.8(a)). *Wrapping:* for every r and ξ, summed over ω, at most $K^{P-1} \cdot 20 \cdot 2^{h_0} \sum_w (6T+1)$ (Lemma 6.7(d), Lemma 6.8(e)). (c) (`sum_labelCutSize_le`) The cut consists of the descents and the violated hop edges. Lemma 4.7(c) bounds the latter by the violated upper hops of the n heads, each of which is bounded by (b), and Lemma 6.8(f) bounds the descents. (d) (`sum_bad_le`) Exchanging the sums, $\sum_\xi |\mathrm{Bad}_{r,\xi}| = \sum_{t<T} \#\{\xi : t \text{ is bad}\}$, and Lemma 6.3(b) bounds every term. □

7.3 The main estimate

The four error terms of the main estimate are, as expectations over uniform random choices, the following. The terms *paid by the grid* are the descents at event times, $8nT/h_0$, the lost hops, $6nT/h_0$ (Theorem A), and the charged hops, $80n^2T/h_0$; the term of *truncation* is $2n^2(1-Q^{-n^2})^jT$ (Lemma D); the term of *exposure* is $2n^2T/Q$ (Proposition 5.11); and the term of the *wraps* collects the wrap times, $Pn(4T+1)/K$, and the wrapping hops, $20\cdot 2^{h_0}n^2(6T+1)/K$. With the parameters of Definition 7.1, each term is at most $\varepsilon T/4$.

Proposition 7.4 (The main estimate)**.** Lean: `exists_avgLabel` [`Main.lean`]

Let $T\in\mathbb{N}$ and let $x_1,\dots,x_k$ be walks, pauses allowed. With the parameters of Definition 7.1, there are a nonempty finite set A and labels Ψ_a, $a\in A$, with values below K^P, such that

$$\sum_{a\in A}|F_{\Psi_a}|\le |A|\cdot\varepsilon T.$$

Outline of the formal proof. The formal proof takes $A=\Omega$ and the labels $\Psi_{r,\omega,\xi}$ of Definition 4.6. They take fewer than K^P values by Lemma 4.7(a), and Ω is nonempty. For $T=0$ there are no edges. Let $T\ge 1$. Since $P=1+2n(jn+1)$, Proposition 7.3(d) applies; insert it into Proposition 7.3(c) and divide by $|\Omega|=h_0K^PQ^\iota$. The result is a sum of four terms, each at most $\varepsilon T/4$ by Lemma 7.2:

(i) *paid by the grid:* $(8n+6n+80n^2)T/h_0=(14n+80n^2)T/h_0\le\varepsilon T/4$;

(ii) *truncation:* $n\cdot 2n(1-Q^{-n^2})^jT=2n^2(1-Q^{-n^2})^jT\le\varepsilon T/4$;

(iii) *exposure:* $n\cdot 2nTQ^{\iota-1}/Q^\iota=2n^2T/Q\le\varepsilon T/4$; here $Q^\iota=Q\cdot Q^{\iota-1}$, since for $k\ge 1$ every head has the root as a frame, so that $\iota\ge 1$, and for $k=0$ the term vanishes;

(iv) *wraps:* $n\big(P(4T+1)+20\cdot 2^{h_0}n(6T+1)\big)/K\le(5Pn+140\cdot 2^{h_0}n^2)T/K\le\varepsilon T/4$, since $4T+1\le 5T$ and $6T+1\le 7T$.

So the average cut is at most εT (`exists_avgLabel`). □

7.4 Theorem T

Theorem 7.5 (Theorem T)**.** Lean: `tcpAt_direct`, `tcp`, `main`, `sr` [`Main.lean`]

For $k\in\mathbb{N}$ and $\varepsilon>0$ let $D(k,\varepsilon)=K^P-1$, with K and P as in Definition 7.1. For every horizon T and all walks $x_1,\dots,x_k$, with or without pauses, the walk DAG of $x_1,\dots,x_k$ before T has a cut of at most εT edges and depth at most $D(k,\varepsilon)$. Consequently:

(a) TCP(k) *holds for every k, for walks with pauses, and hence for pause-free walks: this is the main statement;*

(b) SR(s) *holds for every s: DAGs with a spine whose arcs form s stacks reduce to constant depth.*

Outline of the formal proof. The first statement is `tcpAt_direct`, that is, TCP(k) at $(\varepsilon, D(k,\varepsilon))$ in the sense of Definition 2.3 (`BoundedNesting.TCPat`). By Proposition 7.4 some nonempty finite family of labels with values below K^P (in the formal proof, the labels $\Psi_{r,\omega,\xi}$) has cuts of average size at most εT; by the probabilistic method (Proposition 2.7, `exists_cut_of_avg`) one of these labels has a cut of at most εT edges, whose depth is at most K^P-1 by Proposition 2.2. (a) For every $\varepsilon>0$ the first statement provides the depth $D(k,\varepsilon)$ (`tcp`); the main statement, TCP(k) for every k (`MainStatement`), is `main`. Walks without pauses are walks, so TCP(k) contains its

pause-free form (`BoundedNesting.tcpnp_of_tcp`). (b) (`sr`, `sr_of_main`) By the reduction of the eighth paper (Proposition 2.4, `BoundedNesting.sr_of_tcpnp`), an s-stack DAG on $[0, N)$ is realized by s pause-free walks of length $2N$, and a cut of their walk DAG of at most $(\varepsilon/2)(2N)$ edges and depth D pulls back to a cut of the stack DAG of at most εN edges and depth at most D. The formal proof takes the depth of (a) at $\varepsilon/2$, so every s-stack DAG on $[0, N)$ has a cut of at most εN edges and depth at most $D(s, \varepsilon/2)$. □

The construction never uses pause-freeness: Theorem A and the composition hold for walks with pauses, so the first statement covers them directly, at the depth of k walks, without the reduction to $2k$ pause-free walks of Proposition 2.4. The depth $D(k, \varepsilon)$ does not depend on T, so no family of k-tuples of walks is depth-robust, and with (b) every family of DAGs with a spine whose arcs form a bounded number of stacks is depth-reducible; in particular SR(3), the first open case left by the eighth paper [4, Problem 10.2], holds.

8 Why each ingredient is needed

Beyond the label framework of the eighth paper [4], the proof uses three ingredients: bands measured from homes adapted to the walk (the ladders and frames of Section 3, with their hybrid running maximum), displays of suspended frames (Section 5), and random break bits. This section records why simpler choices fail. The remarks are not formalized and no proof in this paper uses them; each is labelled as a proof sketch, a heuristic or numerical.

Remark 8.1 (Homes at turning points fail)**.** *Proof sketch only.* The simplest band measures the distance from the start of the current monotone run of the walk: it is the band of the frame system whose only frame is the root and whose restart points are the times at which the walk reverses direction. Theorem A fails for it on the *nested mountains*. Let $\mathrm{St}(g) = (+1, +1, -1)^g$ be a slow staircase and $\mathrm{Dn}(g) = (-1)^g$ a fast descent, put $M_0 = (+1, +1, -1, -1)$ and, for $j \geq 1$ and integers $0 \leq a_j \leq 2^{j-1}$,

$$M_j = \mathrm{St}(a_j)\, M_{j-1}\, M_{j-1}\, \mathrm{St}(2^j - a_j)\, \mathrm{Dn}(2^j),$$

and take the walk with the steps of M_J, of length $T = 4(J+1)2^J$. A copy of M_j climbs slowly by 2^j, carrying two copies of M_{j-1} that stay below its summit, and returns in a single run. On a slow climb the last visit to a cell is a reversal, so the hops that end on the lower half of the final descent of a copy of M_j start at band 0; they end at distance more than 2^{j-1} from the summit, at band at least $j-1$, and their extent exceeds 2^{j-1}. For $j \geq h_0 + 1$ their two ends therefore lie in different groups at every offset, and by (P1) of Proposition 2.9 their owners have scale at least $j - 1 \geq h_0$, so they are not ground: they are lost (Definition 3.11) at every offset. Each j contributes at least $2^{J-j} \cdot 2^{j-1} = T/(8(J+1))$ such hops, so at least $(J - h_0)\, T/(8(J+1))$ hops are lost at every offset, a fraction of T that tends to $\frac{1}{8}$ as J grows, whereas Theorem A allows $O(T/h_0)$. Bands that measure from turning points measure speed, not distance. Measuring the distance from the base of the excursion instead (Section 3) makes these walks cheap: the climb of an excursion has no restart, and its descent restarts only at the passages $D(y + 2^e)$.

Remark 8.2 (Frames must be displayed)**.** *Heuristic.* Frames make the hops of each walk cheap on their own (Theorem 3.16), but they create an interaction between walks. While a frame runs, the segment that it suspends keeps its band frozen; when the segment resumes, its band group reappears. A label that shows only the current band groups of the walks therefore changes its shown set at every pop at a high band. Such a change can produce a descent (Lemma 4.4), and it falls under the case (ABOVE) of Lemma V (Lemma 4.3) for every hop of another walk that spans it with a lower

deciding group. On walks whose long excursions carry dense nested structures at many scales these pops are frequent, and the label pays for them. Either the frozen band is shown while the frame runs, or the pop that ends the frame changes the shown set at a high group; this pop-or-freeze dichotomy forces the label to show suspended frames, which is what the entries of Section 5 do.

Remark 8.3 (Exact displays are unbounded)**.** *Proof sketch only.* Call a display *exact* if, with every entry unbroken, its shown set above every group d depends only on the view above d (Section 5.5), so that view invariance alone, with no random bits, would control every hop. The view above d does not distinguish a candidate at d from a seed at its top, and seeds are always shown; so an exact display shows the top of every candidate at every value, and in particular the whole cascade of windows: an entry whose window contains a shown value is shown, and so on upwards. On the *dense tower*, a walk whose stack of suspended frames carries, at a constant fraction of times, unboundedly many entries with distinct tops and windows of more than one group, exact displays therefore have unbounded size, and no bounded number of digits can carry them. Random break bits cut the cascade at the first broken entry: Lemma D (Lemma 5.7) bounds the size of the display independently of the depth of the stacks, and a hop is exposed only through a broken candidate, with probability at most $2n/Q$ (Proposition 5.11).

Remark 8.4 (The three definitional choices)**.** *Numerical.* Three details of Definitions 5.2 and 5.9 matter: an entry is a suspend child (its parent segment is innermost again after it ends), its top is the group of the frozen band of its parent segment at the push, and a rise of the running maximum at the time a frame resumes is a standard event. Under each alternative reading (every suspend frame is an entry; the top is the head's band group just before the push, which differs from the frozen band of the parent at a simultaneous push; a rise at a resumption is not an event), an exact search over pairs of short walks, with $h_0 = 1$ and all bits unbroken, finds a kept hop whose shown sets above its deciding group differ at its two ends although no standard event lies between them; the smallest instances found have lengths 23, 41 and 23. So the analogue of view invariance fails under each alternative reading. With the three choices, view invariance holds for all walks (Proposition 5.10).

9 Consequences for the series and for tape machines

This section combines Theorem T with results of earlier papers of the series. Corollary O, on oblivious machines, Corollary HV, which proves the eleventh paper's conjecture on code-level depth reduction, and Corollary L, on low adaptivity, are formalized: in the frozen machine model of the first paper [6], through the traces, genies, cones and index codes formalized in the eleventh paper's library [7], which this library imports. Corollary N, on the deep-nesting pairs, is formalized through the deep-nesting walk and the cyclic covers of the same library. Throughout, $D(k, \varepsilon) = K^P - 1$ is the depth of Theorem T, with $K = K(k, \varepsilon)$ and $P = P(k, \varepsilon)$ as in Definition 7.1.

9.1 The deep-nesting pairs

The fourth paper singled out the *deep-nesting pair* as the sharpest open instance of TCP(2) [5, Definitions 5.3 and 5.9]. For integers $s_0 > m \geq 0$ it consists of a periodic walk $U = U(s_0, m, 1)$ without pauses, whose strokes are nested at $m + 1$ levels, a stroke of level i moving the head by $\pm 2^{s_0 - i}$, and of its shift $U(\cdot + \xi)$ by an odd ξ. The fourth paper fixes one odd shift, the golden shift; like the eighth paper, which recalls the definition in its Section 2.5 [4], we call the pair a deep-nesting pair for every odd shift. Conjecture A′ of the fourth paper [5, Conjecture 5.25] and its weakening, Conjecture A″ of the eighth paper [4, Conjecture 5.24], bound the number of levels that a cheap

label on a cyclic cover of the pair can serve; the eighth paper showed that either conjecture makes the pairs depth-robust as $m \to \infty$, so that TCP(2) fails [4, Corollary 5.26]. Its turn counter cuts the pair, on a whole number of periods, to depth $\lceil 11 \cdot 2^{m+1}/((m+1)\varepsilon) \rceil - 1$, exponential in m [4, Corollary 5.5], and it asked whether the pairs are depth-robust [4, Problem 10.1(a)]; the tenth paper asks the same question [9, Section 7].

Corollary 9.1 (Corollary N: the deep-nesting pairs are not depth-robust)**.** `Lean`: `dnpPair`, `dnpPair_isWalk`, `dnp_cut`, `IsCoverLabel`, `coverCost`, `ThetaServesLevel`, `doublyServedCount`, `ConjAprime`, `ConjAsecond`, `not_conjAprime`, `not_conjAsecond` [`DeepNesting.lean`,`Conjectures.lean`]

For every $\varepsilon > 0$, all integers $s_0 > m \geq 0$, every shift ξ and every $T \geq 1$, the walk DAG of the pair $x_1 = U(s_0, m, 1)$, $x_2 = x_1(\cdot + \xi)$ on $[0, T)$ has a cut of at most εT edges and depth at most $D(2, \varepsilon)$. Consequently Conjecture A′ of the fourth paper and Conjecture A″ of the eighth paper are false, and the answer to Problem 10.1(a) of the eighth paper is negative.

Outline of the formal proof of the first statement. The pair consists of two walks, so the first statement is the case $k = 2$ of Theorem T (Theorem 7.5). The eleventh paper's library defines the walk U (`MergeDepth.dnpWalk`) and the two heads of the pair (`MergeDepth.headWalk`) [7, Section 7], and proves that U moves by ± 1 at every step (`MergeDepth.dnpStep_pm`). So both heads of the pair `dnpPair` are walks (`dnpPair_isWalk`), and `dnp_cut` is an instance of `tcpAt_direct` for $k = 2$. The Lean statement assumes only $m \leq s_0$. □

Outline of the formal proof of the second statement. The conjectures concern labels on the r-fold covers of the pair, whose hops, classes and errors are those of the eleventh paper's library (`MergeDepth.coverHops`, `MergeDepth.coverClass`, `MergeDepth.hopError`). `IsCoverLabel`, `coverCost`, `ThetaServesLevel` and `doublyServedCount` state the eighth paper's definitions of labels on the cover, their cost and the doubly served levels [4, Section 2.5], and `ConjAprime` and `ConjAsecond` state the two conjectures. The refutation follows the eighth paper's Corollary 5.26 [4] with two simplifications. First, the label is that of Theorem T itself (`exists_label`): for $k = 2$ it has $q = K^P$ values and cost at most εT on the walk DAG of the pair on $[0, T)$, so no label has to be built from a cut. Second, its transfer to the N-fold cover (`periodize`, `coverCost_periodize_le`) uses the crude bound $2\mathrm{cost}(\Psi) + 2P + 1$. That bound counts a hop of the walk DAG once per head, so it needs no parity argument; the hops of the cover with a negative earlier endpoint end in the first period. The per-head form of Lemma Zq on the cover (`sum_hopError_le`), total error at most q times the number of violated hops, follows from a telescoping identity: the earlier endpoints of the hops of a head run over every time of the cover once (`sum_headPrev_eq`). Since a hop has one level (`headHopLevelIs_unique`), the classes split this error (`sum_classError_le`), and a label of cost κ leaves at most $2(m+1)\kappa/(\theta n_{\mathrm{h}})$ levels not doubly θ-served (`unserved_mul_le`). Given θ_0, $\eta < 1$, C and C', take $\varepsilon = \min(1/32, (1-\eta)\theta_0/16)$, the nesting depth m with $(m+1)(1-\eta)/2 > C \log_2 q + C'$, $s_0 = m + 2$ and $N \geq 2/\varepsilon$. At every shift, the label of Theorem T on N periods, read on the N-fold cover, costs at most $2\varepsilon n_{\mathrm{h}} \leq n_{\mathrm{h}}/8$ and doubly serves at least $(m+1)(1-(1-\eta)/4)$ levels, more than Conjecture A″ allows (`not_conjAsecond`). Conjecture A′ is the case $\eta = 0$ (`conjAsecond_of_conjAprime`, `not_conjAprime`). Problem 10.1(a) of the eighth paper asks whether the pairs are depth-robust as $m \to \infty$ for some sequence of odd shifts; by the first statement they are not, at any shifts. □

The lower bounds of the series for this instance, some of them proof sketches or conditional on a conjecture, concern restricted classes of labels, for example per-head band rules, labels that change only at turns, and block labels [5, 4, 9, 7]. Corollary N does not contradict them: the labels of Theorem T lie outside these classes, since they show suspended frames of both heads, count events

at many scales, and have displays cut by random break bits (Sections 4 and 5). Where those papers read their lower bounds as evidence that the pairs are depth-robust, Corollary N shows that the reading does not stand.

9.2 Oblivious machines: the crux alone

We use the language of the fourth and eighth papers [5, Section 4], [4, Section 2.4]. For a machine M with k work tapes and value width at most β (the number of bits in which the value of a step is written; in this section β is not a band), and a trace of M at length L with halting time T, the *dependency DAG* of the trace has the vertex set $[0, T)$, the state edges $t \to t-1$, and an edge from t to the previous visit of the cell under work head j, for every j. A *genie* is a set Γ of revealed steps, of *cost* $\beta|\Gamma|$, and it has *full depth* at most D if every path whose vertices after the first lie outside Γ has at most D edges. If M is oblivious at length L, all inputs share one trace. The statement depth-TCP$(k, C, \beta, \varepsilon, D)$ asks for genies of cost at most εL and full depth at most D for all oblivious machines with k work tapes, value width at most β and halting time at most $C(L+1)$, at all large lengths [5, Definition 4.7]. The *crux* Crux$(\mathcal{S}_{k,C,\beta,D}, \rho)$ says that, for every $\eta > 0$ and all large sizes d, every index code for the side information of the cones of such genies has at least $(\rho - \eta)L$ bits [5, Definition 4.8]. Theorem D of the fourth paper [5, Theorem 4.10], with a written proof there, says: if depth-TCP$(k, C, \beta, \varepsilon, D)$ and Crux$(\mathcal{S}_{k,C,\beta,D}, \rho)$ hold with $0 < \varepsilon < \rho$, then for every machine with k work tapes and value width at most β there is d_0 such that for no $d \geq d_0$ is it oblivious at length $L = d^2 b(d)$ with halting time at most $C(L+1)$ and a transposer of all $d \times d$ matrices with $b(d)$-bit entries. The eighth paper proves the analogue for machines with a data path that is linear over a field F, Theorem $\mathrm{D}_{\mathrm{lin}}$, with linear depth-TCP and a linear crux $\mathrm{LinCrux}_F(\mathcal{S}^{\mathrm{ball}}_{k,C,\beta,D}, \rho)$ in place of the two hypotheses [4, Definition 7.4 and Theorem 7.5]. Corollary O below proves both conclusions again, formally and for all machines at once, with depth reduction supplied by Theorem T.

The formal statements follow these definitions in the frozen machine model.

Definition 9.2 (depth-TCP and the crux)**.** Lean: `DepthTCP`, `depCone`, `coneSystem`, `SCone`, `CruxOn`, `Crux` [`Machines.lean`]

Let $k, \beta, D \in \mathbb{N}$ and $C, \varepsilon, \rho \in \mathbb{R}$. Write $L = d^2 b(d)$.

(a) depth-TCP$(k, C, \beta, \varepsilon, D)$ holds if every machine with k work tapes and value width at most β has a d_0 such that, for every $d \geq d_0$ at which it is oblivious at length L with halting time $T \leq C(L+1)$, the dependency DAG of its trace has a genie Γ with $\beta|\Gamma| \leq \varepsilon L$ and full depth at most D.

(b) The *cone set* $I_\Gamma(t)$ of a step t is the set of input positions read at the ends of the Γ-free paths of the dependency DAG from t. The *cone system* of a trace assigns to every output position ℓ the cone set of the step at which ℓ is appended. $\mathcal{S}_{k,C,\beta,D}$ is the class of the cone systems, at size d, of the machines with k work tapes and value width at most β that are oblivious at length L with halting time $T \leq C(L+1)$ and append exactly L bits, for the sets Γ of full depth at most D.

(c) An *index code* with m bits for a system I is a map $E\colon \{0,1\}^L \to \{0,1\}^m$ such that $E(w)$ and the bits of w at the positions of $I(\ell)$ determine the bit of w at the position $\mathrm{tpos}(\ell)$ that output ℓ copies, for every input w and every ℓ. Crux$(\mathcal{S}_{k,C,\beta,D}, \rho)$ holds if for every $\eta > 0$ there is d_0 such that every index code for every system of $\mathcal{S}_{k,C,\beta,D}$ at a size $d \geq d_0$ has at least $(\rho - \eta)L$ bits.

These are the fourth paper's Definitions 4.7 and 4.8 [5]. The linear crux $\mathrm{LinCrux}_F(\mathcal{S}^{\mathrm{ball}}_{k,C,\beta,D}, \rho)$ of the eighth paper, a minrank bound for the balls of radius D of the same machines [4, Definition 7.4], is formalized in the eleventh paper's library (`MergeDepth.LinCrux`).

Corollary 9.3 (Corollary O: depth reduction for oblivious machines)**.** `Lean: genieDepth, genie_of_trace, depthTCP, crux_excludes, linCrux_excludes` [`Machines.lean`]
Let $k \in \mathbb{N}$, $\beta \geq 1$, $C \geq 1$ and $\varepsilon > 0$. Put $\varepsilon' = \varepsilon/(4\beta C)$ and

$$D_1 = D(k, \varepsilon') + 1, \qquad D = D_1 \lceil 4\beta C/\varepsilon \rceil.$$

(a) For every machine with k work tapes, every input, every $L \geq 1$ and every horizon $T \leq C(L+1)$, the dependency DAG of the trace of the input before T has a genie of cost $\beta|\Gamma| \leq \varepsilon L$ and full depth at most $D-1$. In particular, depth-TCP$(k, C, \beta, \varepsilon, D)$ *holds, with $d_0 = 1$.*

(b) If $\mathrm{Crux}(\mathcal{S}_{k,C,\beta,D}, \rho)$ *holds for some $\rho > \varepsilon$, then there is d_0 such that for no $d \geq d_0$ is a machine with k work tapes and value width at most β oblivious at length $L = d^2 b(d)$ with halting time at most $C(L+1)$ and a transposer of all $d \times d$ matrices with $b(d)$-bit entries.*

(c) If $\mathrm{LinCrux}_F(\mathcal{S}^{\mathrm{ball}}_{k,C,\beta,D}, \rho)$ *holds for a field F and some $\rho > \varepsilon$, then there is d_0 such that for no $d \geq d_0$ is a machine with k work tapes and value width at most β oblivious at length $L = d^2 b(d)$ with halting time at most $C(L+1)$, a transposer of all $d \times d$ matrices with $b(d)$-bit entries, and equipped with an F-linear data path.*

Outline of the formal proof. (a) (`genie`, `genie_cost`, `genie_of_trace`) The work-head walks of a trace are k walks, possibly with pauses (`MergeDepth.Trace.work_isWalk`). Theorem T at ε' cuts their walk DAG by at most $\varepsilon' T$ edges to depth at most $D_1 - 1$. The genie Γ reveals the lower end of every cut edge and every step $t \equiv R - 1 \pmod R$, where $R = \lceil 4\beta C/\varepsilon \rceil$. A Γ-free path of the dependency DAG uses no cut edge, so it has fewer than D_1 edges along long hops, and between two of them, or at either end, fewer than R consecutive state edges. Hence it has at most $D_1 R = D$ vertices. The cost is $\beta|\Gamma| \leq \beta(\varepsilon' T + T/R) \leq \varepsilon T/(2C) \leq \varepsilon L$. This is the eleventh paper's cold genie [7, Lemma 3.1] with Theorem T in place of the eighth paper's Theorem F, so it bounds all Γ-free paths, not only those without deep edges; it replaces the eighth paper's Lemma 2.6. For an oblivious machine the trace is that of any input, and depth-TCP follows with $d_0 = 1$ (`depthTCP`). (b) (`exists_indexCode`, `crux_excludes`) A machine as in (b) is an oblivious transposer along the trace of any input (`MergeDepth.obliviousTransposer_of_transposes`). By the eleventh paper's index-code lemma [7, Lemma 3.3(b)] for the field DAG of the pruning that keeps every edge (`MergeDepth.Pruning.field_isIndexCode`), the values of the steps of Γ take at most $2^{\beta|\Gamma|}$ values and form an index code for the cone sets of that field DAG. A field path projects to a Γ-free path of the dependency DAG, so these cone sets lie in the cone sets I_Γ (`coneI_subset_depCone`); an index code stays one when the side information grows (`IsIndexCode.mono`), and one with at most 2^m code words gives one with m bits (`IsIndexCode.bits`). So the genie of (a) gives an index code with $\beta|\Gamma| \leq \varepsilon L$ bits for a system of $\mathcal{S}_{k,C,\beta,D}$, and for d at least the d_0 of the crux at $\eta = (\rho - \varepsilon)/2$, and $d \geq 1$, this contradicts the bound $(\rho + \varepsilon)L/2$. (c) (`linCrux_excludes`) The Γ-free paths of the data path's network from an append step are paths of the dependency DAG (`MergeDepth.isDepPath_of_paths`) with at most D vertices, so the path sums that define linear depth vanish at every position that no Γ-free path of at most D edges from the append step reads: every output step has linear depth at most D [4, Definition 7.2]. The eighth paper's rigidity at one size, for machines (`MergeDepth.LinDataPath.rigidity_machine`), then bounds the minrank of the ball system of radius D by $\beta|\Gamma| \leq \varepsilon L$, which contradicts the linear crux as in (b). □

So the crux alone, at the depth D and a rate above ε, excludes every oblivious linear-time transposer, whatever its head motion; this extends the eighth paper's Corollary F [4, Corollary 6.1] from heads that enter every cell boundedly often to all heads, and the fourth paper's Corollary B [5, Corollary 4.23] from home-based heads to all heads. For linear machines, part (c) leaves only the linear crux, which the eighth paper states exactly as a question of relative rigidity [4, Problem 10.6].

Remark 9.4 (The rate needed)**.** *Proof sketch only.* Fix k, β and C, write D_ε for the depth D of Corollary 9.3 at ε, and let $\rho^*(D)$ be the supremum of the rates ρ at which $\mathrm{Crux}(\mathcal{S}_{k,C,\beta,D}, \rho)$ holds [5, Remark 4.15]. By Corollary 9.3(b), the oblivious machines with these parameters are excluded as soon as $\rho^*(D_\varepsilon) > \varepsilon$ for a single $\varepsilon \in (0,1]$. By the estimates of Section 10, $\log\log D_\varepsilon \le c_1 k^2 \log(k\beta C/\varepsilon)$ for an absolute constant c_1, that is, $\varepsilon \le k\beta C(\log D_\varepsilon)^{-1/(c_1 k^2)}$. Hence a crux whose rate at every large depth D is at least $k\beta C(\log D)^{-c/k^2}$, for an absolute constant $c < 1/c_1$, suffices. For bounded-entry heads the eighth paper needs a rate of order $1/\log D$ instead [4, Section 6.1]. Both requirements tend to zero with D, the present one much more slowly, since the depth is doubly exponential.

Remark 9.5 (The native half bound, and adaptive machines)**.** *Proof sketch only.* Question 9.1 of *Directed Recovery Structures* [10] asked for a half bound for all storage codes on bounded-degree relations with the paired-distance property; the thirteenth paper answers it negatively [8, Theorem A and Corollary A′]. Its *native form*, the half bound for all codes on the cone relations of linear-time machines, uniform over the schedules and genies as in the conclusion of [10, Proposition 9.2], is open [4, Problem 10.3]. Every fibre of an index code is a binary storage code for the cone relation [5, Remark 4.9(b)], and an index code with m bits has a fibre with at least 2^{L-m} words; so the native half bound for oblivious machines gives $\mathrm{Crux}(\mathcal{S}_{k,C,\beta,D}, \frac{1}{2})$ at every depth D, and by Corollary 9.3(b) with $\varepsilon = \frac{1}{4}$ it excludes oblivious linear-time transposers. This is the conditional statement anticipated in the eighth paper's open problems. In the same way, the proofs of the eighth paper's Corollaries F′ and F″ [4, Corollaries 6.3 and 6.4], with the native bound assumed in place of Question 9.1 as that paper allows, use the bounded entry of the heads only to obtain a cut from its Theorem F, and Theorem T supplies the cut for all walks. So, under the native bound for frozen traces, Corollary F″ holds without its hypothesis (ii): for every machine M, every $c > 0$ and every $\eta < \frac{1}{2}$ there is L_0 such that at no length $L = d^2 b(d) \ge L_0$ does M transpose every input within $c(L+1)$ steps with a trace whose Shannon entropy, for a uniformly random input, is at most ηL. Corollary 9.7 below proves this formally. That hypothesis on the entropy of the trace is not robust [4, Section 6.2]. The deficit form of the eighth paper's Remark 6.5 adapts as well. There the paid trimming works at the incidence degree $\Delta = \lceil B/\delta \rceil$, for a fixed $\delta \in (0,1)$, where $B = \sum_{i \le D}(k+1)^i$ bounds the cone sizes. With the depth D of Corollary 9.3, $\log\log\Delta \le (k\beta C/\varepsilon)^{O(k^2)}$, so a bound $1 - \rho(\Delta) + o(1)$ on the rates of the codes on paired-distance separated relations of incidence degree at most Δ, with $\rho(\Delta)(\log\log\Delta)^a \to \infty$ for every $a > 0$, suffices to exclude oblivious linear-time transposers.

The equivalence between TCP for every number of walks and information-cheap depth reduction [11, Theorem I] now has both sides true. The obstruction that assumes that TCP(k) fails for some k [11, Theorem H] is vacuous, and so is the motivation that the eighth paper gives for its scratch question [4, Problem 10.5].

The same genies settle the eleventh paper's conjecture on code-level depth reduction. For a trace and a radius r, the *ball* $B^r(\ell)$ of an output position ℓ is the set of input positions read at the ends of the dependency paths of at most r edges from the step at which ℓ is appended, and $(B^r(\ell))_\ell$ is the *ball system* of radius r [4, Definitions 7.2 and 7.4]. Conjecture HV* of the eleventh paper [7, Conjecture 3.22] concerns the machines with k work tapes and value width at most β that are oblivious at length $L = d^2 b(d)$ with halting time at most $C(L+1)$ and transpose. It asks for some $c > 0$ and, for every $\tau \ge 1$, a radius r such that, for every $\eta > 0$ and all large d, the ball

system of radius r of every such machine has an index code with at most $\varepsilon L + cN_{\mathcal{R}}/\tau + \eta L$ bits, where $N_{\mathcal{R}}$ counts the deep hops of its trace. Its linear form asks the same of the minrank over a field F for the machines with an F-linear data path of bounded width. The eleventh paper proved the conjecture for codes of bounded merge depth [7, Remark 3.23(a)] and asked for a proof or a refutation [7, Problem 9.1].

Corollary 9.6 (Corollary HV: code-level depth reduction)**.** `Lean: coneSystem_subset_ballSystem, ballSystem_indexCode, ballSystem_minrank, ConjHVStar, ConjHVStarLin, hvStar, hvStar_linear` [`Machines.lean`]

Let $k \in \mathbb{N}$, $\beta \geq 1$, $C \geq 1$ and $\varepsilon > 0$, and let D be as in Corollary 9.3. For every machine with k work tapes and value width at most β that is oblivious at length $L = d^2 b(d)$ with halting time at most $C(L+1)$ and transposes all $d \times d$ matrices with $b(d)$-bit entries, the ball system of radius D of its trace has an index code with at most εL bits; if the machine also has an F-linear data path, of any width, the ball system has minrank at most εL over F. Consequently Conjecture HV* of the eleventh paper holds in both its forms, with $c = 1$, the radius D for every τ, and $d_0 = 1$.

Outline of the formal proof. (`coneSystem_subset_ballSystem`) Under the genie Γ of Corollary 9.3(a) every Γ-free path of the dependency DAG has at most D vertices, so the cone set of every output position lies in its ball of radius D (`MergeDepth.ballSystem`). (`ballSystem_indexCode`) The index code of Corollary 9.3(b), with $\beta|\Gamma| \leq \varepsilon L$ bits, stays an index code when the side information grows (`IsIndexCode.mono`). (`ballSystem_minrank`) The rigidity step of Corollary 9.3(c) bounds the minrank of the ball system of radius D by $\beta|\Gamma| \leq \varepsilon L$, whatever the width of the data path. (`hvStar`, `hvStar_linear`) `ConjHVStar` and `ConjHVStarLin` state the two forms of the conjecture, with $N_{\mathcal{R}}$ the number of deep hops of the trace (`MergeDepth.deepHops`). Since $N_{\mathcal{R}} \geq 0$ and $\eta L \geq 0$, the bound εL implies the conjectured one. The conjecture assumes $k, V \geq 1$, $\beta \geq 2$ and $\varepsilon \leq 1$; the formal statements need none of these. □

Corollary HV answers the eleventh paper's Problem 9.1 affirmatively, in the strong form of its Remark 3.23(b) [7]: the radius does not depend on τ, and no deep-hop term is needed. It excludes no machine that Corollary 9.3 does not already exclude. By the eleventh paper's Remark 3.23(c), the conjecture and the crux for ball systems together exclude oblivious linear-time transposers. Corollary 9.3(b) needs only the crux for cone systems at the same radius, which asks less, since ball systems dominate cone systems [4, Remark 7.6(a)]. What changes is the expected shape of a construction. The eleventh paper read a counterexample to the conjecture, a code that merges deep lineages along a pattern without cheap cuts, as the mechanism an oblivious construction would need [7, Remark 3.23(d)]. No such counterexample exists. By Section 9.3, what a construction needs is codes of rate near 1 on its own cone relations.

The genie of Theorem T also removes the bounded-entry hypotheses from the eighth paper's corollary on low adaptivity, as Remark 9.5 sketched, and from the eleventh paper's Theorem SE, given a native coding bound. For an input X and a scale $w \geq 1$, the *skeleton* $\sigma_w(X)$ of the run records its halting time and, at every earlier time, the blocks of width w that hold the heads and the output count; the *class* of a skeleton is the set of inputs with that skeleton [7, Definition 4.1]. At $w = 1$ the skeleton records the positions of all heads and the output count before the halting time; it is a function of the trace, and its class is a union of the classes of inputs that follow one trace. For a set Γ of steps, the *cone relation* of a run maps each input position q to the input blocks read at the ends of the Γ-free paths of the dependency DAG of the coarse work walks that start at the output time of $\mathrm{tpos}(q)$ [7, Lemma 4.7]. The *native bound* for a machine M at scale w and depth D, with runtime constant c and at a set of widths, says the following. For every $\delta > 0$ there is d_0 such that for $d \geq d_0$, every width b in the set and $L = d^2 b$, every binary storage code for the cone relation has

at most $2^{(\frac{1}{2}+\delta)L}$ words. Here the cone relation is that of the run of M on any input of length L before its halting time $T \le c(L+1)$, for any set Γ of full depth at most D, and with any output times in their output blocks. At $w = 1$ the output times are the append times and this is the native bound for the frozen traces of M. The eighth paper's update to its Corollaries F′ and F″ assumes it uniformly over the machines with k work tapes, at the widths $b(d)$ [4, Section 6.2], which implies it for each such machine. Like the crux, it is open: the thirteenth paper's codes live on relations with no known tape realization [8].

Corollary 9.7 (Corollary L: low adaptivity)**.** Lean: `NativeBound`, `se_class_native`, `se_native`, `fSecond_native_class`, `fSecond_native` [`LowAdaptivity.lean`]

Let M be a machine with k work tapes and $c > 0$, and let β_w be its block width at scale w, the number of bits of a block value [7, Definition 4.4]. Fix a set of widths, for instance all widths or $b = b(d)$ only, and let d and b below range over it.

(a) Let $w \ge 1$ and $\eta > 0$, and suppose that the native bound for M holds at scale w and at the depth D of Corollary 9.3 for $\beta = \beta_w$, $C = \max(c, 1)$ and $\varepsilon = \eta/2$. There is d_0 such that for $d \ge d_0$ and every $b \ge 1$, no skeleton class at scale w with at least $2^{(\frac{1}{2}+\eta)L}$ members is transposed by M within $c(L+1)$ steps. In particular, if M transposes every input within $c(L+1)$ steps, then $H(\sigma_w(X)) > (\frac{1}{2} - \eta)L$ for a uniformly random input X.

(b) Let $\eta < \frac{1}{2}$, and suppose that the native bound for the frozen traces of M holds at the depth D of Corollary 9.3 for $\beta = \beta_1$, $C = \max(c, 1)$ and $\varepsilon = (\frac{1}{2} - \eta)/2$. There is d_0 such that for $d \ge d_0$ and every $b \ge 1$, no skeleton class at scale 1 with at least $2^{(1-\eta)L}$ members is transposed by M within $c(L+1)$ steps. In particular, if M transposes every input within $c(L+1)$ steps, then no trace is followed by $2^{(1-\eta)L}$ inputs, and $H(\sigma_1(X)) > \eta L$, so its trace has entropy above ηL.

Outline of the formal proof. (`se_class_native`) The proof is that of the eleventh paper's Theorem SE [7, Theorem 4.13] (`MergeDepth.thm_SE_class`). A member of a large class gives a genie on the coarse work walks of its run. The revealed block values take at most $2^{\beta_w|\Gamma|}$ values, so some fibre W of the class has at least $2^{(\frac{1}{2}+\eta-\varepsilon)L}$ elements. W is a storage code for the relation of the class (`MergeDepth.blockMachine_isStorageCode_fibre`), which lies inside the cone relation (`MergeDepth.classRel_subset_coneRel`). That proof used bounded entry only for the genie, through the eighth paper's Theorem F (`MergeDepth.skeleton_genie`), and Question 9.1 only to bound W; the native bound is needed only for the runs of M and at the widths of the conclusion. Here the genie is that of Theorem T (`genie_cost`) for the coarse work walks of the member, which are walks (`MergeDepth.isWalk_skel`); it transfers to the skeleton by `MergeDepth.isDepPath_congr`, and its cost is $\beta_w|\Gamma| \le \varepsilon L$. The native bound at $\delta = \eta/4$ bounds $\log_2|W|$ by $(\frac{1}{2} + \eta/4)L$, which contradicts the lower bound $(\frac{1}{2} + \eta/2)L$. (`se_native`) If $H(\sigma_w(X)) \le (\frac{1}{2} - \eta)L$, some skeleton class has at least $2^{(\frac{1}{2}+\eta)L}$ members (`MergeDepth.exists_large_fibre`). (`fSecond_native_class`, `fSecond_native`) Part (b) is part (a) at $w = 1$, with $\frac{1}{2} - \eta$ in place of η; a trace class lies in a skeleton class at scale 1, and $\sigma_1(X)$ is a function of the trace, so its entropy bounds that of the trace from below. □

Part (b) is the eighth paper's Corollary F″ [4, Corollary 6.4] without its hypothesis (ii), that likely traces have bounded entry, with the native bound in place of Question 9.1 as that paper's update allows, and at any set of widths at which the native bound holds. Part (a) removes the bounded-entry hypothesis from the eleventh paper's Theorem SE in the same way. Hypothesis (iii) of Corollary F″, on the entropy of the whole trace, is still not robust [4, Section 6.2].

9.3 What a linear-time transposer would need

The eighth paper summarized the interface between the two halves of Theorem D: an oblivious linear-time transposer must either defeat cheap depth reduction or carry, on its own cone relation, a binary storage code of rate at least $1-\varepsilon$, a relation that is paired-distance separated after a paid trimming [4, Section 9], [10, Theorem 3.3 and Corollary 3.8]. Corollary 9.3(a) removes the first alternative. For every $\varepsilon \in (0,1]$, an oblivious transposer with k work tapes, value width β and running time $C(L+1)$ has a genie of cost at most εL and full depth less than the D of Corollary 9.3; its revealed word is an index code with at most εL bits for the side information of its own cones, and some fibre is a storage code of rate at least $1-\varepsilon$ on its cone relation [5, Theorem 4.10 and Remark 4.9(b)]. An oblivious linear-time transposer therefore exists only if, for every $\varepsilon > 0$, such codes exist on the cone relations of linear-time schedules at depth D. The coding side is the whole question.

Earlier expectations about constructions change accordingly. The fourth paper expected that an oblivious design whose heads are not all home-based would need, if the crux holds, a depth-robust dependency structure, with the deep-nesting pair as the template [5, Section 7]. The eighth paper required unbounded nesting under the crux at the depths of its Corollary F, and located any depth-robustness of a schedule on few cells [4, Section 9]. Now no family of schedules is depth-robust, and under the crux at the depth of Corollary 9.3 no oblivious linear-time transposer exists at all, whatever its head motion.

Theorem T does not decide (T). It says nothing about the crux, which is open; the counterexamples to Question 9.1 have relations of astronomically large degree with no known tape realization, and their codes are not shown to have rate near 1, as a transposer would need [8]. And it reaches adaptive machines only through single traces, as in Remark 9.5: the index-code argument needs a schedule shared by many inputs.

10 Open problems

Theorem 7.5 settles depth reduction for every bounded number of walks and every bounded number of stacks. The questions it leaves are quantitative, algorithmic, or about other layouts; the first item is (T) itself.

1. *Decide (T).* Construct a machine that transposes $d \times d$ matrices with $b(d)$-bit entries in time $O(L)$, or prove that none exists. The first paper's frozen checker accepts either answer [6]. By Section 9, depth reduction is no longer an obstacle on the way to a lower bound for oblivious machines, at the level of schedules or of codes (Corollary 9.6): what remains is the coding side. To exclude the oblivious machines with k work tapes, value width at most β and running time $C(L+1)$, it suffices to prove $\mathrm{Crux}(\mathcal{S}_{k,C,\beta,D}, \rho)$ for a single $\varepsilon \in (0,1]$, at the depth D of Corollary 9.3 and some rate $\rho > \varepsilon$ (Remark 9.4), and to exclude all oblivious linear-time transposers, to do so for all k, β and C. The native half bound would give the crux at rate $\frac{1}{2}$ at every depth (Remark 9.5); for machines with a linear data path the linear crux suffices, which the eighth paper states as a question of relative rigidity [4, Problem 10.6]. Conversely, an oblivious linear-time transposer must carry, for every ε, codes of rate at least $1-\varepsilon$ on its own cone relations at that depth (Section 9.3). Adaptive machines are reached only through single traces: given the native bound, those whose traces carry fewer than $L/2$ bits are excluded (Corollary 9.7).

2. *The order of the depth.* With the parameters of Definition 7.1, the depth $D(k,\varepsilon) = K^P - 1$ satisfies $\log\log D(k,\varepsilon) = O(k^2 \log(k/\varepsilon))$ for $\varepsilon \le \frac{1}{2}$: the number of digits is $P = (O(k^2/\varepsilon))^{4k^2+1}$,

since $Q = O(k^2/\varepsilon)$ and $j = O(Q^{4k^2}\log(k/\varepsilon))$, and the radix satisfies $\log K = O(k^2/\varepsilon + k^2\log(k/\varepsilon))$.

Problem 10.1 (The true depth)**.** *What is the smallest $D(k,\varepsilon)$ for which* TCP(k) *holds at ε? Is it bounded by $2^{\mathrm{poly}(k/\varepsilon)}$, or even by* poly$(k/\varepsilon)$*?*

The doubly exponential bound comes from Lemma D (Lemma 5.7), whose tail decays at the rate $1 - Q^{-n^2}$ per block of n steps: a block stops the front process only when all of its at most n^2 fresh bits are broken. A rate of $1 - Q^{-O(n)}$ per block would lower the exponent $4k^2$ in P to $O(k)$, and so give $\log\log D(k,\varepsilon) = O(k\log(k/\varepsilon))$; a rate bounded away from 1 would give $P = O(k^2\log(k/\varepsilon))$ and, since $\log K = O(k^2/\varepsilon + \log P)$, a depth $2^{\mathrm{poly}(k/\varepsilon)}$. Heuristically, a better rate is plausible, since a front process advances only while some head has an unbroken candidate, and each head offers at most one candidate per value.

3. *Other layouts.* Theorem 7.5 covers DAGs with a spine whose arcs form a bounded number of stacks. Bounded queue number is the natural next class: expanders with bounded queue number exist [12], but expansion is not depth-robustness in the sense of TCP, since the spine is free and only arcs are counted.

   Problem 10.2 (Queues)**.** *Is every family of DAGs with a spine whose arcs form a bounded number of queues depth-reducible?*

4. *Explicit cuts.* The proof is an averaging argument over the random choices (r,ω,ξ) (Proposition 2.7), and the label of one choice is easy to evaluate. Finding a good choice deterministically, and in near-linear time, is open; the number of choices is $h_0K^PQ^\iota$, where ι, the number of pairs of a head and one of its frames, grows with T.

A Index of Lean declarations

Every declaration of this paper's library cited in the text, grouped by the Lean module that contains it, relative to the namespace `BoundedStackDepth`. All of them depend only on the axioms `propext`, `Classical.choice` and `Quot.sound` (Appendix B). The declarations of other libraries named in the text are checked in those libraries: `BoundedNesting.AvoidPath`, `BoundedNesting.Fam.card_piFinset_coord`, `BoundedNesting.Fam.card_wrap_le`, `BoundedNesting.IsHop`, `BoundedNesting.IsLowerHop`, `BoundedNesting.IsLowerHop.neg`, `BoundedNesting.IsOwner`, `BoundedNesting.IsUpperHop`, `BoundedNesting.IsWalk`, `BoundedNesting.SR`, `BoundedNesting.SRat`, `BoundedNesting.StackDag`, `BoundedNesting.TCP`, `BoundedNesting.TCPat`, `BoundedNesting.TCPnp`, `BoundedNesting.TCPnpAt`, `BoundedNesting.above_of_step`, `BoundedNesting.allHops`, `BoundedNesting.arcCount`, `BoundedNesting.depth_le_of_label`, `BoundedNesting.descentsOf`, `BoundedNesting.digit_le_of_no_wrap`, `BoundedNesting.digit_lt_of_no_wrap`, `BoundedNesting.hopOwner`, `BoundedNesting.hopOwner_spec`, `BoundedNesting.hopPairs`, `BoundedNesting.hop_start_unique`, `BoundedNesting.hop_unique`, `BoundedNesting.hops`, `BoundedNesting.labelCut_depth`, `BoundedNesting.labelCut_depth_walkDag`, `BoundedNesting.srAt_of_tcpnpAt`, `BoundedNesting.sr_of_tcpnp`, `BoundedNesting.sw`, `BoundedNesting.tcpAt_of_srAt`, `BoundedNesting.tcp_of_sr`, `BoundedNesting.tcp_of_tcpnp_two_mul`, `BoundedNesting.tcpnp_of_tcp`, `BoundedNesting.upper_no_cross`, `BoundedNesting.violatedOf`, `BoundedNesting.walkDag_isStack` in the eighth paper's library [4] and `MergeDepth.LinCrux`,

MergeDepth.LinDataPath.rigidity_machine, MergeDepth.Pruning.field_isIndexCode, MergeDepth.Trace.work_isWalk, MergeDepth.ballSystem, MergeDepth.blockMachine_isStorageCode_fibre, MergeDepth.classRel_subset_coneRel, MergeDepth.coverClass, MergeDepth.coverHops, MergeDepth.deepHops, MergeDepth.dnpStep_pm, MergeDepth.dnpWalk, MergeDepth.exists_large_fibre, MergeDepth.headWalk, MergeDepth.hopError, MergeDepth.isDepPath_congr, MergeDepth.isDepPath_of_paths, MergeDepth.isWalk_skel, MergeDepth.obliviousTransposer_of_transposes, MergeDepth.skeleton_genie, MergeDepth.thm_SE_class in the eleventh paper's library [7].

BoundedStackDepth.Accounting card_aligned_spanning_lt, card_ground_spanning_lt, card_spanning_near_lt, extent_le_owner.

BoundedStackDepth.Assembly exists_cut_of_avg, labelCutSize.

BoundedStackDepth.BandSums band_succ, sum_band_add_pow, sum_band_sub_eq.

BoundedStackDepth.Basic band, flr, grp.

BoundedStackDepth.Budgets FrameSystem.flr_le_of_lastStint, FrameSystem.flr_le_of_rise, FrameSystem.lastStint_unique, FrameSystem.lastStints, FrameSystem.rise_unique, FrameSystem.rises, FrameSystem.runMax_mono_of_segStart_eq, FrameSystem.segScale_eq, FrameSystem.sum_lastStints_le, FrameSystem.sum_offsets_top_floor_le, FrameSystem.sum_rises_le, FrameSystem.sum_stdWeight_le, FrameSystem.sum_two_pow_flr_le.

BoundedStackDepth.Choice le_natCeil_add_one_mul, one_sub_pow_le_inv, one_sub_pow_natCeil_le_inv, trunc_term_le.

BoundedStackDepth.Composition EntryIdx, card_seeds_le, dispBot, dispTop, onStack_of_cand, oneCandidate_disp, seeds.

BoundedStackDepth.Conjectures ConjAprime, ConjAsecond, IsCoverLabel, ThetaServesLevel, conjAsecond_of_conjAprime, coverCost, coverCost_periodize_le, doublyServedCount, exists_label, headHopLevelIs_unique, not_conjAprime, not_conjAsecond, periodize, sum_classError_le, sum_headPrev_eq, sum_hopError_le, unserved_mul_le.

BoundedStackDepth.CostModel Aligned, Ground, card_lost_le, hopCost, hopCost_le_of_bands.

BoundedStackDepth.CutDescents card_descents_le, card_eventTimes_le, eventTimes, globalCount_sub_le, shown_eq_of_not_eventTime, sum_card_fiber_le, sum_card_wrapTimes_le, wrapTimes.

BoundedStackDepth.CutHops Charged, card_chargedSet_le, card_exposed_hop_le, card_truncSet_le_bad, card_violated_le_cut, card_wrap_hop_le, chargedSet, exposedSet, globalCount_sub_le_events, lostSet, popsAtZero_headFrames, sum_card_eq_sum_card_filter, sum_card_exposedSet_le, sum_card_wrapSet_le, truncSet, views_eq_of_no_event, wrapSet.

BoundedStackDepth.Decomposition card_upperHops_add_card_tops_le, eq_of_mem_ladder, exists_first_visit, exists_ladder_cell, exists_ladder_outer, exists_last_visit, height, image_cell_ladder, ladder, ladder_above, ladder_cell_iff, ladder_eq_of_cell, ladder_first_visit, ladder_last_visit, ladder_lt, ladder_nested, lowerHops, mem_ladder_hopOwner, sum_height_eq, top_ne_end, top_ne_start, top_spec, tops, upperHops, upperHops_neg, upperHops_union_lowerHops.

BoundedStackDepth.DeepNesting dnpPair, dnpPair_isWalk, dnp_cut.

BoundedStackDepth.Display Disp, OneCandidate, dispVals, shownSet.

BoundedStackDepth.DisplayTail Running, Running.anti, cands, card_cands_le, card_newCands_le, card_running_le, card_shownSet_le, card_shownSet_tail, card_ucands_add_le, card_ucands_succ_lt, card_zeroTops_le, disp_top, exists_front_eq_top, exists_mem_newCands, exists_newCands_unbroken, front, frontStep, front_add_of_not_running, front_congr, isRevealing_newCands, lt_frontStep, newCands, shownSet_subset, zeroTops.

BoundedStackDepth.DisplayView Reach, aboveVals, card_some_broken_le, disp_of_cand, mem_dispVals_iff_reach, shown_above_eq, visPairs.

BoundedStackDepth.Entries FrameSystem.IsEntry, FrameSystem.entryBot, FrameSystem.entryTop, FrameSystem.entryTop_le_entryBot, FrameSystem.entry_candidate_unique, FrameSystem.headGroup, FrameSystem.innerEntries, FrameSystem.parent, FrameSystem.parent_within.

BoundedStackDepth.EventWeight band_le_self, card_segStarts_le, le_flr, lt_of_lastStint, lt_of_rise, runMax_le_self, sum_segStarts_weight_le, sum_turns_weight_le.

BoundedStackDepth.Events beta_le_segScale, decGroup, eventCount, eventCount_lt_decGroup, eventCount_mono, owner_turn, segEventCount, segStarts, turnCount.

BoundedStackDepth.Excursions exc_laminar, exc_range, exc_structure, exists_framesOf_nested, exists_switch, framesOf, framesOf_bounds, framesOf_disjoint, framesOf_laminar_of_nested, frames_laminar, ladder_time_not_mem_framesOf, not_lt_ladD_lt, not_lt_ladL_lt, top_eq_of_end, top_eq_of_mem, top_eq_of_start, upperFrames, upper_laminar.

BoundedStackDepth.Exposure card_exposed_le, exposed_of_ne, mem_aboveVals_iff, mem_visPairs_iff, views_eq.

BoundedStackDepth.FrameDynamics FrameSystem.Restart, FrameSystem.runMax_frozen, FrameSystem.runMax_le_succ, FrameSystem.runMax_of_restart, FrameSystem.runMax_start, FrameSystem.runMax_succ_of_not_own, FrameSystem.runMax_succ_of_own, FrameSystem.segStart_of_restart, FrameSystem.segStart_start, FrameSystem.segStart_succ.

BoundedStackDepth.FrameSystem FrameSystem, FrameSystem.Own, FrameSystem.Own.unique, FrameSystem.band_dist_home_le_beta, FrameSystem.beta, FrameSystem.exists_own, FrameSystem.home, FrameSystem.inner, FrameSystem.nested_of_contains, FrameSystem.runMax, FrameSystem.segStart, FrameSystem.segStart_eq.

BoundedStackDepth.FrameWeight FrameSystem.segMax, FrameSystem.segOf, FrameSystem.segTimes, FrameSystem.segments, FrameSystem.sum_segWeight_le, sup_add_one_le_card.

BoundedStackDepth.FrameWeightU card_framesOf_le, card_levels, card_suspendFrames_le, exists_outer_subframe, own_frame_end, own_step, segMax_add_one_le, sum_segWeight_upperFrames_le, x_frame_ends, x_hangFrame_fst.

BoundedStackDepth.FramesU base, descFrame, exc, hangFrame, innerDescents, ladD, ladL, ladderHop, levels, passages, restartPts, suspendFrames.

BoundedStackDepth.Global card_violated_le_heads, decGroup_mem_shown, globalCount, globalCount_lt_of_head, globalCount_mono, headFrames, headGroup_mem_shown, headOf, headWalk, headWalk_down, headWalk_up, hops_subset_heads, labelOf, labelOf_lt, shown, trunc_or_above_or_wrap_head, zero_mem_shown.

BoundedStackDepth.Grid card_dvd_add_le_one, card_flr_grp_succ_le, card_grp_ne_le, card_lt_flr_grp_le, card_one_le_grp_le, exists_floor_of_grp_lt, flr_grp_injOn, flr_grp_le, grp_mono, lt_flr_grp_succ, sum_two_pow_flr_grp_le.

BoundedStackDepth.HopData FrameMicro, Lost, bandP, bandU, cstar, homeBand, ownerScale, pushMax, segScale, sum_card_lost_le_sum_cstar, sum_cstar_le_of_ladders.

BoundedStackDepth.Label card_wrap_offsets_le, digits, digits_lt, label, label_le_label_of_eq, label_lt, label_lt_label, lexRank, lexRank_le_of_le, lexRank_lt_of_lt, lexRank_lt_pow, lt_of_topRank_lt, shown_ne_or_wrap, toLex_lt_of_prefix, topRank, topRank_eq_of_above, topRank_injOn, topRank_lt_card, topRank_lt_of_lt, trunc_or_above_or_wrap.

BoundedStackDepth.LadderBands Free, beta_ladD_bounds, beta_ladL_bounds, beta_last_eq_zero, beta_le_band_height, excStart_spec, frameAt_walk, frame_cases, free_of_own, home_mem_exc, inner_fst_cases, inner_snd_cases, ladL_lt_segStart_ladD, own_of_free, own_of_le_level, passage_spec, restart_inside, runMax_last_le_one, runMax_le_height, segStart_eq_excStart, segStart_inner_mem, segStart_inner_mem_Icc, segStart_ladD, segStart_ladD_of_not, struct_exc, x_excStart.

BoundedStackDepth.LowAdaptivity NativeBound, fSecond_native, fSecond_native_class, se_class_native, se_native.

BoundedStackDepth.Machines ConjHVStar, ConjHVStarLin, Crux, CruxOn, DepthTCP, IsIndexCode.bits, IsIndexCode.mono, SCone, ballSystem_indexCode, ballSystem_minrank, coneI_subset_depCone, coneSystem, coneSystem_subset_ballSystem, crux_excludes, depCone, depthTCP, exists_indexCode, genie, genieDepth, genie_cost, genie_of_trace, hvStar, hvStar_linear, linCrux_excludes.

BoundedStackDepth.Main exists_avgLabel, main, parH, parH_spec, parJ, parJ_spec, parK, parK_spec, parP, parQ, parQ_spec, sr, sum_bad_le, sum_labelCutSize_le, sum_violated_head_le, sum_weight_events_head_le, tcp, tcpAt_direct.

BoundedStackDepth.PerExcursion band_eq_of_mem_Ico, clog_eq_of_lt_of_le, ladder_sum_le_six, sum_range_le_six_of_ladder, sum_range_two_pow_sub_band, sum_sub_band_two_pow_clog, uband.

BoundedStackDepth.Prominence active_unique, card_active_le, card_spanning_hops_lt, colL, colL_le_of, colR, colR_le_of, extent, lt_colL, lt_colR, peak, prom, sub_le_peak, sub_le_prom, sum_peak_le, sum_prom_le, sum_prom_le_two_mul, sum_valley_le, valley.

BoundedStackDepth.Revealing IsRevealing, RevAgree, card_blocks_le, card_le_pow_mul_card_block, card_le_pow_mul_card_zero.

BoundedStackDepth.Statement MainStatement, sr_of_main.

BoundedStackDepth.TheoremA cstar_exc_le, cstar_ladder_le, ownerScale_ladderHop, sum_ladder_cstar_le, thmA_lower, thmA_upper.

BoundedStackDepth.TopProminence height_le_prom, prom_le_of_left, prom_le_of_right, valley_eq_zero_of_step_up.

BoundedStackDepth.Totals card_events_head_le, card_frames_head_le, card_upperHops_head_le, sum_descents_le, sum_events_head_le, weight_events_head_le.

BoundedStackDepth.Truncation badTimes, card_bad_le, card_trunc_le_bad.

BoundedStackDepth.ViewInvariance FrameSystem.InnermostCommon, FrameSystem.LastStintAt, FrameSystem.LocalPost, FrameSystem.LocalPre, FrameSystem.PopsAtZero, FrameSystem.Returns, FrameSystem.RiseAt, FrameSystem.above_post_iff, FrameSystem.above_pre_iff, FrameSystem.entryBot_popped, FrameSystem.entryBot_post_le_iff, FrameSystem.entryBot_pre_le_iff, FrameSystem.entryBot_pushed, FrameSystem.entryTop_eq_zero, FrameSystem.exists_innermostCommon, FrameSystem.headAbove, FrameSystem.headGroup_eq_zero_of_push, FrameSystem.headGroup_pred_eq_zero, FrameSystem.headVisible, FrameSystem.localPost_iff, FrameSystem.localPre_iff, FrameSystem.localPre_iff_localPost, FrameSystem.max_entryBot_eq, FrameSystem.not_returns_of_restart, FrameSystem.parent_eq_of_popped, FrameSystem.parent_of_pushed, FrameSystem.popped_child, FrameSystem.pushed_child, FrameSystem.stackEntries, FrameSystem.view_interval, FrameSystem.view_step, FrameSystem.within_of_contains_not.

BoundedStackDepth.Walks IsHop.upper_or_lower, isOwner_hopOwner_of_upper.

BoundedStackDepth.Wrap flr_grp_succ_le, flr_le_iff, sum_hops_events_le, sum_two_pow_flr_succ_le.

BoundedStackDepth.WrapGlobal card_kept_spanning_le, card_wrap_le_sum, flr_mono, segEventCount_sub, sum_kept_family_le, sum_ten_two_pow_flr_le, turnCount_sub.

B Building and verifying the formalization

The Lean project is the directory `lean/` next to this paper. It uses Lean and Mathlib, both at version v4.34.1 (`lean-toolchain`, `lakefile.toml`, `lake-manifest.json`), and requires the libraries of the eighth paper [4] and, for the corollaries, of the eleventh paper [7] through relative paths; those libraries require the earlier libraries of the series in the same way. Keep the repository's directory layout, so that these sibling directories stay in place.

Build. With `elan` installed, run in `lean/`:

```
lake exe cache get && lake build
```

The first command downloads the compiled Mathlib; the second compiles the library and the parts of the earlier libraries that it uses. Before the first build, fetch the second paper's pinned external dependency as its README describes (`python3 verification/fetch_openai_metric.py`, run in that paper's directory).

Axioms. The file `PaperAxioms.lean`, generated from the LaTeX sources of this paper by `tools/lean_index.py`, runs `#print axioms` on every declaration of this library named here (Appendix A):

```
lake env lean PaperAxioms.lean
```

Every declaration depends at most on `propext`, `Classical.choice` and `Quot.sound`. No Lean file of this library uses `sorry`, `admit` or `native_decide`, declares an `axiom`, or raises `maxHeartbeats`. The script regenerates both the index of Appendix A and `PaperAxioms.lean`; with the option `-index-only` it writes only the index.

Where things are. The library `lean/BoundedStackDepth/` has six layers, each a group of modules.

- *Statements and foundations:* `Statement.lean` (the main statement and its corollary for stack DAGs), `Assembly.lean` (the probabilistic method for label cuts), `Basic.lean`, `Grid.lean`, `BandSums.lean`, `Choice.lean`, `Walks.lean`, `Prominence.lean` and `TopProminence.lean`.
- *One walk* (Section 3): `Decomposition.lean`, `Excursions.lean`, `FramesU.lean`, `FrameSystem.lean`, `FrameDynamics.lean`, `FrameWeight.lean`, `FrameWeightU.lean`, `LadderBands.lean`, `CostModel.lean`, `HopData.lean`, `Events.lean`, `PerExcursion.lean` and `TheoremA.lean`.
- *The label* (Section 4): `Label.lean` and `Global.lean`.
- *Displays* (Section 5): `Display.lean`, `Revealing.lean`, `DisplayTail.lean`, `DisplayView.lean`, `Entries.lean`, `Composition.lean`, `ViewInvariance.lean` and `Exposure.lean`.
- *Accounting and the proof* (Sections 6 and 7): `Budgets.lean`, `EventWeight.lean`, `Accounting.lean`, `Truncation.lean`, `Wrap.lean`, `WrapGlobal.lean`, `CutDescents.lean`, `CutHops.lean`, `Totals.lean` and `Main.lean`.
- *Consequences* (Section 9): `DeepNesting.lean` and `Conjectures.lean`, Corollary N; `Machines.lean`, Corollaries O and HV; `LowAdaptivity.lean`, Corollary L.

The root module `BoundedStackDepth.lean` imports all of them.

References

[1] Joël Alwen, Jeremiah Blocki, and Krzysztof Pietrzak. Depth-robust graphs and their cumulative memory complexity. In *Advances in Cryptology – EUROCRYPT 2017*, volume 10212 of *Lecture Notes in Computer Science*, pages 3–32. Springer, 2017. doi:10.1007/978-3-319-56617-7_1.

[2] Michael A. Bender and Martín Farach-Colton. The level ancestor problem simplified. *Theoretical Computer Science*, 321(1):5–12, 2004. doi:10.1016/j.tcs.2003.05.002.

[3] Jean Bourgain and Amir Yehudayoff. Expansion in $\mathrm{SL}_2(\mathbb{R})$ and monotone expanders. *Geometric and Functional Analysis*, 23(1):1–41, 2013. doi:10.1007/s00039-012-0200-9.

[4] Hans Brende and Claude Opus 5.5 (Anthropic). Bounded nesting and depth reduction for matrix transposition on tapes. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 1699424e236a72d14644f7e2d81c4da7467ba8a4. URL: https://github.com/HansBrende/papers/tree/1699424e236a72d14644f7e2d81c4da7467ba8a4/Bounded-Nesting-and-Depth-Reduction-for-Matrix-Transposition-on-Tapes.

[5] Hans Brende and Claude Opus 5.5 (Anthropic). Curved motion, depth reduction, and storage codes for matrix transposition on tapes. Manuscript and partial Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 1699424e236a72d14644f7e2d81c4da7467ba8a4. URL: https://github.com/HansBrende/papers/tree/1699424e236a72d14644f7e2d81c4da7467ba8a4/Curved-Motion-Depth-Reduction-and-Storage-Codes-for-Matrix-Transposition-on-Tapes.

[6] Hans Brende and Claude Opus 5.5 (Anthropic). Matrix transposition and the tape coding conjecture. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 1699424e236a72d14644f7e2d81c4da7467ba8a4. URL: https://github.com/HansBrende/papers/tree/1699424e236a72d14644f7e2d81c4da7467ba8a4/Matrix-Transposition-and-the-Tape-Coding-Conjecture.

[7] Hans Brende and Claude Opus 5.5 (Anthropic). Merge depth, input steering, and the coding core of matrix transposition on tapes. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 1699424e236a72d14644f7e2d81c4da7467ba8a4. URL: https://github.com/HansBrende/papers/tree/1699424e236a72d14644f7e2d81c4da7467ba8a4/Merge-Depth-Input-Steering-and-the-Coding-Core-of-Matrix-Transposition-on-Tapes.

[8] Hans Brende and Claude Opus 5.5 (Anthropic). Storage codes above half rate under paired-distance separation. Manuscript and Lean formalization, repository HansBrende/papers, November 2026. Pinned revision 1699424e236a72d14644f7e2d81c4da7467ba8a4. URL: https://github.com/HansBrende/papers/tree/1699424e236a72d14644f7e2d81c4da7467ba8a4/Storage-Codes-above-Half-Rate-under-Paired-Distance-Separation.

[9] Hans Brende and Gemini 4 Argon (Google DeepMind). Directed reciprocity and deep-nesting bounds for matrix transposition on tapes. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 1699424e236a72d14644f7e2d81c4da7467ba8a4. URL: https://github.com/HansBrende/papers/tree/1699424e236a72d14644f7e2d81c4da7467ba8a4/Directed-Reciprocity-and-Deep-Nesting-Bounds-for-Matrix-Transposition-on-Tapes.

[10] Hans Brende and GPT-6 Astra (OpenAI). Directed recovery structures in tape computations. Manuscript and partial Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 1699424e236a72d14644f7e2d81c4da7467ba8a4. URL: https://github.com/HansBrende/papers/tree/1699424e236a72d14644f7e2d81c4da7467ba8a4/Directed-Recovery-Structures-in-Tape-Computations.

[11] Hans Brende and GPT-6 Astra (OpenAI). TCP and information-cheap depth reduction. Memo to the fourth paper of the series, written proofs, repository HansBrende/papers, October 2026. Pinned revision 1699424e236a72d14644f7e2d81c4da7467ba8a4. URL: https://github.com/HansBrende/papers/tree/1699424e236a72d14644f7e2d81c4da7467ba8a4/Curved-Motion-Depth-Reduction-and-Storage-Codes-for-Matrix-Transposition-on-Tapes/memo-information-depth.

[12] Vida Dujmović, Anastasios Sidiropoulos, and David R. Wood. Layouts of expander graphs. *Chicago Journal of Theoretical Computer Science*, 2016:Article 1, 2016. Preprint arXiv:1501.05020.

[13] Zeev Dvir and Avi Wigderson. Monotone expanders: Constructions and applications. *Theory of Computing*, 6:291–308, 2010. Article 12. doi:10.4086/toc.2010.v006a012.

[14] Paul Erdős, Ronald L. Graham, and Endre Szemerédi. On sparse graphs with dense long paths. *Computers and Mathematics with Applications*, 1(3–4):365–369, 1975. doi:10.1016/0898-1221(75)90037-1.

[15] Zvi Galil, Ravi Kannan, and Endre Szemerédi. On 3-pushdown graphs with large separators. *Combinatorica*, 9(1):9–19, 1989. doi:10.1007/BF02122679.

[16] Zvi Galil, Ravi Kannan, and Endre Szemerédi. On nontrivial separators for k-page graphs and simulations by nondeterministic one-tape Turing machines. *Journal of Computer and System Sciences*, 38(1):134–149, 1989. Preliminary version in Proc. 18th ACM STOC, 1986, pp. 39–49. doi:10.1016/0022-0000(89)90036-6.

[17] David Harvey and Joris van der Hoeven. Integer multiplication is at least as hard as matrix transposition. arXiv:2503.22848; also in Proc. FOCS 2025, pp. 1366–1381 (numbering as in arXiv), 2025. URL: https://arxiv.org/abs/2503.22848.

[18] John Hopcroft, Wolfgang Paul, and Leslie Valiant. On time versus space. *Journal of the ACM*, 24(2):332–337, 1977. doi:10.1145/322003.322015.

[19] Wolfgang Paul and Rüdiger Reischuk. On alternation II. A graph theoretic approach to determinism versus nondeterminism. *Acta Informatica*, 14(4):391–403, 1980. doi:10.1007/BF00286494.

[20] Wolfgang J. Paul, Nicholas Pippenger, Endre Szemerédi, and William T. Trotter. On determinism versus non-determinism and related problems. In *24th Annual Symposium on Foundations of Computer Science (FOCS 1983)*, pages 429–438, 1983.

[21] Rahul Santhanam. On separators, segregators and time versus space. In *Proceedings of the 16th Annual IEEE Conference on Computational Complexity*, pages 286–294, 2001. doi:10.1109/CCC.2001.933895.

[22] Georg Schnitger. On depth-reduction and grates. In *24th Annual Symposium on Foundations of Computer Science (FOCS 1983)*, pages 323–328, 1983. doi:10.1109/SFCS.1983.38.

[23] Leslie G. Valiant. Graph-theoretic arguments in low-level complexity. In *Mathematical Foundations of Computer Science (MFCS)*, volume 53 of *Lecture Notes in Computer Science*, pages 162–176. Springer, 1977.